

# 408 计算机网络学习笔记

当前进度：已阅读并整理《计算机网络-第1-3章-体系结构至数据链路层.pdf》第 1-19 页，**第一章 计算机网络体系结构** 已全部完成。PDF 第 20 页开始进入第二章 “物理层”，不属于本 act 处理范围。

下次任务：本章已完成，无需继续阅读本章内容；进入 AnyWorkflow 结束流程。

## 第一章 计算机网络体系结构

### 1.1 计算机网络概述

#### 1.1.1 计算机网络的概念

计算机网络可以理解为：把许多分散、自治的计算机系统，通过通信线路和通信设备连接起来，再依靠功能完善的软件与协议，实现**资源共享**和**信息传递**的系统。

把这个定义拆开看，重点有三层：

##### 1. 分散、自治

网络中的计算机不是一台大型计算机的终端，而是各自能独立运行的系统。每台主机可以独立完成计算任务，也可以通过网络与其他主机协作。

##### 2. 通信线路与通信设备连接

网络由节点和链路组成。节点可以是计算机、集线器、交换机、路由器等；链路是连接节点的通信线路，例如双绞线、光纤、无线信道等。

##### 3. 软件与协议保证通信规则一致

只把设备连起来并不等于能通信。网络通信必须依赖协议，协议规定了数据如何封装、如何寻址、如何转发、如何确认和差错处理等规则。

这里要特别区分 **internet** 和 **Internet**：

| 名称                | 含义                             | 是否专指全球互联网 | 是否必须使用 TCP/IP |
|-------------------|--------------------------------|-----------|---------------|
| internet（互连网）     | 多个计算机网络互连形成的通用概念               | 否         | 不一定           |
| Internet（互联网/因特网） | 当前全球最大的、开放的、由众多网络和路由器互连而成的特定网络 | 是         | 是             |

因此，**internet 是普通名词，Internet 是专有名词**。考试中如果问“互连网”和“互联网”的区别，核心就在于是否专指全球互联网，以及是否必须采用 TCP/IP 协议族。

普通用户通常通过互联网服务提供商 ISP 接入 Internet。ISP 从互联网管理机构获得大量 IP 地址，并拥有通信线路、路由器等联网设备。用户向 ISP 缴费后获得 IP 地址的使用权，再通过 ISP 接入互联网。

#### 1.1.2 计算机网络的组成

计算机网络可以从不同角度划分组成，常见有三种视角。

| 划分角度   | 组成        | 理解重点                            |
|--------|-----------|---------------------------------|
| 从组成部分看 | 硬件、软件、协议  | 协议是网络的核心，硬件提供连接基础，软件提供应用和资源共享能力 |
| 从工作方式看 | 边缘部分、核心部分 | 边缘部分面向用户，核心部分负责连通与交换            |
| 从功能组成看 | 通信子网、资源子网 | 通信子网负责“传数据”，资源子网负责“提供资源”        |

从组成部分看：

- **硬件**包括主机、通信链路、交换设备等。主机如台式机、笔记本、平板、智能手机；通信链路如双绞线、光纤；交换设备如交换机、路由器。
- **软件**包括系统软件和应用软件，例如资源共享相关的软件、电子邮件程序、FTP 程序、聊天程序等。
- **协议**规定通信双方必须遵守的规则，是网络能正常工作的核心。

从工作方式看，互联网可分为：

### 1. 边缘部分

由连接在互联网上、供用户直接使用的主机组成。用户在这里进行通信、数据传输和资源共享。

### 2. 核心部分

由大量网络以及连接这些网络的路由器组成，为边缘部分提供连通性与交换服务。

可以用下面的结构理解：

互联网

├ 边缘部分：主机、用户设备、应用程序

└ 核心部分：网络、路由器、交换设备，负责转发和连通

从功能组成看：

- **通信子网**：由传输介质、通信设备和相关协议组成，负责数据传输、交换、控制和存储，重点解决“如何把数据送到对方”。
- **资源子网**：由提供资源共享功能的设备和软件组成，向用户提供硬件资源、软件资源和数据资源，重点解决“共享什么资源”。

这组概念容易混淆，可以这样记：**通信子网偏底层、偏传输；资源子网偏上层、偏应用资源。**

## 1.1.3 计算机网络的功能

计算机网络的功能很多，其中最核心的是以下五类。

### 1. 数据通信

这是计算机网络最基本、最重要的功能，用来实现联网计算机之间的信息传输。文件传输、电子邮件、网页访问、即时通信等都依赖数据通信。

### 2. 资源共享

共享对象可以是硬件、软件或数据。资源共享让不同主机上的资源互通有无，提高资源利用率。例如共享打印机、共享文件、共享数据库等。

### 3. 分布式处理

当某台计算机负载过重时，可以把部分复杂任务分配给网络中的其他计算机处理，从而提高整个系统的效率。

### 4. 提高可靠性

网络中的多台计算机可以互为备份。某些设备或系统出现故障时，其他节点仍可继续提供服务，从而提升可靠性。

### 5. 负载均衡

将任务较均衡地分配给网络中的各台计算机，避免某些计算机过载而另一些计算机空闲。负载均衡在服务器集群、云计算、分布式系统中非常常见。

除上述功能外，计算机网络还支持电子化办公、远程教育、娱乐服务等应用。这些属于功能的具体体现，复习时重点应放在前五类基础功能上。

## 1.1.4 电路交换、报文交换与分组交换

网络核心部分中起关键作用的是路由器，它通过对收到的数据进行存储和转发来实现分组交换。要理解分组交换，需要先比较三种典型数据交换方式：**电路交换**、**报文交换**、**分组交换**。

考点追踪：电路交换的时延分析（2025）

**电路交换**最典型的例子是传统电话网。它的本质是：通信前先为双方建立一条专用的端到端物理通路，通信过程中双方独占这条通路，通信结束后释放连接。

电路交换分为三个阶段：

建立连接 → 传输数据 → 释放连接

电路交换的特点可以概括为：**先占线，再通信，最后释放**。

优点：

1. **通信时延小**：连接建立后，数据基本沿专用通路直达，传输效率高。
2. **有序传输**：数据按发送顺序到达，不存在失序问题。
3. **没有冲突**：通信双方独占物理信道，不与其他通信共享该通路。
4. **实时性强**：物理通路一旦建立，双方可随时通信。

缺点：

1. **建立连接时间长**：真正传输前要先建立专用通路。
2. **线路利用率低**：通路被双方独占，即使暂时不传数据，也不能给其他用户使用。计算机通信常具有突发性，因此用电路交换时通信线路大部分时间可能空闲，利用率可能不足 **10%**，甚至低于 **1%**。
3. **灵活性差**：中间节点或链路发生故障时，往往需要重新建立连接。
4. **难以进行差错控制**：中间节点不具备存储和检错能力，难以及时发现并纠正传输错误。

从时延角度理解，电路交换的总时间大致由建立连接、数据传输、传播和释放连接组成：

$$T_{\text{电路交换}} \approx t_{\text{建立}} + \frac{L}{R} + t_{\text{传播}} + t_{\text{释放}}$$

其中， $L$  表示数据量， $R$  表示传输速率。若传输的是连续大量数据，且建立连接时间相对不大，电路交换可能有较短的完成时间；但若数据通信很突发，线路利用率会很低。

考点追踪：报文交换的时延分析（2013，2025）

**报文交换**不需要事先建立专用通路。源主机把要发送的数据连同源地址、目的地址等控制信息封装成一个完整报文，报文在网络中逐跳转发。每个交换节点先完整接收并存储整个报文，再根据转发表选择下一跳继续转发，这称为**存储转发**。

报文交换的工作方式可以理解为：

完整报文到达节点 → 节点缓存整个报文 → 查表选择下一跳 → 转发完整报文

优点：

- 1. **无建立和释放连接的时延**：通信前不需要建立专用线路，用户可随时发送报文。
- 2. **线路分配灵活**：交换节点可以根据当前链路状态选择较优路径；某条链路故障时，还可能动态选择其他路径。
- 3. **线路利用率高**：报文只在实际传输时占用链路，不长期独占线路资源。
- 4. **支持差错控制**：交换节点可以对缓存中的报文进行差错检测。

缺点：

- 1. **存储转发时延高**：节点必须完整接收整个报文后才能转发，报文越长，等待时间越明显。
- 2. **缓存开销大**：报文长度没有固定上限，交换节点需要准备较大缓存。
- 3. **错误处理效率低**：报文越长，出错概率越大；一旦出错，重传整个报文的代价较大。

若一个报文长度为  $L$ ，经过  $k$  段链路，且每段链路速率近似为  $R$ ，忽略排队和处理时延，则报文交换由于每一跳都要完整存储转发，发送时延可粗略理解为：

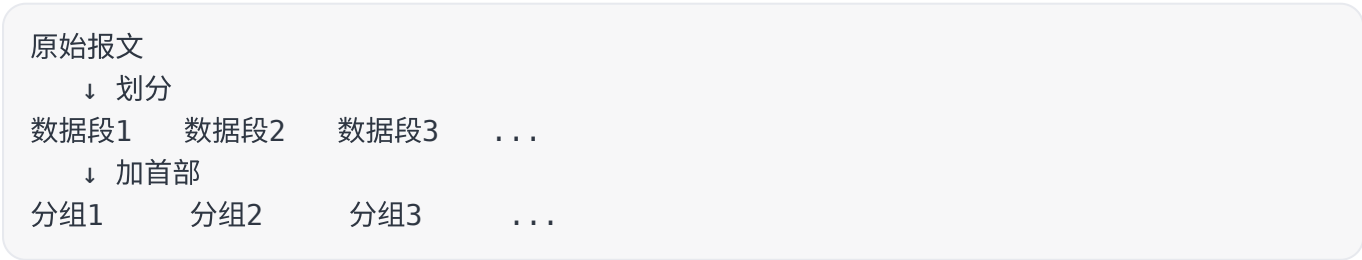
$$T_{\text{报文交换}} \approx k \cdot \frac{L}{R} + t_{\text{传播总和}}$$

这也是为什么报文交换在大报文场景下延迟较高。

考点追踪：分组交换的时延分析（2010，2013，2023，2025）

**分组交换**同样采用存储转发思想，但它把长报文划分为若干较短、较固定长度的数据段，并在每个数据段前加上首部，形成一个个分组。首部中通常包含源地址、目的地址、分组编号等控制信息。

可以这样理解报文到分组的过程：



分组交换的基本过程是：源主机将分组发送到分组交换网，路由器收到分组后先缓存，然后读取首部，根据转发表转发给下一跳。经过多个路由器后，分组最终到达目的主机。

与报文交换相比，分组交换的重要改进是：**不再让一个很长的报文整体占用缓存和链路，而是让较短的分组逐个转发**。这样可以形成流水线式传输，即前一个分组到达下一跳时，后一个分组可以同时在上一段链路上传输，从而缩短整体传输时间。

优点：

- 1. **便于存储管理，转发开销小**：分组长度较固定，缓存区大小可预设，管理更高效。
- 2. **传输效率高**：分组可逐个发送，形成流水线操作，减少整体传输时间。
- 3. **降低出错概率与重传代价**：分组较短，出错概率相对降低；即使出错，也只需重传出错分组，而不是重传整个报文。
- 4. **线路利用率很高**：多个用户的分组可以共享链路资源，适合突发式数据通信。

缺点：

- 1. **仍存在存储转发时延**：每个分组到达路由器后仍需缓存、查表和转发。
- 2. **需要额外控制信息**：每个分组都要加首部，信息量会增加约 **5% - 10%**，控制复杂度也提高。
- 3. **可能失序、丢失或重复**：采用数据报服务时，不同分组可能走不同路径，目的主机需要编号重排，处理较复杂。若采用虚电路服务，可以避免失序，但需要经历建立连接、数据传输、释放连接三个阶段。

若报文被划分为  $m$  个分组，每个分组长度近似为  $P$ ，经过  $k$  段链路，且链路速率相同为  $R$ ，忽略排队、处理和首部开销时，分组交换的流水线传输时间可粗略理解为：

$$T_{\text{分组交换}} \approx (k + m - 1) \cdot \frac{P}{R} + t_{\text{传播总和}}$$

这个式子的含义是：第一个分组需要经过  $k$  段链路，后续  $m - 1$  个分组可以流水线跟上，所以整体不是简单的  $k \cdot m \cdot \frac{P}{R}$ 。这正是分组交换比报文交换更适合计算机网络的关键原因之一。

三种交换方式的核心差异如下。

| 特性          | 电路交换      | 报文交换       | 分组交换             |
|-------------|-----------|------------|------------------|
| 基本思想        | 建立专用通路后通信 | 整个报文逐跳存储转发 | 报文切成分组后逐跳存储转发    |
| 完成传输所需时间    | 最少        | 最多         | 较少               |
| 存储转发时延      | 无         | 高          | 较低               |
| 通信前是否需要建立连接 | 是         | 否          | 否（数据报方式）；虚电路方式需要 |
| 节点缓存开销      | 无         | 高          | 低                |
| 是否支持差错控制    | 不支持       | 支持         | 支持               |
| 数据是否有序到达    | 是         | 是          | 不一定，数据报方式可能失序    |
| 是否需要额外控制信息  | 否         | 是          | 是，且占比较大          |

| 特性      | 电路交换 | 报文交换 | 分组交换 |
|---------|------|------|------|
| 线路分配灵活性 | 不灵活  | 灵活   | 非常灵活 |
| 线路利用率   | 低    | 高    | 非常高  |

复习时要抓住一句话：**电路交换适合连续、大量、实时的通信；报文交换减少了专线独占但整报文存储转发太慢；分组交换继承存储转发优点，又用短分组和流水线降低时延，是现代计算机网络的核心交换方式。**

1.1.5 计算机网络的分类

计算机网络可以从多个角度分类。复习时不要死记每一种名称，而要抓住分类依据：**范围看覆盖多大，技术看怎样传，拓扑看怎样连，用户看谁能用，介质看用什么传。**

1. 按分布范围分类

| 类型    | 英文缩写 | 覆盖范围与特点                              | 典型理解               |
|-------|------|--------------------------------------|--------------------|
| 广域网   | WAN  | 用于长距离通信，覆盖范围通常为几十到数千千米，常用于构建互联网的骨干部分 | 跨城市、跨地区、跨国家的网络     |
| 城域网   | MAN  | 覆盖一个城市或几个街区，直径约为 5-50 km             | 城市级网络，常采用以太网技术     |
| 局域网   | LAN  | 覆盖范围较小，一般为几十米到几千米                    | 校园网、实验室网络、楼宇网络     |
| 个人区域网 | PAN  | 围绕个人工作或活动区域，用无线技术连接个人设备              | 手机、平板、耳机、智能手表等设备互联 |

广域网的重点是**远距离、大范围、骨干连接**。它常把不同地区的局域网或城域网连接起来，节点交换机之间通过高速链路互联，通信容量较大。

城域网覆盖一个城市范围，范围比局域网大、比广域网小。现在城域网常采用以太网技术，因此有时也会被放入局域网相关讨论中。

局域网通常把较小范围内的主机连接起来。传统局域网常采用广播技术，但现代以太网中交换机已经非常普遍，因此不能简单认为“局域网一定只用广播”。

个人区域网更贴近个人设备之间的连接，若使用无线技术，也称无线个人区域网（WPAN）。

2. 按传输技术分类

| 类型    | 核心思想               | 典型场景                | 易错点                       |
|-------|--------------------|---------------------|---------------------------|
| 广播式网络 | 所有联网计算机共享一个公共通信信道  | 传统总线以太网、无线局域网、卫星通信网 | 所有主机都能“听到”，但只有目的地址匹配的主机接收 |
| 点对点网络 | 每条物理链路连接一对计算机或通信设备 | 广域网中的路由器互连          | 两台主机不直接相连时，分组要经中间节点存储转发   |

广播式网络中，一台主机发送报文分组后，其他主机都能收到该分组，但每台主机要检查目的地址，只有匹配的主机才真正接收。无线网络天然具有广播特征，因此很适理解这一类网络。



点对点网络没有共享广播信道，数据通常沿一条由多个中间节点组成的路径转发。广域网多采用点对点方式，因为它更适合大范围互连和路由选择。

3. 按拓扑结构分类

网络拓扑结构指网络中节点与通信链路之间的几何排列关系。常见拓扑包括总线形、星形、环形和网状。

| 拓扑结构 | 连接方式               | 优点                | 缺点                  | 常见范围     |
|------|--------------------|-------------------|---------------------|----------|
| 总线形  | 所有主机连接到一条共享总线      | 建网容易，增减节点方便，节省线路  | 总线故障会影响全网，重负载时通信效率低 | 多用于早期局域网 |
| 星形   | 终端通过独立线路连接到中央设备    | 便于集中控制与管理         | 中央设备故障会影响全网，线路成本较高  | 现代以太网常见  |
| 环形   | 节点连接成闭合环路，信号沿环单向传输 | 结构清晰，可用单环或双环提高可靠性 | 环路故障会影响通信，维护较复杂     | 令牌环等局域网  |
| 网状   | 节点之间存在多条路径相连       | 可靠性高，容错能力强        | 控制复杂，线路成本高          | 广域网较常见   |

可以用一句话记忆：**总线省线但怕总线坏，星形好管但怕中心坏，环形按环传，网状可靠但贵且复杂。**

上述基本拓扑还能相互组合，形成更复杂的混合网络。现实网络通常不是单一拓扑，而是多种拓扑叠加的结果。

4. 按使用者分类

| 类型  | 含义                              | 例子               |
|-----|---------------------------------|------------------|
| 公用网 | 由电信运营商建设，面向公众开放，只要遵守规定并缴纳费用即可使用 | 公众互联网接入网络        |
| 专用网 | 由特定单位为满足自身业务需求建设，只供内部使用，不向外提供服务 | 铁路、电力、军队等部门的专用网络 |

公用网强调“公众可接入”，专用网强调“内部专用”。考试中若问二者区别，重点看**服务对象和建设目的**。

5. 按传输介质分类

按传输介质可分为有线网络和无线网络。

| 类型   | 传输介质         | 例子               |
|------|--------------|------------------|
| 有线网络 | 双绞线、同轴电缆、光纤等 | 有线以太网、光纤接入网络     |
| 无线网络 | 无线电波、微波、红外等  | 蓝牙、Wi-Fi、无线电通信网络 |

有线网络通常稳定、抗干扰能力强；无线网络部署灵活、移动性好，但更容易受到干扰，且共享信道带来的竞争问题更明显。

1.1.6 计算机网络的性能指标

计算机网络的性能指标用于衡量网络性能。这里是第一章的高频重点，尤其是**速率、带宽、吞吐量、时延、时延带宽积、往返时延、信道利用率**。其中时延相关计算最容易出题。

1. 速率

速率指连接到网络上的节点在数字信道上传送数据的速率，也称数据传输速率、数据率或比特率。单位是 bit/s，常写作 b/s 或 bps。

常用单位换算如下：

| 单位   | 含义                   |
|------|----------------------|
| kb/s | $10^3 \text{ bit/s}$ |
| Mb/s | $10^6 \text{ bit/s}$ |
| Gb/s | $10^9 \text{ bit/s}$ |

注意区分数据通信中的单位和存储容量中的单位：

注意：描述数据量时， $K$ 、 $M$ 、 $G$  常用 2 的幂次表示，例如  $1\text{KB} = 2^{10}\text{B}$ ；描述速率时， $k$ 、 $M$ 、 $G$  常用 10 的幂次表示，例如  $1\text{kb/s} = 10^3\text{b/s}$ 。通常前者使用大写  $K$ ，后者使用小写  $k$ ，但其他前缀多为大写。

这里最常见的易错点是 **B 和 b 的区别**：

type = 单位区分  
1 B = 8 b  
1000B 的分组长度 = 8000b

如果题目给的是字节  $B$ ，计算发送时延时一定要先换成比特  $b$ 。

2. 带宽

带宽在不同语境中含义不同：

| 语境    | 带宽含义                      | 单位    |
|-------|---------------------------|-------|
| 通信领域  | 通信线路允许通过的信号频率范围           | Hz    |
| 计算机网络 | 网络通信线路所能传送数据的能力，即最高数据传输速率 | bit/s |

在计算机网络中，带宽可以粗略理解为“这条链路理论上每秒最多能发送多少比特”。但实际可用带宽还会受到中间链路、交换设备、接口速率等因素限制，整体传输能力往往取决于路径上的瓶颈链路。

例如一条路径中某段链路只有  $80\text{Mb/s}$ ，即使其他链路是  $100\text{Mb/s}$  或更高，端到端传输速率也可能被  $80\text{Mb/s}$  限制。

考点追踪：分组交换的吞吐量分析（2024）



3. 吞吐量

吞吐量指单位时间内通过某个网络、信道或接口的实际数据量。它通常用于衡量真实网络能达到的数据传输能力。

速率、带宽、吞吐量三者的关系可以这样区分：

| 指标  | 更偏向           | 理解方式            |
|-----|---------------|-----------------|
| 速率  | 某节点或链路发送数据的速度 | “发送端往链路里推数据有多快” |
| 带宽  | 链路的理论能力上限     | “这条路最多能承载多快”    |
| 吞吐量 | 实际通过的数据量      | “最后真正跑出来有多快”    |

吞吐量通常不会超过瓶颈带宽。考试中若问端到端吞吐量，往往要找路径中的最小链路速率；若有多个流共享链路，还要结合共享关系分析。

4. 时延

时延指数据从网络或链路的一端传送到另一端所需要的总时间。它由四部分组成：

total\_delay = 发送时延 + 传播时延 + 处理时延 + 排队时延

用公式表示为：

$$D_{总} = D_{发送} + D_{传播} + D_{处理} + D_{排队}$$
$$D_{发送} = \frac{\text{分组长度}}{\text{发送速率}}$$
$$D_{传播} = \frac{\text{信道长度}}{\text{电磁波在信道上的传播速率}}$$

考点追踪：网络时延的分析（2010，2013，2023，2025）

四种时延的本质如下。

| 时延类型        | 由什么决定          | 核心理解                    |
|-------------|----------------|-------------------------|
| 发送时延，也称传输时延 | 分组长度、发送速率      | 把分组所有比特“推入链路”所需时间       |
| 传播时延        | 链路长度、传播速度      | 一个比特在介质中从一端传播到另一端所需时间   |
| 处理时延        | 节点处理能力、协议处理复杂度 | 路由器解析首部、差错检测、查表转发等处理时间  |
| 排队时延        | 网络拥塞程度、队列长度    | 分组在输入队列或输出队列中等待处理或转发的时间 |

注意：发送时延和传播时延必须区分。发送时延取决于**分组长度和发送速率**；传播时延取决于**链路距离和传输介质**。发送时延不是“数据在路上走的时间”，传播时延也不是“把数据发出去的时间”。

处理时延和排队时延通常较难精确计算，所以考试中一般默认不考虑，除非题目明确给出。

结合 PDF 中的典型分组交换网例子：主机 A 经过一个路由器向主机 B 发送长度为 1000B 的分组。链路 1 速率为 100Mb/s，链路 2 速率为 80Mb/s。

若忽略传播时延、处理时延和排队时延，则：

$$\begin{aligned}D_{\text{发送1}} &= \frac{1000\text{B} \times 8}{100\text{Mb/s}} = 0.08\text{ms} \\D_{\text{发送2}} &= \frac{1000\text{B} \times 8}{80\text{Mb/s}} = 0.1\text{ms} \\D_{\text{总}} &= 0.08\text{ms} + 0.1\text{ms} = 0.18\text{ms}\end{aligned}$$

若链路 1 和链路 2 的传播时延分别为 0.01ms 和 0.05ms，则：

$$D_{\text{总}} = 0.01\text{ms} + 0.08\text{ms} + 0.05\text{ms} + 0.1\text{ms} = 0.24\text{ms}$$

若还要考虑路由器的处理时延和排队时延，且二者合计为 0.04ms，则：

$$D_{\text{总}} = 0.01\text{ms} + 0.08\text{ms} + 0.04\text{ms} + 0.05\text{ms} + 0.1\text{ms} = 0.28\text{ms}$$

这个例子说明：**分组经过多段链路时，每一段链路上的发送时延都要算；传播时延、处理时延、排队时延是否计算，要看题目是否给出。**

## 5. 时延带宽积

时延带宽积指发送端发送的第一个比特即将到达接收端时，发送端已经发出但尚未到达接收端的最大比特数。公式为：

$$\text{时延带宽积} = \text{传播时延} \times \text{信道带宽}$$

可以把链路想象成一条空心管道：

管道长度：传播时延  
管道横截面积：链路带宽  
管道容量：时延带宽积

所以，时延带宽积衡量的是“链路中最多能容纳多少正在传播的比特”。长距离、高带宽链路的时延带宽积很大，意味着在确认到达之前，链路上可能已经装入了大量比特。

## 6. 往返时延 RTT

往返时延 RTT 指从发送端发出一个短分组开始，到发送端收到接收端返回的确认信号为止所经历的总时间。

RTT 通常包括：

- 发送端到接收端的传播时延；
- 接收端处理确认所需的处理时延；
- 接收端到发送端的返回传播时延；
- 在互联网环境中，还可能包括中间节点的处理时延和排队时延。

需要注意：**RTT 不包括数据分组本身的发送时延**。若忽略确认分组的发送时延，可近似理解为：

$$RTT \approx \text{正向传播时延} + \text{接收端处理时延} + \text{反向传播时延}$$

RTT 是分析可靠传输、超时重传、网络交互延迟时非常重要的指标。对于网页访问、远程登录、在线游戏等交互式应用，RTT 往往比单纯的链路带宽更影响体验。

## 7. 信道利用率

信道利用率指信道有数据通过的时间占总时间的百分比：

$$\text{信道利用率} = \frac{\text{有效数据通过的时间}}{\text{有效数据通过的时间} + \text{无数据通过的时间}}$$

信道利用率不是越高越好。若信道利用率过低，说明大量网络资源空闲，会造成浪费；若信道利用率过高，分组在路由器中的排队时延会明显增加，容易引发网络拥塞。

可以把信道利用率类比为公路车流量：车太少，路资源被浪费；车太多，虽然“路面利用率高”，但拥堵会让每辆车通过的时间变长。网络也是同样道理，利用率要适中，而不是盲目追求接近 100%。

本小节最需要掌握的公式汇总如下：

$$\begin{aligned} D_{\text{发送}} &= \frac{\text{分组长度}}{\text{发送速率}} \\ D_{\text{传播}} &= \frac{\text{信道长度}}{\text{传播速率}} \\ D_{\text{总}} &= D_{\text{发送}} + D_{\text{传播}} + D_{\text{处理}} + D_{\text{排队}} \\ \text{时延带宽积} &= \text{传播时延} \times \text{信道带宽} \\ \text{信道利用率} &= \frac{\text{有效数据通过的时间}}{\text{总时间}} \end{aligned}$$

复习时可以按“能发多快、实际过多少、路上等多久、管道能装多少、来回一趟多久、信道忙不忙”这一条线串起来：速率和带宽描述能力，吞吐量描述实际效果，时延描述等待时间，时延带宽积描述链路容量，RTT 描述往返交互耗时，信道利用率描述资源使用程度。

# 1.2 计算机网络体系结构与参考模型

## 1.2.1 计算机网络分层结构

计算机网络是一个非常复杂的系统。若把所有问题混在一起处理，设计、实现、维护和排错都会极其困难。因此，计算机网络采用**分层设计思想**：把庞大复杂的问题拆成若干较小、相对独立的局部问题，每一层只关注自己负责的功能。

考点追踪：网络体系结构的概念（2010）

计算机网络各层及其协议的集合称为**网络体系结构**。更准确地说，体系结构描述的是网络及其部件应完成哪些功能，它强调的是“应该做什么”；而这些功能最终由什么硬件、什么软件、什么程序代码实现，属于**实现问题**，强调的是“具体怎么做”。

这一区分很重要：

| 概念   | 关注点             | 抽象程度 | 例子                 |
|------|-----------------|------|--------------------|
| 体系结构 | 网络各层应完成的功能与层间关系 | 抽象   | OSI 七层模型、TCP/IP 模型 |
| 实现   | 用具体硬件和软件把功能做出来  | 具体   | 网卡驱动、路由器转发表、协议栈代码  |

分层设计的基本原则可以概括为五点：

- 1. **每层实现相对独立的功能**，这样可以降低整个系统的复杂度。
- 2. **层与层之间接口清晰、简洁**，层间依赖越少，越便于理解和维护。
- 3. **每层功能的定义独立于具体实现方法**，同一层可以采用不同技术实现。
- 4. **下层向上层提供服务**，上层使用下层服务，而不关心下层内部细节。
- 5. **整个分层结构应促进标准化工作**，便于不同厂家、不同系统之间互联。

在分层结构中，有几个概念需要重点掌握。

| 概念    | 含义                        | 记忆方法       |
|-------|---------------------------|------------|
| 实体    | 第 $n$ 层中能够发送或接收信息的硬件或软件进程 | 真正“干活”的对象  |
| 对等层   | 不同机器中处于同一层的层次             | 逻辑上像在同一层对话 |
| 对等实体  | 不同机器中同一层的实体               | 协议作用的对象    |
| 服务提供者 | 第 $n$ 层实体                 | 给上一层提供服务   |
| 服务用户  | 第 $n + 1$ 层实体             | 使用下一层提供的服务 |

分层通信表面上看是“同层之间通信”，例如发送方传输层似乎直接与接收方传输层通信；但实际上，同层实体之间并不存在一条真实的直接信道。真实数据流是沿发送端从高层逐层向下，再经过物理链路传输，最后在接收端逐层向上交付。

可以用下面的逻辑理解数据在层间的传递：

发送端应用数据  
↓ 第 5 层加首部

```

5 - PDU
  ↓ 第 4 层加首部
4 - PDU
  ↓ 第 3 层加首部
3 - PDU
  ↓ 第 2 层加首部/尾部
2 - PDU
  ↓ 第 1 层转换成比特流
物理链路上传输

```

这里涉及三个高频术语：

| 术语     | 英文缩写 | 含义                               |
|--------|------|----------------------------------|
| 协议数据单元 | PDU  | 对等层之间传送的数据单位，由服务数据单元和协议控制信息组成    |
| 服务数据单元 | SDU  | 相邻层之间交换的数据单位，是上层交给下层的“有效载荷”      |
| 协议控制信息 | PCI  | 为完成协议控制而加入的信息，如首部、尾部、地址、编号、校验信息等 |

三者之间的基本关系是：

$$n\text{-SDU} + n\text{-PCI} = n\text{-PDU} = (n - 1)\text{-SDU}$$

这个公式的含义是：第  $n + 1$  层交给第  $n$  层的数据，在第  $n$  层看来是  $n\text{-SDU}$ ；第  $n$  层加上自己的控制信息  $n\text{-PCI}$  后，形成  $n\text{-PDU}$ ；当这个整体继续交给下一层时，又变成了第  $n - 1$  层的  $(n - 1)\text{-SDU}$ 。

各层 PDU 有自己的常用名称，复习时要记住几个典型名称：

| 层次    | PDU 常用名称         |
|-------|------------------|
| 物理层   | 比特流              |
| 数据链路层 | 帧                |
| 网络层   | IP 分组，常称分组或数据报   |
| 传输层   | TCP 报文段或 UDP 数据报 |

分层结构的含义还可以进一步概括为：

1. 第  $n$  层不仅依靠第  $n - 1$  层的服务实现自身功能，还要向第  $n + 1$  层提供服务。第  $n$  层提供的服务，本质上是第  $n$  层及其以下所有层共同提供服务的总和。
2. 最低层只提供服务，是整个层次结构的基础；最高层面向用户提供服务。
3. 上层只能通过相邻层接口使用下层服务，不能跨层调用。
4. 通信时，对等层在逻辑上表现为直接交换信息，但真实数据传输必须逐层封装、逐层传递。

理解分层结构时最容易犯的错误是把“逻辑通信”和“实际传输”混为一谈。正确理解是：协议发生在对等层之间，是逻辑上的；服务发生在相邻层之间，是垂直方向上的；数据实际传输要沿层次结构上下移动。

1.2.2 计算机网络协议、服务、接口的概念

这一小节的核心是区分三个词：**协议、服务、接口**。它们经常一起出现，但含义完全不同。

1. 协议

协议是控制两个对等实体进行通信的规则集合。也就是说，协议是**水平的**，它规定同一层的发送方和接收方应如何交换信息。

协议由三部分组成：

考点追踪：同步的概念（2020）

| 组成          | 含义                         | 例子                      |
|-------------|----------------------------|-------------------------|
| 语法          | 规定数据与控制信息的格式               | TCP 报文段首部有哪些字段、字段长度如何安排 |
| 语义          | 规定需要发出什么控制信息、完成什么动作、做出什么响应 | TCP 连接建立时收到 SYN 后应如何响应  |
| 同步，也称<br>时序 | 规定各种操作的条件、顺序和事件发生的先后关系     | TCP 三次握手中各报文的先后顺序       |

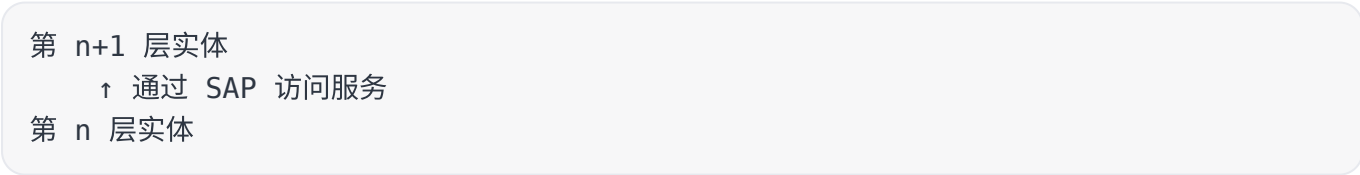
语法解决“长什么样”，语义解决“表示什么、要做什么”，同步解决“什么时候做、按什么顺序做”。

需要注意：协议只存在于**对等实体**之间。比如，主机 A 的传输层与主机 B 的传输层之间可以说有传输层协议；但主机 A 的传输层与主机 B 的网络层不属于对等层，二者之间没有协议关系。

2. 接口

接口是同一节点内部相邻两层交换信息的逻辑边界。更具体地说，服务通过**服务访问点** SAP 提供给上层。

可以这样理解 SAP：



每层只能和紧邻的上、下层定义接口，不允许任意跨层定义接口。第  $n$  层的 SAP，就是第  $n + 1$  层访问第  $n$  层服务的入口。

3. 服务

服务是下层向紧邻上层提供的功能调用，是**垂直的**。对等实体在协议控制下协同工作，使本层能够向上一层提供服务；而本层要实现这些协议，又必须依赖下一层提供的服务。

协议和服务有本质区别：

| 对比项 | 协议 | 服务 |
|-----|----|----|
| 方向  | 水平 | 垂直 |



| 对比项     | 协议         | 服务         |
|---------|------------|------------|
| 发生对象    | 对等层实体之间    | 相邻层之间      |
| 关注内容    | 通信规则       | 下层向上层提供的功能 |
| 对上层是否透明 | 协议细节对上层不可见 | 上层只感知服务本身  |

因此，不要把协议和服务混为一谈。只有协议正确实现，才能保证向上一层提供服务；但上一层用户只需要知道服务能用，不需要知道下层协议的具体细节。

计算机网络提供的服务可从三个角度分类。

### (1) 面向连接服务与无连接服务

| 类型     | 工作方式                     | 特点                 | 典型理解       |
|--------|--------------------------|--------------------|------------|
| 面向连接服务 | 通信前建立连接，通信结束后释放连接        | 过程包括连接建立、数据传输、连接释放 | 类似先打通电话再讲话 |
| 无连接服务  | 通信前不建立连接，发送方直接发送带目的地址的分组 | 网络动态选择路径，分组独立转发    | 类似寄一封封信    |

面向连接服务要求通信双方先建立连接并分配必要资源，例如缓存区；这有利于保障通信过程的可靠性。无连接服务不提前建立连接，每个分组独立携带目的地址并独立转发，通常称为“尽最大努力交付”，本身属于不可靠服务。

### (2) 可靠服务与不可靠服务

| 类型    | 含义                                | 可靠性保障来源       |
|-------|-----------------------------------|---------------|
| 可靠服务  | 网络通过检错、纠错、确认、重传等机制，保证数据正确、完整、有序到达 | 网络层次内部提供保障    |
| 不可靠服务 | 网络只尽力传送，不保证数据正确、完整、有序到达           | 需要高层协议或应用自行保障 |

如果底层网络只提供不可靠服务，那么可靠性可以由更高层来补足。例如接收方校验数据完整性，出错时通知发送方重传，从而在高层机制中把不可靠传输转化为可靠传输。

### (3) 有应答服务与无应答服务

| 类型    | 含义                       | 例子                            |
|-------|--------------------------|-------------------------------|
| 有应答服务 | 接收方收到数据后，由传输系统自动向发送方返回应答 | 文件传输常用确认来保证数据完整               |
| 无应答服务 | 接收方不自动返回应答，如需确认需由高层实现    | WWW 服务中，客户端收到网页后通常不向服务器自动发送应答 |

这里的“应答”是传输系统自动产生的确认，不等于应用层业务上的回复。复习时要把它和“用户是否回消息”区分开。

### 1.2.3 OSI 参考模型和 TCP/IP 模型

这一小节是第一章的重点之一。复习时应先把 OSI 七层的顺序、每层传输单位、每层核心功能记牢，再比较 OSI 模型与 TCP/IP 模型的差异。

#### 1. OSI 参考模型

考点追踪：OSI 低三层的网络设备及其功能（2016）

国际标准化组织 ISO 提出的开放系统互连参考模型 OSI/RM，把网络体系结构划分为 7 层。自下而上依次为：

- 7 应用层
- 6 表示层
- 5 会话层
- 4 传输层
- 3 网络层
- 2 数据链路层
- 1 物理层

可以用一句话记忆：**物、链、网、传、会、表、应**。从发送端看，数据通常从应用层逐层向下封装；从接收端看，数据从物理层逐层向上拆封。

在 OSI 模型中，端系统通常具有完整的 7 层功能；中继系统一般只需要实现较低层功能。例如在图示结构中，中继系统通常涉及物理层、数据链路层和网络层，负责把数据沿网络路径转发。

考点追踪：OSI 参考模型的层次结构（2013、2014、2017、2019）

##### （1）物理层

物理层的传输单位是**比特**。它的功能是在物理介质上为数据端设备透明地传输原始比特流。

物理层主要研究两类问题：

1. **通信链路与节点之间如何连接**，包括接口的机械特性和电气特性。例如接口形状、尺寸、引脚数量与排列方式等。
2. **比特如何在信道上表示和传输**，包括编码含义和电气特征。例如约定某种信号  $X$  表示比特 0，发送方发出  $X$  代表 0，接收方收到  $X$  就解释为 0。

注意：双绞线、光纤、无线信道等传输介质本身不属于物理层协议范围，而是位于物理层之下。因此有时会吧物理介质形象地称为“第 0 层”。

##### （2）数据链路层

考点追踪：OSI 参考模型中数据链路层的功能（2022）

数据链路层的传输单位是**帧**。它负责在相邻节点之间建立逻辑上无差错的数据链路，把网络层交下来的分组封装成帧，并可靠地传送到相邻节点的网络层。

数据链路层的核心功能包括：

- **组帧**：把网络层分组封装成帧，明确一帧从哪里开始、到哪里结束。
- **差错控制**：检测并处理由于噪声、干扰导致的比特错误，必要时丢弃错误帧。
- **流量控制**：协调发送方与接收方速率，避免发送过快导致接收方来不及接收。
- **访问控制**：在广播式网络中，解决多个节点如何共享同一信道的问题。

数据链路层强调的是**节点到节点**通信。例如节点 A 到节点 B 之间的一段链路，它关心的是这一段链路上的帧能否正确交付。

### (3) 网络层

网络层的传输单位是**数据报或分组**。需要注意，广义上无论哪一层的协议数据单元，都可以笼统称为“分组”；但在分层名称中，网络层的 PDU 最常称为分组或数据报。

网络层的主要任务是把分组从源主机传送到目的主机，为不同主机提供通信服务。其核心功能包括：

- **路由选择**：在多条可能路径中选择合适路径，使分组能够到达目的节点。
- **拥塞控制**：当网络中大量分组导致负载过高时，采取措施缓解拥塞。
- **差错控制**：通过检错机制确保向上层提交的数据尽量无差错，无法纠正时按协议处理。
- **流量控制**：协调源主机与目的主机之间的数据收发速率。
- **网际互联**：连接不同网络，使分组能跨网络传输。

网络层既可以提供面向连接的**虚电路服务**，也可以提供无连接的**数据报服务**。在路由结构图中，节点 A 向节点 B 发送分组时，可能有多条路径可选，例如经上方节点转发，也可能经下方节点转发。网络层要利用路由算法选择合适路径，确保分组顺利到达目的节点。

### (4) 传输层

考点追踪：OSI 参考模型中传输层的功能（2009）

传输层又称运输层，负责不同主机中**进程到进程**之间的端到端通信。它提供连接管理、可靠传输、流量控制、差错控制等服务。

这里要把几个通信范围分清：

| 层次    | 通信范围      | 地址或标识侧重点 |
|-------|-----------|----------|
| 数据链路层 | 相邻节点到相邻节点 | MAC 地址   |
| 网络层   | 源主机到目的主机  | IP 地址    |
| 传输层   | 源进程到目的进程  | 端口号      |

一台主机可以同时运行多个进程，因此传输层还必须具有**复用和分用**功能：

- **复用**：多个应用进程可以同时使用传输层服务，把数据交给传输层发送。
- **分用**：传输层收到数据后，根据端口号等信息，把数据准确交付给对应的上层进程。

需要注意：OSI 参考模型中的传输层只提供**面向连接的可靠服务**。

### (5) 会话层

考点追踪：OSI 参考模型中会话层的功能（2019）

会话层负责管理不同主机上进程之间的会话，包括会话连接的建立、维护和终止。它为表示层或用户进程提供同步机制，支持在连接上有序地传输数据。

会话层的典型功能是引入**检查点**。如果通信中断，可以从最近的检查点恢复会话，而不是从头开始传输，这类似于“断点续传”的可靠性思想。

(6) 表示层

考点追踪：OSI 参考模型中表示层的功能（2013）

表示层负责解决异构系统之间信息表示不一致的问题。不同主机可能使用不同的数据格式、编码方式或数据结构，表示层通过统一的表示规则，使双方能够正确理解数据。

表示层的典型功能包括：

- **数据格式转换**，例如不同系统之间的数据结构表示转换。
- **标准编码**，例如通过 ASN.1 等抽象语法定义数据结构。
- **数据压缩**，减少传输数据量。
- **加密与解密**，提高数据传输安全性。

(7) 应用层

应用层是 OSI 参考模型的最高层，直接作为用户与网络之间的接口，为各种网络应用提供访问网络环境的手段。

应用层协议数量最多，也最贴近实际应用，例如文件传输、电子邮件、网页浏览等。由于应用类型丰富，应用层协议通常也最复杂。

到这里为止，OSI 七层可以按“底层负责传输，上层负责应用表达，中间层负责端到端与主机间通信”来理解：

| 层次    | 关键词               | 复习抓手        |
|-------|-------------------|-------------|
| 物理层   | 比特、接口、电气特性        | 把 0/1 传过去   |
| 数据链路层 | 帧、相邻节点、差错/流量/访问控制 | 把一段链路变可靠    |
| 网络层   | 分组、路由、拥塞、网际互联     | 把分组送到目的主机   |
| 传输层   | 端到端、进程、端口、复用/分用   | 把数据送到正确进程   |
| 会话层   | 会话管理、同步、检查点       | 管理一次会话过程    |
| 表示层   | 编码、格式、压缩、加密       | 解决数据表示问题    |
| 应用层   | 网络应用、用户接口         | 给具体应用提供网络服务 |

复习 OSI 模型时，最重要的不是机械背七个名字，而是能回答：某个功能属于哪一层、某种通信范围对应哪一层、某种 PDU 叫什么、某个设备大致工作在哪一层。

OSI 七层功能可以再从“任务表”角度压缩记忆。这个表特别适合考选择题或简答题，重点不是逐字背，而是抓住每层最典型的功能边界。

| OSI 层次 | 主要任务和功能                              | 复习提醒                      |
|--------|--------------------------------------|---------------------------|
| 应用层    | 提供应用与网络的接口                           | 面向具体网络应用，如文件传输、电子邮件、Web   |
| 表示层    | 数据格式转换                               | 处理编码、格式、压缩、加密等“表示问题”      |
| 会话层    | 会话管理                                 | 管理会话建立、维护、同步、恢复和释放        |
| 传输层    | 复用和分用、差错控制、流量控制、连接管理、可靠传输管理          | 端到端、进程到进程，常与端口号联系         |
| 网络层    | 路由选择、拥塞控制、网际互联、差错控制、流量控制、连接管理、可靠传输管理 | 主机到主机，核心是路由与跨网络转发         |
| 数据链路层  | 差错控制、流量控制                            | 相邻节点之间传输帧，常见功能是组帧、检错、访问控制 |
| 物理层    | 定义电路接口参数、信号含义和电气特性等                  | 只关心比特如何通过物理介质传输           |

这里容易混淆的是：传输层和网络层都可能出现差错控制、流量控制、连接管理、可靠传输等说法，但二者的**服务对象不同**。网络层面对的是主机到主机或网络到网络的分组交付；传输层面对的是主机中应用进程之间的端到端通信。

## 2. TCP/IP 模型

考点追踪：TCP/IP 模型的层次结构（2021）

TCP/IP 模型从低到高通常分为四层：**网络接口层、网际层、传输层、应用层**。它和 OSI 参考模型之间大致有如下对应关系：

OSI：      应用层   表示层   会话层   传输层   网络层   数据链路层   物理层  
TCP/IP： ┌───────────┐ 应用层 ┌───────────┐ 传输层   网际层   ┌── 网络接口层 ─┐

### （1）网络接口层

网络接口层大致对应 OSI 的物理层和数据链路层。它负责从主机或节点接收 IP 分组，并将其发送到指定的物理网络上。

需要注意：TCP/IP 模型并没有严格规定网络接口层内部的具体功能和协议细节，只要求主机能够通过某种底层协议接入网络并传送 IP 分组。因此，实际底层网络可以是以太网、无线局域网，也可以是电话网、广域网等。

### （2）网际层

考点追踪：TCP/IP 模型中网际层的功能（2011、2021）

网际层是 TCP/IP 体系结构的核心，功能与 OSI 网络层非常相似。它负责把分组发往任意网络，并为每个分组独立选择路由，但不保证分组有序到达，也不保证可靠交付。

网际层的核心协议是 **IP 协议**。IP 提供的是**无连接、不可靠**的服务，传输单位为 **IP 数据报**。这里的“不可靠”不是说一定会出错，而是说 IP 本身不承诺纠错、重传、有序交付等可靠性保证。

常见网际层协议如下：

| 协议   | 作用                        |
|------|---------------------------|
| IP   | 网际层核心协议，负责寻址、分组和路由转发      |
| IPv4 | 当前广泛使用的 IP 版本             |
| IPv6 | 下一代 IP 版本，解决地址空间等问题       |
| ARP  | 地址解析协议，用于 IP 地址与硬件地址之间的映射 |
| ICMP | 网际控制报文协议，用于差错报告和网络控制      |
| IGMP | 网际组管理协议，服务于组播管理           |

(3) 传输层

TCP/IP 模型中的传输层与 OSI 的传输层类似，负责发送端和接收端主机上对等进程之间的逻辑通信。

传输层主要有两个重要协议：

| 协议  | 全称                                   | 服务特点                    | 数据单位  |
|-----|--------------------------------------|-------------------------|-------|
| TCP | Transmission Control Protocol，传输控制协议 | 面向连接，传输前先建立连接，提供可靠交付    | 报文段   |
| UDP | User Datagram Protocol，用户数据报协议       | 无连接，不保证可靠性，只提供“尽最大努力交付” | 用户数据报 |

可以把 TCP 和 UDP 的差异理解为：TCP 把可靠性做到传输层里，适合需要可靠交付的应用；UDP 保持简单快速，适合实时性强、能容忍少量丢失或由应用层自行控制可靠性的场景。

(4) 应用层

TCP/IP 的应用层合并了 OSI 参考模型中会话层、表示层和应用层的功能，包含所有高层协议。常见应用层协议包括：

- FTP：文件传输协议；
- DNS：域名解析服务；
- HTTP：超文本传输协议；
- SMTP：电子邮件发送相关协议。

从 TCP/IP 协议族的整体关系看，**IP 是互联网的核心协议**。许多应用协议都建立在 IP 之上，即“Everything over IP”；而 IP 又可以运行在各种底层网络之上，即“IP over Everything”。这两句话体现了 TCP/IP 体系的高度通用性和灵活性，也是互联网能够发展到今天规模的重要原因。

3. TCP/IP 模型与 OSI 参考模型的比较



TCP/IP 模型与 OSI 参考模型有相似之处：

- 1. 二者都采用分层体系结构，各层功能大体相似。
- 2. 二者都基于独立协议栈的思想。
- 3. 二者都试图解决异构网络互联问题，使不同厂商、不同类型的计算机能够通信。

但二者也有显著差异。

| 对比项   | OSI 参考模型                  | TCP/IP 模型                        |
|-------|---------------------------|----------------------------------|
| 概念区分  | 精确定义了服务、协议、接口三个核心概念       | 对服务、协议、接口没有作严格区分                 |
| 层次数量  | 7 层                       | 4 层                              |
| 层次合并  | 会话层、表示层、应用层分开；物理层和数据链路层分开 | 会话层、表示层并入应用层；物理层、数据链路层合并为网络接口层   |
| 形成过程  | 先有模型，后有协议                 | 先有协议，后归纳出模型                      |
| 适用范围  | 理论上更通用，不偏向具体协议            | 更贴近 TCP/IP 协议族，不适用于所有非 TCP/IP 网络 |
| 网络层服务 | 网络层支持无连接和面向连接通信模式         | 网际层只提供无连接服务                      |
| 传输层服务 | 传输层只提供面向连接服务              | 传输层同时支持面向连接的 TCP 和无连接的 UDP       |

OSI 的优点在于理论体系清晰，特别是服务、协议、接口的区分很适合学习网络体系结构；缺点是模型过于理想化，实际实现复杂，缺乏足够的市场和商业驱动力。TCP/IP 的优点在于简洁、实用、先被广泛部署，因此最终在实际网络中占据主导地位。

为了兼顾理论和实践，学习计算机网络时常采用一种 **5 层协议体系结构**：

5 应用层  
4 传输层  
3 网络层  
2 数据链路层  
1 物理层

它可以看作把 OSI 的会话层、表示层、应用层合并为应用层，同时保留物理层和数据链路层的分离。这样既避免 OSI 七层过于复杂，又比 TCP/IP 四层更适合讲清底层功能。

考点追踪：应用层数据的逐层封装过程（2021）

使用协议栈通信时，发送方的数据并不是直接从应用层“飞到”接收方应用层，而是要在发送端逐层封装、经过物理链路传输，再在接收端逐层解封装。

基本过程如下：



发送端：应用层数据  
↓ 加应用层控制信息  
应用层 PDU  
↓ 作为传输层 SDU，加传输层控制信息  
传输层 PDU  
↓ 作为网络层 SDU，加网络层控制信息  
网络层 PDU  
↓ 作为数据链路层 SDU，加链路层首部和尾部  
数据链路层帧  
↓ 变成比特流经物理层传输  
物理链路  
↓  
接收端：按相反方向逐层拆封，最终交给应用进程

这里涉及三个常见缩写：

| 缩写  | 含义                                  | 理解                      |
|-----|-------------------------------------|-------------------------|
| SDU | Service Data Unit，服务数据单元            | 上层交给本层的数据               |
| PCI | Protocol Control Information，协议控制信息 | 本层为了实现协议而添加的控制信息，如首部、尾部 |
| PDU | Protocol Data Unit，协议数据单元           | 本层真正向对等层表达的数据单位         |

从公式上可写成：

$$PDU_n = PCI_n + SDU_n$$

其中，数据链路层常常还会在数据后面添加尾部，因此可以理解为：

帧 = 链路层首部 + 上层数据 + 链路层尾部

这个过程的核心是：**发送端层层封装，接收端层层拆封**；协议是对等层之间的逻辑规则，真实数据则沿协议栈上下移动。

# 1.3 本章小结及疑难点

## 1.3.1 为什么互联网的 IP 是无连接、不可靠的

互联网使用的 IP 协议是无连接的，因此它本身提供的是不可靠传输。这一点容易让人产生疑问：既然可靠通信很重要，为什么最初不直接把互联网的网络层设计成可靠的？

要理解这个问题，需要先看到传统电信网和计算机网络的差异。

传统电信网主要用于电话通信，普通电话机只是“哑终端”，本身几乎没有智能处理能力。因此，电信公司必须把网络本身设计得高度可靠，依靠网络基础设施保证通话质量。

而计算机网络起源于数据通信环境。以 ARPANET 为代表的早期计算机网络连接的是具有强大处理能力的主机。设计者面临两种选择：

- 1. **由网络负责可靠性**：网络内部提供确认、重传、有序交付等机制，把通信基础设施做得复杂而可靠。
- 2. **由端系统负责可靠性**：网络层只提供简单的分组转发服务，可靠性由用户主机或更高层协议补足。

互联网最终采用了第二种思路，也就是端到端可靠传输的设计哲学：网络层使用简单、无连接、不可靠的 IP 协议；传输层再通过面向连接的 TCP 协议实现端到端可靠性。

这样做有两个重要好处：

- **降低网络基础设施复杂度和成本**：路由器主要负责尽力转发，不必为每条通信维护复杂状态。
- **提高网络扩展性和灵活性**：不同应用可以根据需要选择 TCP 或 UDP，不必把所有应用都强行放进同一种可靠传输模式。

因此，“IP 不可靠”不是缺陷，而是一种设计取舍：把网络核心做简单，把复杂性更多放到端系统和高层协议中。

## 1.3.2 端到端通信和点到点通信的区别

点到点通信和端到端通信都在讲“数据从一处到另一处”，但它们的层次和关注对象不同。

从通信子网角度看，物理层、数据链路层和网络层共同负责提供主机到主机的通信服务；而传输层进一步为主机上的应用进程提供端到端通信服务。

**点到点通信**强调数据在一段通信关系中的传递。在链路层语境下，它常指相邻节点之间的传输；在通信子网整体语境下，也可理解为主机到主机之间的数据传递。它只保证数据从一台机器传到另一台机器，不关心具体是哪两个应用进程在通信。

**端到端通信**建立在点到点通信的基础上。一个端到端连接往往要经过多段点到点链路转发，最终实现不同主机上应用进程之间的通信。这里的“端”不是简单指一台机器，而更准确地说是应用进程的端口。

可以用下面的层次关系记忆：

| 通信类型  | 典型层次            | 关注对象         | 解决的问题              |
|-------|-----------------|--------------|--------------------|
| 点到点通信 | 物理层、数据链路层、网络层相关 | 节点或主机之间的数据传递 | 数据能否从这一点传到下一点或目的主机 |

| 通信类型  | 典型层次 | 关注对象        | 解决的问题            |
|-------|------|-------------|------------------|
| 端到端通信 | 传输层  | 应用进程之间的数据传递 | 数据能否交给正确主机上的正确进程 |

核心区别是：**点到点更偏机器之间、链路之间；端到端更偏应用进程之间。** 网络层把数据送到目的主机，传输层再通过端口号等信息把数据送到正确进程。

### 1.3.3 如何理解传输速率和传播速率

传输速率和传播速率是第一章时延问题中的高频易混概念。

**传输速率**指主机或路由器在数字信道上发送数据的速率，单位通常是比特每秒，即 b/s。它描述的是发送端把比特“推入链路”的快慢，也常与链路带宽联系在一起。

**传播速率**指电磁波在物理介质中向前传播的速度，单位通常是米每秒，即 m/s。它描述的是某个已经进入链路的比特沿介质向前“跑”的快慢。

二者对应的时延公式不同：

$$\begin{aligned}\text{发送时延} &= \frac{\text{数据长度}}{\text{传输速率}} = \frac{L}{R} \\ \text{传播时延} &= \frac{\text{链路长度}}{\text{传播速率}} = \frac{s}{v}\end{aligned}$$

例如，若链路的传播速率为  $2 \times 10^8 \text{ m/s}$ ，则电磁波在  $1\mu\text{s}$  内可传播：

$$2 \times 10^8 \times 10^{-6} = 200 \text{ m}$$

若链路带宽为  $1 \text{ Mb/s}$ ，则主机在  $1\mu\text{s}$  内只能向链路发送 1 比特数据。于是可以得到这样的过程：

t = 0：开始发送数据  
t =  $1\mu\text{s}$ ：第 1 比特已经沿链路传播约 200m  
t =  $2\mu\text{s}$ ：第 2 比特发出，第 1 比特到达约 400m 处  
t =  $3\mu\text{s}$ ：第 3 比特发出，第 1 比特到达约 600m 处

由此可见：链路上同一时刻能存在多少比特，取决于传输速率和传播时延的乘积；而每个比特在某一时刻处于链路的什么位置，取决于传播速率。

这个乘积也称为**带宽时延积**：

$$\text{带宽时延积} = R \cdot \frac{s}{v}$$

它表示链路中“正在飞行”的比特数量。复习时可抓住一句话：**传输速率决定往链路里塞比特的速度，传播速率决定已经塞进去的比特在链路上跑得多快。**

当前进度：已完成第2章“物理层”全部正文、例题与典型分析内容，对应 PDF 第20-32页。  
下次任务：第2章内容已完成；按 AnyWorkflow 结束流程上传最终 Markdown 产物并结束本 act。  
本章状态：已完成。

## 第二章 物理层

本章研究的是：**如何在传输介质上传输比特流**。它不关心“这段数据属于哪个网页、哪个应用、哪个文件”，而关心怎样把 0 和 1 变成可在介质上传播的电信号、光信号或无线电信号，并尽量高效、可靠地传过去。

本章可分为三块：

- 1. **通信基础**：数据、信号、码元、速率、带宽、奈奎斯特定理、香农定理、编码与调制。
- 2. **传输介质**：双绞线、同轴电缆、光纤、无线传输介质。
- 3. **物理层设备**：中继器、集线器。

复习时要把重点放在**速率计算、带宽与信噪比、奈奎斯特定理和香农定理、编码与调制、设备工作特点**上。

### 2.1 通信基础

#### 2.1.1 基本概念

##### 1. 数据、信号与码元

**数据**是信息的载体。例如文字、图像、音频、视频等都可以看作数据。  
**信号**是数据在传输过程中的表现形式，可以是电信号、电磁信号、光信号等。  
简单说：

信息要被表达出来 -> 形成数据  
数据要在线路上传输 -> 转换成信号  
信号要被接收端识别 -> 还原成数据

按照取值是否连续，可以分为：

| 类型   | 含义        | 例子           | 关键特点        |
|------|-----------|--------------|-------------|
| 模拟信号 | 取值连续变化的信号 | 连续变化的电压、声音波形 | 在任意时刻可能取连续值 |
| 数字信号 | 取值离散的信号   | 0/1 电平、高低电压  | 只在有限个状态中取值  |

**码元**是一个固定时长的信号波形，用来表示一个符号。这个固定时长称为**码元宽度**或**信号周期**。  
码元能携带多少信息，取决于它有多少种不同状态。若一个码元有 2 种状态，则它能表示 1 位二进制数；若一个码元有 4 种状态，则它能表示 2 位二进制数。一般地，若一个码元有  $V$  种状态，则每个码元可携带的信息量为：

$$\log_2 V \text{ bit}$$

因此：

| 码元状态数 $V$ | 每个码元可表示的二进制位数               |
|-----------|-----------------------------|
| 2         | $\log_2 2 = 1 \text{ bit}$  |
| 4         | $\log_2 4 = 2 \text{ bit}$  |
| 8         | $\log_2 8 = 3 \text{ bit}$  |
| 16        | $\log_2 16 = 4 \text{ bit}$ |

这里容易考的点是：**码元不一定只表示 1 bit**。当采用多进制码元或多元调制时，一个码元可以携带多个比特。

2. 信源、信道与信宿

一个基本通信系统可以抽象为：



各部分含义如下：

| 部分   | 作用              | 理解          |
|------|-----------------|-------------|
| 信源   | 产生并发送数据         | 数据从哪里来      |
| 变换器  | 把原始数据转换成适合传输的信号 | 发送端编码、调制等   |
| 信道   | 信号传输的通路         | 电缆、光纤、无线空间等 |
| 噪声源  | 对信号产生干扰         | 信道或设备中的随机干扰 |
| 反变换器 | 把收到的信号还原成数据     | 接收端解调、译码等   |
| 信宿   | 接收数据的终点         | 数据到哪里去      |

需要注意，实际通信系统大多是**双向通信**。所谓“单向通信系统模型”只是为了便于分析，把一个方向上的传输过程抽象出来。

按照传输信号形式，信道可分为：

| 分类角度  | 类型   | 含义               |
|-------|------|------------------|
| 按信号形式 | 模拟信道 | 传输模拟信号           |
| 按信号形式 | 数字信道 | 传输数字信号           |
| 按传输介质 | 有线信道 | 通过双绞线、同轴电缆、光纤等传输 |
| 按传输介质 | 无线信道 | 通过无线电波、微波、红外等传输  |

根据信号是否经过调制，传输方式又可分为：

| 传输方式 | 含义                      | 典型理解         |
|------|-------------------------|--------------|
| 基带传输 | 直接传输来自信源的原始电信号，不搬移到高频载波 | 数字基带信号在线路上传输 |
| 宽带传输 | 把基带信号调制到高频载波上再传输        | 利用载波承载信号     |

这里的“宽带传输”不是日常说的“宽带上网套餐”，而是通信原理中的调制传输方式。

按数据传输方式，可分为：

| 类型   | 含义             | 适用场景             |
|------|----------------|------------------|
| 串行传输 | 比特按时间顺序逐位传输    | 远距离通信，如计算机网络     |
| 并行传输 | 多个比特通过多条信道同时传输 | 短距离通信，如计算机内部部件之间 |

串行传输速度看起来“逐位传”较慢，但它布线简单、抗干扰相对更好、适合远距离；并行传输虽然一次传多位，但线路多、同步困难，更适合短距离。

从通信双方信息交互方向看，可分为：

| 通信方式  | 方向特点              | 例子         | 信道需求            |
|-------|-------------------|------------|-----------------|
| 单向通信  | 只能一个方向传输，没有反向交互   | 无线电广播、电视广播 | 只需一个方向的信道       |
| 半双工通信 | 双方都能发送和接收，但不能同时进行 | 对讲机        | 可以共用同一信道，分时双向传输 |
| 全双工通信 | 双方可同时发送和接收        | 电话通信       | 通常需要两个方向独立的信道   |

容易混淆的是：**半双工不是单向**。半双工可以双向通信，只是不能同时双向通信。

### 3. 速率、波特与带宽

速率是物理层最核心的计算概念之一，主要包括**码元传输速率**和**信息传输速率**。

考点追踪：比特率与波特率的关系（2011、2014、2022）

**码元传输速率**也称**波特率**或**调制速率**，表示数字通信系统每秒传输多少个码元，单位是 Baud。

**信息传输速率**也称**比特率**，表示数字通信系统每秒传输多少个比特，单位是 b/s。

二者关系的关键在于：一个码元到底携带多少比特。

若码元状态数为  $V$ ，波特率为  $B$  Baud，则比特率  $R$  为：

$$R = B \log_2 V$$

等价地，如果一个码元携带  $n$  bit，则：

$$R = Bn$$

| 情况    | 波特率      | 每码元携带信息 | 比特率      |
|-------|----------|---------|----------|
| 二进制码元 | $B$ Baud | 1 bit   | $B$ b/s  |
| 四进制码元 | $B$ Baud | 2 bit   | $2B$ b/s |
| 八进制码元 | $B$ Baud | 3 bit   | $3B$ b/s |

因此，**波特率和比特率是两个概念**：

- 波特率看的是“每秒多少个码元”；
- 比特率看的是“每秒多少个比特”；
- 二进制码元时，二者数值相等；
- 多进制码元时，比特率通常大于波特率。

**带宽**在不同语境中含义略有差别：

| 场景         | 带宽含义                     | 单位  |
|------------|--------------------------|-----|
| 模拟通信       | 信道能通过的频率范围，等于最高频率与最低频率之差 | Hz  |
| 数字通信或计算机网络 | 通信线路传输数据的能力，通常指最大数据传输速率  | b/s |

所以看到题目中的“带宽”时，要先判断题目语境。物理信道频率范围通常用 Hz；网络中常说的链路带宽通常用 b/s。

## 2.1.2 信道的极限容量

实际信道不可能无限提高传输速率，因为信号在信道中会受到两类主要限制：

1. **码间串扰**：码元传得太快，接收端看到的波形会相互重叠，难以分辨每个码元。
2. **噪声干扰**：信号在传输过程中受到噪声影响，失真严重时接收端会误判码元。

因此，信道容量问题本质上是在问：**给定信道条件下，最高能以多快的速率可靠传输信息？**

### 1. 奈奎斯特定理

考点追踪：奈氏准则的应用（2009、2014、2022、2023）

奈奎斯特定理讨论的是**理想低通信道**，也就是没有噪声、带宽有限的信道。它说明：即使没有噪声，只要带宽有限，码元传输速率也有上限。

在理想低通信道中，若信道带宽为  $W$  Hz，码元有  $V$  种离散电平，则极限数据传输速率为：

$$R_{\max} = 2W \log_2 V \quad \text{b/s}$$

其中：

- $W$ ：信道带宽，单位 Hz；
- $V$ ：每个码元可能取的离散电平数；
- $2W$ ：理想低通信道中的最高码元传输速率，单位 Baud；
- $\log_2 V$ ：每个码元携带的比特数。

奈奎斯特定理可以拆成两层理解：



最高码元速率：2W Baud  
每个码元携带信息： $\log_2 V$  bit  
所以最高比特率：2W  $\log_2 V$  b/s

它给出的重要结论有：

1. **码元传输速率有上限。**超过上限会出现严重码间串扰，接收端无法正确识别码元。
2. **信道频带越宽，最高码元传输速率越高。**
3. **奈奎斯特定理只限制码元速率，不直接限制每个码元携带的比特数。**因此可以通过增加码元状态数  $V$ ，例如使用 QAM-16、QAM-64 等多元调制，提高信息传输速率。

常见计算模板：

已知带宽  $W$ 、码元状态数  $V$ ：  
极限数据传输速率 =  $2W \log_2 V$

例如，若  $W = 3\text{kHz}$ ， $V = 4$ ，则：

$$R_{\max} = 2 \times 3000 \times \log_2 4 = 12000 \text{ b/s}$$

## 2. 香农定理

考点追踪：香农定理的应用（2014、2016）

香农定理讨论的是**有噪声信道**的极限数据传输速率。实际信道中一定存在噪声，所以香农定理比奈奎斯特定理更贴近真实通信环境。

若信道带宽为  $W$  Hz，信号平均功率为  $S$ ，噪声平均功率为  $N$ ，则信道的极限数据传输速率为：

$$C = W \log_2(1 + S/N) \text{ b/s}$$

其中  $S/N$  是信噪比，表示信号功率与噪声功率之比。信噪比越大，说明信号相对噪声越强，信道质量越好。

信噪比有两种表示方式：

| 表示方式  | 公式                             | 说明          |
|-------|--------------------------------|-------------|
| 无单位比值 | $S/N$                          | 直接用于香农公式    |
| 分贝表示  | $10 \log_{10}(S/N) \text{ dB}$ | 题目常给出，需要先换算 |

若题目给出信噪比为  $x$  dB，则应先换算：

$$S/N = 10^{x/10}$$

例如，若信噪比为 30 dB，则：

$$S/N = 10^{30/10} = 10^3 = 1000$$

然后才能代入香农公式：

$$C = W \log_2(1 + 1000)$$

考点追踪：信道带宽的影响因素（2014）

香农定理给出的重要结论：

- 1. 信道带宽  $W$  越大，极限数据传输速率越高。
- 2. 信噪比  $S/N$  越大，极限数据传输速率越高。
- 3. 对给定的  $W$  和  $S/N$ ，信息传输速率存在确定上限。
- 4. 只要实际传输速率低于该极限，就理论上可以找到某种方法实现近似无差错传输。
- 5. 香农公式给出的是理论上限，实际信道能达到的速率通常低于该值。

考点追踪：奈氏准则与香农定理的对比（2017）

奈奎斯特定理和香农定理经常放在一起考，区别如下：

| 对比项       | 奈奎斯特定理                   | 香农定理                    |
|-----------|--------------------------|-------------------------|
| 信道条件      | 理想低通信道，无噪声               | 有噪声信道                   |
| 主要限制      | 带宽导致的码间串扰                | 带宽和信噪比共同限制              |
| 极限对象      | 码元传输速率及由此得到的数据率          | 信息传输速率                  |
| 典型公式      | $R_{\max} = 2W \log_2 V$ | $C = W \log_2(1 + S/N)$ |
| 是否涉及码元状态数 | 涉及 $V$                   | 不直接涉及 $V$               |
| 考试关键词     | 理想低通、无噪声、码间串扰            | 噪声、信噪比、dB               |

做题时可按下面方式判断用哪个公式：

看到“理想低通信道、无噪声、码元状态数  $V$ ” -> 优先考虑奈奎斯特定理  
看到“噪声、信噪比  $S/N$ 、dB” -> 优先考虑香农定理

但如果题目同时给出二者条件，需要分别算出两个上限，并取更严格的限制。也就是说，真实可达的速率不能超过任何一个物理约束。

### 2.1.3 编码与调制

信号是数据在传输介质上的具体表现形式。为了让数据能够在线路、光纤或无线信道上传输，发送端必须把数据转换成合适的信号形式，这个过程主要分为**编码**和**调制**：

| 概念 | 含义         | 典型理解               |
|----|------------|--------------------|
| 编码 | 把数据转换成数字信号 | 用不同电平、跳变方式表示 0 和 1 |
| 调制 | 把数据转换成模拟信号 | 用载波的幅度、频率、相位承载信息   |

从“数据类型”和“信号类型”的组合看，常见方式有四类：

| 数据类型 | 转换后的信号类型 | 典型方式                 |
|------|----------|----------------------|
| 数字数据 | 数字信号     | 归零编码、非归零编码、曼彻斯特编码等   |
| 模拟数据 | 数字信号     | PCM 脉冲编码调制           |
| 数字数据 | 模拟信号     | ASK、FSK、PSK、QAM      |
| 模拟数据 | 模拟信号     | AM、FM、PM，或语音信号直接模拟传输 |

这部分容易和“基带传输、宽带传输”联系起来：数字数据编码为数字信号，通常用于基带传输；数字数据调制为模拟信号，通常用于需要载波的宽带传输。

1. 数字数据编码为数字信号

数字数据编码的核心问题是：**怎样用电平或电平跳变表示二进制 0 和 1**。考试常考的不是画得多漂亮，而是判断编码波形、同步能力、带宽占用和抗干扰能力。

考点追踪：NRZ 与 NRZI 编码的波形特性（2015）

| 编码方式         | 基本规则                         | 自同步能力   | 带宽占用 | 抗干扰能力 | 重点理解                             |
|--------------|------------------------------|---------|------|-------|----------------------------------|
| 归零编码 RZ      | 一个码元周期中间回到零电平                | 有       | 浪费   | 弱     | 码元内有归零跳变，可辅助同步，但有效传输时间变短         |
| 非归零编码 NRZ    | 整个码元周期都保持有效电平                | 无       | 不浪费  | 弱     | 编码效率最高，但长串 0 或长串 1 时缺少跳变，接收端难以同步 |
| 反向非归零编码 NRZI | 用码元开始处是否跳变表示数据               | 需要增加冗余位 | 不太浪费 | 弱     | 跳变既可表示数据，也能辅助时钟恢复，避免 RZ 的部分带宽浪费  |
| 曼彻斯特编码       | 每个码元中间都有一次电平跳变               | 有       | 浪费   | 强     | 中间跳变同时承担时钟同步和数据表示功能              |
| 差分曼彻斯特编码     | 每个码元中间跳变只用于同步，数据由码元开始处是否跳变决定 | 有       | 浪费   | 强     | 关注“是否变化”，比绝对电平更不易受极性反转影响         |

常见判别要点如下：

- **RZ**：用高电平表示 1、低电平表示 0，或按相反约定，但每个码元中间都会回到零电平。优点是能帮助同步；缺点是归零过程占用部分带宽，传输效率受影响。
- **NRZ**：不归零，一个码元周期都用来传输数据，因此编码效率高；但由于缺少码元内部时钟信息，收发双方通常需要额外时钟线或同步机制。
- **NRZI**：不直接看高低电平，而看码元起始处是否发生跳变。PDF 中采用的约定是：无跳变表示 1，有跳变表示 0。跳变可以辅助时钟恢复，USB 2.0 等标准采用这种编码思想。

考点追踪：曼彻斯特编码的波形特性（2013、2015）

- 曼彻斯特编码：每个码元中间都有一次跳变。常见约定是下降沿表示 1，上升沿表示 0，也可能采用相反约定。关键不是死记“高到低一定是 1”，而是先看题目采用哪种约定。

考点追踪：差分曼彻斯特编码的波形特性（2021）

- 差分曼彻斯特编码：中间跳变只用于时钟同步，不用于直接表示数据；数据由码元开始处是否跳变表示。PDF 中采用的约定是：码元开始处无跳变表示 1，有跳变表示 0。这种方式抗干扰能力更强。

这几种编码可以用一句话抓住主线：

NRZ：省带宽，但同步差  
RZ：有归零，能同步，但浪费带宽  
Manchester：每码元必跳变，同步强，但带宽开销大  
Differential Manchester：看“变化”而不是看“绝对电平”，抗干扰更强

2. 模拟数据编码为数字信号

模拟数据要进入数字通信系统，通常要先数字化。最典型的方法是 PCM（Pulse Code Modulation，脉冲编码调制），它包含三个步骤：

| 步骤 | 作用               | 直观理解            |
|----|------------------|-----------------|
| 采样 | 按固定时间间隔取模拟信号的瞬时值 | 把时间连续变成时间离散     |
| 量化 | 把采样得到的连续幅值按等级取整  | 把幅值连续变成幅值离散     |
| 编码 | 把量化后的数值转换为二进制码   | 把离散数值变成 0/1 比特流 |

采样部分要联系奈奎斯特采样定理：若模拟信号最高频率为  $f_{max}$ ，为了无失真恢复原信号，采样频率至少应满足：

$$f_s \geq 2f_{max}$$

这里的“至少两倍”很重要。它不是香农定理中的信噪比上限，而是模拟信号数字化时的采样频率要求。

3. 数字数据调制为模拟信号

数字数据调制是指：发送端把数字信号加载到模拟载波上，接收端再通过解调恢复数字数据。载波通常可写成正弦波形式，能被改变的主要参数有三个：幅度、频率、相位。

考点追踪：数字调制技术的分类与特点（2024）

考点追踪：调制阶数与码元比特数的关系（2011、2022）

| 调制方式   | 英文缩写 | 改变的载波参数 | 表示 0/1 的方式     | 特点            |
|--------|------|---------|----------------|---------------|
| 幅移键控   | ASK  | 幅度      | 有幅度/无幅度，或不同幅度  | 实现简单，但抗干扰能力差  |
| 频移键控   | FSK  | 频率      | 不同频率表示不同数字值    | 实现较简单，抗干扰能力较强 |
| 相移键控   | PSK  | 相位      | 不同相位表示不同数字值    | 属于绝对相位调制，效率较高 |
| 正交幅度调制 | QAM  | 幅度和相位   | 幅度、相位组合成多个信号状态 | 有限带宽内可实现更高数据率 |

对 ASK、FSK、PSK 的理解可以抓住下面三句话：

ASK：看“振幅”变不变  
FSK：看“频率”变不变  
PSK：看“相位”变不变

需要注意 **DPSK（差分相移键控）** 与 PSK 不同。DPSK 是一种相对调相方式，它通过检测当前码元与前一个码元的载波相位差来传输数字信息，例如用相位是否发生变化分别表示 1 和 0。它关注的是“相位变化关系”，而不是某个码元自身的绝对相位。

考点追踪：QAM 调制阶数与码元比特数的关系（2009、2023）

**\*\*QAM（Quadrature Amplitude Modulation，正交幅度调制）\*\***是在载波频率相同的前提下，同时调制幅度和相位，形成复合信号。由于它把“幅度维度”和“相位维度”组合起来使用，所以能够在有限带宽内表示更多码元状态。

若波特率为  $B$ ，采用  $m$  个相位，每个相位有  $n$  种振幅，则 QAM 的数据传输速率为：

$$R = B \log_2(m \times n) \quad \text{b/s}$$

其中  $m \times n$  是不同信号状态数，每个码元携带的信息量为：

$$\log_2(m \times n) \quad \text{bit}$$

典型例子：

| 调制方式   | 信号状态数 | 每个码元携带比特数                   |
|--------|-------|-----------------------------|
| QAM-16 | 16    | $\log_2 16 = 4 \text{ bit}$ |
| QAM-64 | 64    | $\log_2 64 = 6 \text{ bit}$ |

所以做题时，一旦看到 QAM、星座图、调制阶数，就要立即联想到：

每个码元携带 bit 数 =  $\log_2(\text{星座点个数})$   
比特率 = 波特率  $\times$  每码元 bit 数

4. 模拟数据直接传输为模拟信号

模拟数据也可以不经过数字化，直接以模拟信号形式传输。例如传统电话系统中，语音信号可以通过双绞线直接传输到本地交换机。

如果要通过高频信道传输，则通常把模拟信号加载到高频载波上，采用模拟调制技术：

| 模拟调制方式 | 改变的参数 | 典型含义          |
|--------|-------|---------------|
| 调幅 AM  | 载波幅度  | 用幅度变化承载原始模拟信息 |
| 调频 FM  | 载波频率  | 用频率变化承载原始模拟信息 |
| 调相 PM  | 载波相位  | 用相位变化承载原始模拟信息 |

这一部分与数字调制的区别在于：数字调制承载的是离散的 0/1 数据；模拟调制承载的是连续变化的模拟数据。

2.2 传输介质

2.2.1 双绞线、同轴电缆、光纤与无线传输介质

**传输介质**也称传输媒体，是数据传输系统中发送器和接收器之间的物理通路。物理层讨论传输介质时，重点不是“传什么内容”，而是“比特流依靠什么物理载体传播”。

按电磁波是否被限制在固体介质中传播，传输介质可以分为两类：

| 类型      | 典型介质          | 传播特点          | 直观理解              |
|---------|---------------|---------------|-------------------|
| 导向传输介质  | 双绞线、同轴电缆、光纤   | 电磁波或光波沿固体介质传播 | 信号沿着“线”走          |
| 非导向传输介质 | 空气、真空、海水等自由空间 | 电磁波在空间中传播     | 信号通过“无线空间”扩散或定向传播 |

这一节要掌握各种介质的**结构、适用场景、抗干扰能力、传输距离和带宽特点**。考试通常不是考死记硬背，而是通过场景问“应选哪种介质”“哪种抗干扰强”“哪种适合远距离高速通信”。

1. 双绞线

双绞线是最常用的传输介质，在局域网和传统电话网中广泛使用。它由两根相互绝缘的铜导线按一定规则绞合在一起构成。

双绞线之所以要“绞”，核心目的不是为了美观，而是为了**减小相邻导线之间的电磁干扰**。两根线绞合后，外界干扰在相邻小段中产生的影响会相互抵消一部分，从而提高传输质量。

双绞线按是否带屏蔽层可分为：

| 类型     | 英文缩写 | 结构特点         | 优点         | 典型理解        |
|--------|------|--------------|------------|-------------|
| 非屏蔽双绞线 | UTP  | 外部没有金属屏蔽层    | 便宜、轻便、安装方便 | 常见网线多属于这一类  |
| 屏蔽双绞线  | STP  | 外部有金属丝编织的屏蔽层 | 抗干扰能力更强    | 成本更高，布线要求更高 |

双绞线的特点可以概括为：

- **价格低、使用广**，是局域网常见传输介质。
- 既可用于**模拟传输**，也可用于**数字传输**。
- 通信距离通常从几千米到数十千米不等，具体取决于线缆质量、信号类型、传输速率等因素。
- 其带宽受铜线粗细、传输距离和线缆类别影响。距离越远，衰减越明显，高速传输越困难。

对“远距离信号衰减”的处理要区分模拟传输和数字传输：

| 传输类型 | 远距离后信号问题    | 常用处理设备 | 作用            |
|------|-------------|--------|---------------|
| 模拟传输 | 信号幅度衰减      | 放大器    | 放大衰减后的模拟信号    |
| 数字传输 | 波形失真、码元难以识别 | 中继器    | 对失真的数字信号整形、再生 |

这里容易混淆：放大器主要是“把信号放大”；中继器更强调“识别并再生数字信号”。中继器不是简单地把噪声也一起放大，而是尽量恢复出标准的数字波形。

2. 同轴电缆

同轴电缆由内导体、绝缘层、外导体屏蔽层和绝缘保护套层构成。它的内外导体共轴排列，因此称为同轴电缆。

按阻抗和用途，同轴电缆通常分为两类：

| 类型       | 主要用途     | 典型场景   |
|----------|----------|--------|
| 50Ω 同轴电缆 | 基带数字信号传输 | 早期局域网  |
| 75Ω 同轴电缆 | 宽带信号传输   | 有线电视系统 |

同轴电缆由于有外导体屏蔽层，抗干扰能力较好，适合较高速率的数据传输。早期局域网曾广泛采用同轴电缆，但随着交换技术和集线器的普及，局域网领域逐渐以双绞线作为主流传输介质。

可以这样记忆：

双绞线：便宜、常用、局域网主流

同轴电缆：屏蔽好、抗干扰强、早期局域网和有线电视常见

3. 光纤

光纤通信利用光导纤维传输光脉冲来进行通信。一般可以理解为：

有光脉冲 -> 表示 1

无光脉冲 -> 表示 0

光纤系统具有极大的带宽潜力，因为光的频率非常高。光纤主要由**纤芯**和**包层**组成：

| 部分 | 折射率特点 | 作用              |
|----|-------|-----------------|
| 纤芯 | 折射率较高 | 光主要在其中传播        |
| 包层 | 折射率略低 | 使光在纤芯和包层界面发生全反射 |



光纤能够导光的关键是**全反射**。当光从折射率较高的纤芯射向折射率较低的包层时，如果入射角大于临界角，就会在界面发生全反射，光线不断在纤芯中反射并向前传播。

光纤可分为多模光纤和单模光纤：

| 类型   | 传播方式              | 传输损耗/色散     | 光源要求          | 适用场景  |
|------|-------------------|-------------|---------------|-------|
| 多模光纤 | 允许多条不同角度入射的光线同时传播 | 损耗较大，脉冲容易展宽 | 要求相对较低        | 短距离通信 |
| 单模光纤 | 只允许单一模式的光传播       | 衰减小、失真小     | 需要定向性好的半导体激光器 | 远距离通信 |

多模光纤中，不同路径的光到达接收端的时间不同，输入的窄脉冲在输出端会变宽，这会限制高速远距离传输。单模光纤直径接近光波波长，几乎只允许一种传播模式，因此更适合长距离、高速通信。

光纤的主要优点：

- 1. **传输损耗小，中继距离长**，远距离传输时特别经济。
- 2. **抗雷电和电磁干扰能力强**，适合强电磁干扰环境。
- 3. **无串音干扰，保密性好**，不易被窃听或截取数据。
- 4. **体积小、重量轻**，在电缆管道拥塞时很有优势。

光纤部分的考试判断常围绕下面几个关键词展开：

高速、大容量、远距离、抗电磁干扰、保密性好、轻便

4. 无线传输介质

无线通信已经广泛应用于蜂窝移动通信、无线局域网、军事和野外通信等场景。无线传输介质本质上是自由空间，信号以电磁波形式传播。

常见无线传输方式如下：

| 类型   | 传播特点                   | 优点            | 局限              | 典型应用        |
|------|------------------------|---------------|-----------------|-------------|
| 无线电波 | 穿透能力较强，传播距离远，常以全向方式扩散  | 接收端无需精确对准发送端  | 易受干扰，安全性相对有限    | 移动通信、WLAN   |
| 微波   | 频率较高，带宽大，通信容量高，通常沿直线传播 | 速率高、容量大       | 地面传输距离受限，常需中继   | 地面微波通信、卫星通信 |
| 红外线  | 方向性较强，短距离传播            | 设备简单，适合近距离点到点 | 易受障碍物阻挡，不能穿墙    | 遥控器、室内短距通信  |
| 激光   | 方向性强，可用于自由空间光通信        | 带宽高、定向性强      | 受障碍物、天气和对准精度影响大 | 短距离点到点链路    |

微波、红外线和激光通信都需要较强的视距条件。微波地面通信通常需要中继站接力；卫星通信则利用地球同步卫星作为中继节点，可以有效突破地面微波传输的距离限制。

卫星通信的特点可以概括为：

| 优点                | 缺点              |
|-------------------|-----------------|
| 通信容量大、覆盖范围广、传输距离远 | 保密性较差，端到端传播时延较大 |

红外线和激光通信通常在自由空间中直接传播，常用于短距离点到点链路或室内通信。这类通信容易被障碍物阻挡，且无法穿透墙壁。

## 2.2.2 物理层接口的特性

物理层关注的是如何在各种传输介质上传输比特流，而不是传输介质本身。由于网络中的硬件设备和传输介质种类繁多，通信方式也各不相同，物理层需要尽可能屏蔽底层差异，使数据链路层不必关心这些具体差异，从而专注于本层协议与服务。

考点追踪：物理层接口的特性（2012、2018）

物理层的主要任务是定义与传输介质接口相关的一些特性，通常包括四类：

| 接口特性 | 规定内容                         | 例子/理解                     |
|------|------------------------------|---------------------------|
| 机械特性 | 接口连接器的形状、尺寸、引脚数目和排列、固定和锁定装置等 | 插头长什么样、接口怎么插、针脚如何排列       |
| 电气特性 | 接口电缆各线路上的电压范围、传输速率、距离限制等     | 高电平/低电平电压范围、最高速率、最远传输距离   |
| 功能特性 | 某条线路上出现某一电平所代表的意义，以及各线路的具体功能 | 哪根线表示发送、哪根线表示接收、某电平代表什么状态 |
| 过程特性 | 各种功能事件出现的顺序，以及时序关系           | 先建立连接还是先发送数据，控制信号按什么顺序变化  |

这四类特性可以用一句话压缩记忆：

机械看“形状尺寸”，电气看“电压速率距离”，功能看“信号含义”，过程看“先后时序”。

## 2.3 物理层设备

### 2.3.1 中继器

中继器的主要功能是对信号进行**整形、放大并转发**，以消除信号经过长距离电缆后产生的失真和衰减，使信号波形和强度重新达到要求，从而扩大网络的传输距离。

中继器处理的是物理层信号，其工作对象是比特流，而不是帧、分组或报文。它有两个端口，数据从一个端口输入后，经整形、放大，再从另一个端口输出。

理解中继器时要抓住两点：

1. **中继器连接的是同一网络的不同网段**。被中继器连接起来的网段仍然属于同一个局域网。
2. **中继器再生的是数字信号，不是进行高级转发决策**。它不会根据地址选择路径，也不会隔离冲突域或广播域。

中继器理论上似乎可以不断串联，使网络无限延长；但实际并不能这样做。原因是网络标准对信号传播时延有明确限制，如果中继器数量过多、链路太长，信号往返时延会超出协议允许范围，可能导致网络故障。

以采用同轴电缆的 10Base-5 以太网为例，曾有著名的 **5-4-3 规则**：

最多 5 段电缆  
最多 4 个中继器串联  
其中只有 3 段可以连接主机  
其余 2 段只能作为扩展通信范围的链路段

这个规则不要求死背细节，但要理解它体现的是：**物理层扩展网络距离受到传播时延和标准限制，不能无限扩展。**

2.3.2 集线器

集线器（Hub）本质上是一个**多端口中继器**。当一个端口接收到数据信号后，集线器会在传输过程中对信号进行整形和放大，并把信号转发到除输入端口以外的所有处于工作状态的端口。

集线器的行为可以抽象成：

一个端口收到比特流  
↓  
整形、放大  
↓  
广播到其他所有端口

因此，集线器不具备真正的定向转发能力。它不会像交换机那样根据目的 MAC 地址选择某个端口发送，而是把信号扩散给所有端口。若两个或多个端口同时发送信号，就会发生冲突，导致相关帧传输失败。

集线器的核心特点：

| 特点     | 含义              | 考试判断           |
|--------|-----------------|----------------|
| 物理层设备  | 只处理比特流和电信号      | 不看 MAC 地址，不处理帧 |
| 多端口中继器 | 对收到的信号整形、放大、转发  | 本质不是交换机        |
| 广播式转发  | 一个端口输入，其他所有端口输出 | 没有定向转发能力       |
| 半双工    | 同一时刻不能同时收发      | 多主机同时发送会冲突     |
| 共享带宽   | 所有端口共享同一链路能力    | 主机越多，平均可用带宽越低  |
| 不隔离冲突域 | 所有端口属于同一个冲突域    | 不能分割冲突域        |

考点追踪：物理层设备对冲突域和“广播域”的影响（2010、2020）

集线器不能分割冲突域，其所有端口都属于同一个冲突域。它在一个时钟周期内只能传输一组信息；当连接的主机数量较多，且多台主机频繁同时通信时，会产生大量冲突，使工作效率明显下

降。

例如，一个带宽为 10Mb/s 的集线器连接了 8 台计算机。若这些计算机同时工作，并粗略按共享带宽理解，则每台计算机平均可用带宽约为：

$$\frac{10}{8}\text{Mb/s} = 1.25\text{Mb/s}$$

集线器组网时，物理拓扑通常是星形结构：所有主机分别连接到中心集线器。但从逻辑通信方式看，它仍然像一条共享总线，因此可以说：

集线器组网：物理上是星形网，逻辑上仍是总线形网。

还要注意，集线器和中继器都不具备缓存能力，也不能完成链路层协议转换。若用集线器连接两个不同链路层协议或不同速率的网段，就可能出现問題。若两个网段速率分别为 10Mb/s 和 10/100Mb/s，连接后整个网络通常只能按较低的 10Mb/s 工作。

最后把中继器和集线器放在一起对比：

| 设备  | 本质     | 端口数量 | 工作层次 | 是否定向转发 | 是否隔离冲突域 |
|-----|--------|------|------|--------|---------|
| 中继器 | 信号再生设备 | 2 个  | 物理层  | 否      | 否       |
| 集线器 | 多端口中继器 | 多个   | 物理层  | 否      | 否       |

这部分最容易和数据链路层的交换机混淆。可以先用一句话区分：

中继器/集线器：只扩大传输范围，不提高智能转发能力，也不隔离冲突域。  
交换机：看 MAC 地址转发帧，可以隔离冲突域。

## 2.4 本章小结及疑难点

### 2.4.1 传输介质与物理层的关系

传输介质与物理层关系很近，但二者不能直接划等号。**传输介质不属于物理层**，它通常被非正式地称为“第 0 层”。

可以这样理解：

物理层协议 -> 规定信号如何解释为比特流  
传输介质 -> 承载电信号、光信号或电磁波的通道

传输介质本身只负责“让信号过去”，并不知道某个电平、某束光或某段电磁波到底表示 1 还是 0。真正把原始信号解释为有意义比特流的是物理层，因为物理层定义了电气、机械、功能和过程等接口特性。

二者区别可以压缩成下面的表：

| 对象   | 所处位置             | 主要作用            | 是否理解比特含义 |
|------|------------------|-----------------|----------|
| 传输介质 | 物理层之下，非正式称为第 0 层 | 承载电信号、光信号或无线电信号 | 否        |

| 对象  | 所处位置        | 主要作用             | 是否理解比特含义 |
|-----|-------------|------------------|----------|
| 物理层 | OSI/RM 的最底层 | 规定接口特性，把信号解释为比特流 | 是        |

图示关系可以用文字表示为：

物理层规则：100110101100 这串比特流应该怎样变成信号、怎样被识别

↓

传输介质：只负责让信号通过，如双绞线、同轴电缆、光纤、无线空间

↓

物理层规则：接收端再按规则把信号恢复成比特流

因此，考试中若问“光纤、双绞线是不是物理层”，更准确的说法是：**它们是物理层下面的传输介质，物理层协议会规定如何利用这些介质传输比特流。**

## 2.4.2 基带传输、频带传输与宽带传输

**基带传输**是直接发送来自信源的原始数字信号，例如高、低电平序列，不经过调制。它占用整个信道带宽，常用于短距离通信，例如局域网或计算机内部连接。

常见基带编码包括：

| 编码方式     | 核心理解                      |
|----------|---------------------------|
| 归零编码     | 每个码元中间回到零电平               |
| 非归零编码    | 电平保持，不一定回零                |
| 曼彻斯特编码   | 一个码元中间必有跳变，可兼顾数据和同步       |
| 差分曼彻斯特编码 | 通过是否在码元开始处跳变表示数据，中间跳变用于同步 |

**频带传输**是把数字信号调制到高频载波上再传输。调制后的信号更适合在模拟信道上传输，典型例子是电话线上网、Wi-Fi 等。频带传输不只让数字信号能够借助传统模拟信道传输，也支持复用，提高信道利用率。

可以这样理解调制：

原始数字信号不适合直接远距离传输

↓

把它“搭载”到高频载波上

↓

在模拟信道中传输

↓

接收端解调，恢复数字信息

**宽带传输**在传统通信语境中，强调利用频分复用等技术，把一条物理线路划分为多个频带信道，每个信道可独立进行频带传输。这样多个信号可以并行传输，链路容量显著提高。

基带、频带、宽带三者的比较：

| 传输方式 | 是否调制     | 是否占用整条信道   | 典型用途      | 记忆关键词     |
|------|----------|------------|-----------|-----------|
| 基带传输 | 不调制      | 通常占用整个信道   | 短距离数字通信   | 原始数字信号直接传 |
| 频带传输 | 调制到高频载波  | 占用某个频带     | 远距离或无线传输  | 数字信号搭载载波  |
| 宽带传输 | 通常结合频分复用 | 一条线路划分多个频带 | 有线电视、宽带接入 | 多路并行，提高容量 |

这里最容易混淆的是“频带传输”和“宽带传输”。可以记成：**频带传输强调是否调制到载波；宽带传输强调利用多个频带信道进行复用。**

### 2.4.3 奈奎斯特定理与香农定理的区别

奈奎斯特定理和香农定理都研究信道极限速率，但它们的侧重点不同。

| 定理     | 适用信道   | 主要限制因素     | 结论形式                     | 核心含义                  |
|--------|--------|------------|--------------------------|-----------------------|
| 奈奎斯特定理 | 理想低通信道 | 带宽有限导致码间串扰 | $R_{\max} = 2W \log_2 V$ | 给定带宽下，码元传输速率有上限       |
| 香农定理   | 有噪声信道  | 带宽与信噪比共同限制 | $C = W \log_2(1 + S/N)$  | 给定带宽和信噪比下，信息传输速率有理论上限 |

二者的关键区别是：

1. **奈奎斯特定理不考虑噪声**，只考虑带宽受限时码元传输速率不能无限增大。
2. **香农定理考虑噪声**，说明即使编码技术再强，给定带宽和信噪比后，信道容量也不能突破理论上限。
3. 奈奎斯特定理中的  $V$  是码元离散电平数，提高  $V$  可以让每个码元携带更多比特。
4. 香农定理中没有直接出现  $V$ ，它关注的是带宽  $W$  和信噪比  $S/N$ 。

在实际题目中，经常需要先判断题干给出的条件：

只给带宽  $W$  和码元状态数  $V$ ，且强调理想无噪声 -> 奈奎斯特定理  
 给带宽  $W$  和信噪比  $S/N$  或分贝  $dB$  -> 香农定理

若信噪比用分贝形式给出，需要先换算：

$$SNR_{dB} = 10 \log_{10}(S/N)$$

例如信噪比为 20dB，则：

$$20 = 10 \log_{10}(S/N)$$

所以：

$$S/N = 100$$

分贝形式更常用，是因为通信系统中信号功率和噪声功率可能相差很大，若直接用线性比值，数字会很长；用分贝表示更简洁，也更不容易写错。

2.4.4 本章核心知识框架

第二章物理层可以按照“信号如何传、能传多快、通过什么传、用什么设备扩展”来建立框架。

| 主线    | 核心问题          | 重点内容                              |
|-------|---------------|-----------------------------------|
| 通信基础  | 数据如何变成信号并传输   | 数据、信号、码元、基带/频带、串行/并行、单工/半双工/全双工   |
| 速率与容量 | 信道最高能传多快      | 波特率、比特率、带宽、奈奎斯特定理、香农定理            |
| 编码与调制 | 如何让信号适合传输     | 数字数据编码为数字信号、数字数据调制为模拟信号、模拟数据编码/调制 |
| 传输介质  | 信号通过什么通道传播    | 双绞线、同轴电缆、光纤、无线传输介质                |
| 接口特性  | 设备如何统一连接和通信   | 机械特性、电气特性、功能特性、过程特性               |
| 物理层设备 | 如何延长传输距离或扩展连接 | 中继器、集线器                           |

复习时要特别抓住以下判断：

- 1. 物理层传输的是比特流，不负责成帧、寻址、差错控制和路由选择。
- 2. 传输介质不属于物理层，只是物理层下面的信号通道。
- 3. 波特率是码元速率，比特率是信息速率；多进制码元下，二者通常不相等。
- 4. 奈奎斯特定理看带宽和码元状态数，香农定理看带宽和信噪比。
- 5. 中继器和集线器都属于物理层设备，不看 MAC 地址，不隔离冲突域，也不隔离广播域。
- 6. 集线器物理拓扑可以是星形，但逻辑上仍是共享总线。
- 7. 光纤高速、远距离、抗电磁干扰强；无线介质方便移动，但更容易受干扰或受视距限制。

最后用一张简表把常见设备层次先固定住，后续学习数据链路层时会反复对比：

| 设备  | 工作层次  | 处理对象      | 是否看 MAC 地址      | 是否隔离冲突域 |
|-----|-------|-----------|-----------------|---------|
| 中继器 | 物理层   | 比特流/信号    | 否               | 否       |
| 集线器 | 物理层   | 比特流/信号    | 否               | 否       |
| 网桥  | 数据链路层 | 帧         | 是               | 是       |
| 交换机 | 数据链路层 | 帧         | 是               | 是       |
| 路由器 | 网络层   | 分组/IP 数据报 | 不依据 MAC 做最终转发决策 | 是       |

本章复习的最核心目标不是背很多传输介质名称，而是建立一个完整链条：

信息 -> 数据 -> 信号 -> 传输介质 -> 接收端恢复信号 -> 比特流交给数据链路层

只要能围绕这条链条解释速率、编码、调制、介质和设备，就基本掌握了物理层的主干内容。



当前进度：已阅读并整理《计算机网络-第1-3章-体系结构至数据链路层.pdf》第 3 章第 33-77 页，完成第 3 章开篇、3.1 数据链路层的功能、3.2 组帧、3.3 差错控制、3.4 流量控制与可靠传输机制、3.5 介质访问控制、3.6 局域网、3.7 广域网、3.8 数据链路层设备，以及 3.9 本章小结及疑难点。第三章内容已全部完成。

下次任务：本章已完成，后续按 AnyWorkflow 结束 Skill 进行最终产物上传与 Activity 收尾。

## 第三章 数据链路层

本章主线是：数据链路层把网络层交下来的分组封装成帧，并在相邻节点之间传输。它关注的是“一跳链路”上的通信，而不是端到端通信。围绕“帧”这一单位，本章依次讨论封装成帧、透明传输、差错控制、流量控制、可靠传输、介质访问控制、局域网、广域网和链路层设备。

复习时可以先建立总框架：

数据链路层

- 基本功能：封装成帧、透明传输、差错检测、流量控制、可靠传输
- 组帧：字符计数、字节填充、零比特填充、违规编码
- 差错控制：奇偶校验、CRC、海明码
- 可靠传输：停止-等待、GBN、SR
- 介质访问控制：信道划分、随机访问、轮询访问
- 局域网：以太网、无线局域网、VLAN
- 广域网：PPP
- 链路层设备：网卡、网桥、交换机、集线器

### 3.1 数据链路层的功能

数据链路层位于物理层之上、网络层之下。物理层只负责比特流的传输，而数据链路层要把比特流组织成有边界、有校验、有控制意义的帧。

数据链路层常见信道有两类：

| 信道类型  | 含义         | 典型场景      | 重点问题         |
|-------|------------|-----------|--------------|
| 点对点信道 | 两个节点直接通信   | PPP、专线链路  | 组帧、差错检测、链路管理 |
| 广播信道  | 多个节点共享同一信道 | 以太网、无线局域网 | 介质访问控制、冲突处理  |

#### 3.1.1 数据链路层所处的地位

主机之间的通信看似是端到端的，但数据链路层实际负责的是相邻节点之间的传输。例如：

主机 H1 -> 路由器 R1 -> 路由器 R2 -> 主机 H2

第1段链路第2段链路第3段链路

每经过一个路由器，路由器都要先在链路层接收上一段链路的帧，取出网络层分组，再重新封装成适合下一段链路的帧。因此，不同链路段可以使用不同的数据链路层协议。

需要区分三个概念：

| 概念   | 含义                | 记忆      |
|------|-------------------|---------|
| 链路   | 相邻节点之间的物理线路       | 强调物理连接  |
| 数据链路 | 物理链路加上协议控制形成的逻辑通道 | 强调协议能力  |
| 帧    | 数据链路层的协议数据单元      | 链路层传输单位 |

一句话：链路是路，数据链路是在路上加规则，帧是按规则传输的数据单位。

### 3.1.2 链路管理

链路管理指链路的建立、维持和释放。面向连接的链路层协议在正式传输数据前，通常要先建立链路连接，协商必要参数；通信过程中维持连接状态；通信结束后释放连接。

链路管理本身不是计算难点，但常与可靠传输、确认、编号、超时重传等机制联系在一起。需要维护状态的链路层协议，通常也更依赖链路管理。

### 3.1.3 封装成帧与透明传输

封装成帧就是在网络层数据前后加上首部和尾部，形成完整帧。首部和尾部不仅携带控制信息，还用于帧定界，使接收方能知道一帧从哪里开始、到哪里结束。

HDLC 帧可抽象为：

| 字段     | 作用                       |
|--------|--------------------------|
| 标志字段 F | 标识帧开始或结束，典型比特串为 01111110 |
| 地址字段 A | 标识站点地址                   |
| 控制字段 C | 标识帧类型、编号、确认等             |
| 数据字段 I | 承载上层数据                   |
| FCS    | 帧检验序列，用于差错检测             |
| 标志字段 F | 标识帧结束                    |

透明传输是指：无论数据字段中出现什么比特组合，都不能被误认为帧定界符或控制信息。字节填充、零比特填充和违规编码，本质上都在解决透明传输问题。

### 3.1.4 流量控制

流量控制解决发送方发送过快、接收方处理不过来的问题。链路层流量控制控制的是相邻节点之间的数据流；TCP 流量控制控制的是端到端的数据流。

常见思路是接收方通过确认、窗口大小等方式反馈自己的接收能力，发送方根据反馈限制发送速率，避免接收缓存溢出。

### 3.1.5 差错检测与可靠传输

链路上传输可能出现位错和帧错：

| 错误类型 | 含义        | 例子        |
|------|-----------|-----------|
| 位错   | 某些比特发生翻转  | 0 变 1     |
| 帧错   | 帧丢失、重复、失序 | 帧未到达或重复到达 |

数据链路层至少要能检测差错，通常用 CRC 等检错码发现错误帧。是否进一步提供可靠传输，取决于链路质量：无线链路误码率高，链路层重传更有价值；高质量有线链路误码率低，往往只检错丢弃，可靠性由上层承担。

核心结论：数据链路层应避免把错误帧交给网络层，但不一定保证每个帧都可靠交付。

## 3.2 组帧

组帧的目的有三个：确定帧边界、实现帧同步、保证透明传输。常见方法包括字符计数法、字节填充法、零比特填充法、违规编码法。

### 3.2.1 字符计数法

字符计数法在帧首部设置计数字段，说明本帧总长度。接收方按长度定位帧结束位置。

```
[5 | 数据4B] [8 | 数据7B]
  ↑
  计数字段说明本帧总长度
```

优点是简单，缺点是计数字段一旦出错，接收方会失去帧同步，后续多个帧都可能被错误解析。因此它更多用于理解组帧思想，实际可靠性较差。

### 3.2.2 字节填充法

字节填充法用特殊字符表示帧起始和结束，例如 SOH 表示开始、EOT 表示结束。若数据中也出现这些特殊字符，就在其前插入转义字符 ESC。

```
原始数据: A EOT B ESC C
发送数据: A ESC EOT B ESC ESC C
接收还原: A EOT B ESC C
```

本质：用转义机制保护数据中的特殊字符，使其不被误认为控制字符。

### 3.2.3 零比特填充法

考点追踪：HDLC 的比特填充规则（2013）

零比特填充法使用 01111110 作为帧定界符。为了避免数据中出现同样的比特串，发送方扫描数据字段，每遇到 5 个连续的 1，就在后面插入一个 0；接收方每遇到 5 个连续的 1 后跟一个 0，就删除这个 0。

记忆口诀：发端见五个 1 插 0，收端见五个 1 删 0。

注意插入时机不是看到 6 个连续 1 后再处理，而是一看到 5 个连续 1 就立刻插 0。

3.2.4 违规编码法

违规编码法利用物理层编码中未使用的信号状态表示帧边界。例如曼彻斯特编码中，正常数据必须有电平跳变，若出现不符合规则的电平组合，就可作为帧边界。  
它不需要填充，透明性好，但前提是物理层编码存在冗余状态。

| 方法     | 核心思想                    | 主要问题     |
|--------|-------------------------|----------|
| 字符计数法  | 用长度字段定界                 | 计数字段错会失步 |
| 字节填充法  | 控制字符定界，ESC 转义           | 按字节处理    |
| 零比特填充法 | 定界符为 01111110，见五个 1 插 0 | 按比特处理    |
| 违规编码法  | 用物理编码违规状态定界             | 依赖编码冗余   |

3.3 差错控制

差错控制通过冗余编码发现或纠正传输错误。

| 类型   | 能力            | 典型方法     |
|------|---------------|----------|
| 检错编码 | 发现是否出错，通常不能定位 | 奇偶校验、CRC |
| 纠错编码 | 发现并定位错误，可直接修正 | 海明码      |

考点追踪：码距的计算与特性（2025）

码距是两个码字对应位不同的个数；一个编码集合的码距是任意两个合法码字距离的最小值。若码距为  $l$ ，可检测错误位数为  $d$ ，可纠正错误位数为  $c$ ，则满足：

$$l = d + c + 1, \quad d \geq c$$

常用结论：检测  $d$  位错误需要码距至少  $d + 1$ ；纠正  $c$  位错误需要码距至少  $2c + 1$ 。

3.3.1 检错编码

奇偶校验码由数据位加 1 位校验位组成，使整个码字中 1 的个数满足奇校验或偶校验。

| 校验方式 | 规则             |
|------|----------------|
| 奇校验  | 整个码字中 1 的个数为奇数 |
| 偶校验  | 整个码字中 1 的个数为偶数 |

奇偶校验能检测所有奇数位错误，但不能检测偶数位错误，也不能定位错误位。按码距理论，它能保证检测 1 位错误。

考点追踪：CRC 校验码的计算规则（2023）

CRC 的基本过程：

- 1. 双方约定生成多项式  $G(x)$ ，对应位串长度为  $r + 1$ ，阶数为  $r$ 。

2. 发送方在数据  $M$  后补  $r$  个 0。
3. 用生成多项式对应位串做模 2 除法，余数为 FCS。
4. 用 FCS 替换补上的  $r$  个 0，形成发送帧。
5. 接收方用同一生成多项式除收到的帧，余数为 0 则认为未检测到错误，否则丢弃。

模 2 除法的关键是减法不借位，等价于按位异或。

CRC 只能说明“没有检测到差错”，不能证明绝对没有差错。但在实际网络中，CRC 检错能力强、实现效率高，因此非常常用。

### 3.3.2 纠错编码

海明码是一种典型纠错码。它通过在数据位中插入若干校验位，使接收方能根据校验结果定位错误位。

若数据位数为  $m$ ，校验位数为  $r$ ，为了能定位所有 1 位错误并区分无错状态，需要：

$$2^r \geq m + r + 1$$

海明码常把校验位放在序号为  $2^0, 2^1, 2^2, \dots$  的位置，即第 1、2、4、8 位等。每个校验位负责一组位置，接收方根据校验结果组成错误位置编号。

考研中重点掌握公式和思想即可：冗余位越多，可区分的错误状态越多；海明码能纠正 1 位错误，通常也能检测 2 位错误。

## 3.4 流量控制与可靠传输机制

可靠传输通常依靠确认、编号、超时重传和滑动窗口实现。流量控制防止发送方过快；可靠传输保证接收方最终收到正确且按序的数据。

### 3.4.1 流量控制、可靠传输与滑动窗口机制

滑动窗口用来限制发送方连续发送的数据量。发送窗口表示发送方在未收到确认前最多可发送多少帧；接收窗口表示接收方当前可以接收哪些序号的帧。

窗口大小决定协议能力：

| 协议         | 发送窗口 | 接收窗口 | 特点              |
|------------|------|------|-----------------|
| 停止-等待      | 1    | 1    | 最简单，效率低         |
| 后退 N 帧 GBN | 大于 1 | 1    | 可连续发送，出错后回退重传   |
| 选择重传 SR    | 大于 1 | 大于 1 | 只重传出错帧，效率高但实现复杂 |

### 3.4.2 停止-等待协议

停止-等待协议要求发送方每发送一帧，就等待接收方确认，收到确认后才发送下一帧。

可能出现的问题及处理：

| 情况       | 处理             |
|----------|----------------|
| 数据帧出错或丢失 | 接收方不确认，发送方超时重传 |

| 情况    | 处理                  |
|-------|---------------------|
| 确认帧丢失 | 发送方超时重传，接收方用序号识别重复帧 |
| 确认帧迟到 | 发送方可能已重传，迟到确认要能被识别  |

停止-等待协议只需 1 bit 序号即可区分相邻新帧和重复帧。信道利用率为：

$$U = \frac{T_D}{T_D + RTT + T_A}$$

其中  $T_D$  为数据帧发送时间， $RTT$  为往返传播时延， $T_A$  为确认帧发送时间。传播时延越大，停止-等待效率越低。

### 3.4.3 后退 N 帧协议 GBN

GBN 允许发送方连续发送多个帧，但接收方只按序接收，接收窗口为 1。若某帧出错或丢失，接收方丢弃该帧及其后的失序帧，并重复确认最后一个按序正确收到的帧。发送方超时后，从出错帧开始重传后续所有未确认帧。

GBN 的确认常采用累计确认：确认  $k$  表示  $k$  及其以前的帧都已正确接收。

若序号字段为  $n$  bit，GBN 的发送窗口通常满足：

$$1 \leq W_T \leq 2^n - 1$$

窗口不能为  $2^n$ ，否则新一轮帧与旧帧重传会出现序号混淆。

### 3.4.4 选择重传协议 SR

SR 协议允许接收方缓存失序帧，只要求发送方重传出错或丢失的帧。它效率高，但接收方需要较大缓存，确认和重传逻辑也更复杂。

为避免新旧帧序号混淆，若序号字段为  $n$  bit，通常要求：

$$W_T \leq 2^{n-1}, \quad W_R \leq 2^{n-1}$$

且常取  $W_T = W_R = 2^{n-1}$ 。

GBN 与 SR 对比如下：

| 项目    | GBN          | SR     |
|-------|--------------|--------|
| 接收窗口  | 1            | 大于 1   |
| 失序帧   | 丢弃           | 缓存     |
| 重传范围  | 出错帧及其后所有未确认帧 | 只重传出错帧 |
| 实现复杂度 | 较低           | 较高     |
| 信道利用率 | 中等           | 较高     |

### 3.5 介质访问控制

广播信道中多个节点共享介质，需要介质访问控制 MAC 决定谁可以发送。MAC 协议分为信道划分、随机访问、轮询访问三大类。

#### 3.5.1 信道划分介质访问控制

信道划分把共享信道分成多个互不干扰的子信道，常见方法如下：

| 方法          | 划分依据   | 特点          |
|-------------|--------|-------------|
| 频分复用 FDM    | 频率     | 用户独占频带，同时通信 |
| 时分复用 TDM    | 时间     | 用户轮流使用信道    |
| 统计时分复用 STDM | 按需分配时隙 | 比固定 TDM 更灵活 |
| 波分复用 WDM    | 光波波长   | 光纤通信用       |
| 码分复用 CDM    | 码片序列   | 用户可同频同时通信   |

码分复用中，不同站点的码片序列应相互正交。接收方用目标站点码片序列与收到的叠加信号做规格化内积，结果为 1 表示发送比特 1，为 -1 表示发送比特 0，为 0 表示未发送。

#### 3.5.2 随机访问介质访问控制

随机访问协议不预先固定谁发送，节点有数据就尝试发送，因此可能冲突。

ALOHA 协议思想简单：纯 ALOHA 中站点有数据就发，冲突后随机等待再发；时隙 ALOHA 把时间划分成时隙，只允许在时隙开始发送，减少冲突概率。

CSMA 在发送前先监听信道，常见变体如下：

| 协议        | 监听到空闲      | 监听到忙       |
|-----------|------------|------------|
| 1-坚持 CSMA | 立即发送       | 持续监听，一空闲就发 |
| 非坚持 CSMA  | 立即发送       | 随机等待后再监听   |
| p-坚持 CSMA | 以概率 $p$ 发送 | 等待下一时隙继续判断 |

CSMA/CD 用于传统半双工以太网，核心是“先听后发、边听边发、冲突停发、随机重发”。冲突检测要求最短帧发送时间不小于冲突窗口  $2\tau$ 。以太网最小帧长 64B 就与此有关。

二进制指数退避算法：第  $i$  次冲突后，从 0 到  $2^k - 1$  中随机选一个数， $k = \min(i, 10)$ ，等待相应争用期后重发；重传 16 次仍失败则放弃。

CSMA/CA 用于无线局域网。无线环境难以边发送边检测冲突，因此采用冲突避免：发送前监听、等待 DIFS、随机退避、接收方回 ACK，必要时用 RTS/CTS 预约信道，缓解隐藏站问题。

#### 3.5.3 轮询访问介质访问控制

轮询访问通过集中控制或令牌控制分配发送权。典型方法有轮询协议和令牌传递协议。

轮询协议由主节点依次询问从节点是否有数据，缺点是主节点故障会影响全网。令牌传递协议中只有持有令牌的站点可以发送，避免冲突，但令牌丢失或管理复杂会带来额外问题。



### 3.6 局域网

局域网覆盖范围小、速率高、误码率低，常用于一个校园、办公室或家庭内部。常见拓扑有星形、总线形、环形和树形。现代以太网通常采用星形物理拓扑，中心设备为交换机。

#### 3.6.1 局域网的基本概念和体系结构

IEEE 802 将数据链路层拆成 LLC 子层和 MAC 子层：

| 子层  | 作用                     |
|-----|------------------------|
| LLC | 为网络层提供统一接口，屏蔽不同 MAC 差异 |
| MAC | 负责介质访问控制、MAC 地址、帧格式等   |

当前实际应用中，MAC 子层更重要，考试也更多围绕 MAC 地址、以太网帧和介质访问控制展开。

#### 3.6.2 以太网与 IEEE 802.3

以太网采用无连接、不可靠服务。发送方不编号、不确认、不重传；接收方检测到错帧就丢弃。  
以太网 MAC 地址长度为 48 bit，通常写成 6 个十六进制字节。全 1 地址 `FF-FF-FF-FF-FF-FF` 是广播地址。

以太网帧格式常见字段：

| 字段   | 长度       | 作用        |
|------|----------|-----------|
| 目的地址 | 6B       | 目的 MAC 地址 |
| 源地址  | 6B       | 源 MAC 地址  |
| 类型   | 2B       | 标识上层协议    |
| 数据   | 46~1500B | 承载网络层数据   |
| FCS  | 4B       | CRC 检错    |

以太网最小帧长为 64B，最大帧长通常为 1518B。若数据字段不足 46B，要填充到 46B，保证发送方在发送过程中能检测到冲突。

#### 3.6.3 无线局域网 IEEE 802.11

无线局域网使用 CSMA/CA 而不是 CSMA/CD。原因是无线设备难以在发送时同时检测冲突，而且存在隐藏站、暴露站等问题。

802.11 帧通常包含多个地址字段，用于适配无线终端、接入点 AP、分布式系统之间的转发。考试重点通常不要求死记复杂帧格式，而要理解：无线链路需要 AP、确认帧、退避机制和可选 RTS/CTS。

#### 3.6.4 VLAN 基本概念

VLAN 是虚拟局域网，用逻辑方式把同一物理交换网络划分成多个广播域。不同 VLAN 之间默认不能直接二层通信，需要三层设备或三层交换机转发。

VLAN 的优点：

- 限制广播范围，减少广播风暴影响。
- 按部门或业务逻辑划分网络，而不受物理位置限制。
- 提高安全性和管理灵活性。

IEEE 802.1Q 在以太网帧中插入 4B VLAN 标记，其中 VLAN ID 通常占 12 bit。

### 3.7 广域网

广域网覆盖范围大，常跨城市、地区甚至国家。它通常采用点对点链路和分组交换技术，由节点交换机逐跳转发数据。

#### 3.7.1 广域网的基本概念

广域网与局域网不是包含关系，而是覆盖范围、技术特点和应用目标不同的两类网络。二者都可以作为互联网的组成部分。

| 项目   | 广域网         | 局域网      |
|------|-------------|----------|
| 覆盖范围 | 大，跨区域       | 小，局部区域   |
| 连接方式 | 点对点链路为主     | 广播或交换式结构 |
| 关注层次 | 物理层、链路层、网络层 | 物理层、链路层  |
| 重点   | 长距离互连、资源共享  | 局部高速通信   |

节点交换机负责广域网内部逐跳转发；路由器负责不同网络之间的互连。二者都能转发数据，但工作目标不同。

#### 3.7.2 点对点协议 PPP

PPP 是最常见的点对点链路层协议，常用于用户接入 ISP 或设备间专线连接。PPPoE 是把 PPP 封装在以太网帧中传输。

PPP 由三部分组成：

| 组成   | 作用                 |
|------|--------------------|
| LCP  | 建立、配置、测试链路         |
| NCP  | 为不同网络层协议配置逻辑连接     |
| 封装方法 | 将 IP 数据报等封装进 PPP 帧 |

PPP 帧格式：

|      |      |      |    |         |     |     |
|------|------|------|----|---------|-----|-----|
| 标志F  | 地址A  | 控制C  | 协议 | 信息字段    | FCS | 标志F |
| 1B   | 1B   | 1B   | 2B | 0~1500B | 2B  | 1B  |
| 0x7E | 0xFF | 0x03 |    |         |     |     |

PPP 的特点：

1. 点对点、全双工。
2. 支持多种网络层协议。

3. 面向字节。
4. 数据传输阶段不使用编号和确认机制，不保证可靠交付。
5. 使用 FCS 检错，错帧丢弃。
6. 不使用 CSMA/CD，也不存在以太网最小帧长限制。

PPP 状态转换可按顺序记：链路静止 → 链路建立 → 鉴别 → 网络层协议 → 链路打开 → 链路终止 → 链路静止。

## 3.8 数据链路层设备

链路层设备按工作层次和功能区分如下：

| 设备  | 工作层次  | 是否识别 MAC   | 冲突域 | 广播域   |
|-----|-------|------------|-----|-------|
| 集线器 | 物理层   | 否          | 不隔离 | 不隔离   |
| 网桥  | 数据链路层 | 是          | 隔离  | 不隔离   |
| 交换机 | 数据链路层 | 是          | 隔离  | 通常不隔离 |
| 路由器 | 网络层   | 不以 MAC 为核心 | 隔离  | 隔离    |

### 3.8.1 网桥的基本概念

网桥工作在数据链路层，能读取 MAC 地址并根据转发表过滤或转发帧。它可以隔离冲突域，但通常不能隔离广播域。网桥是交换机的前身，交换机可看作多端口高速网桥。

### 3.8.2 以太网交换机

以太网交换机根据 MAC 地址转发帧，支持多个端口并行通信。每个端口通常形成独立冲突域，端口直接连接主机时一般可全双工通信，不需要 CSMA/CD。

考点追踪：以太网交换机的特点（2015）

交换方式：

| 方式   | 过程          | 优点           | 缺点     |
|------|-------------|--------------|--------|
| 直通交换 | 读到目的地址后立即转发 | 时延小          | 可能转发错帧 |
| 存储转发 | 收完整帧并检错后转发  | 可靠，可适配不同速率端口 | 时延较大   |

考点追踪：以太网交换机基于 MAC 地址的转发机制（2009）

交换机自学习规则：收到帧后，先把源 MAC 地址和入端口写入交换表，再根据目的 MAC 地址决定转发方式。

收到帧：

1. 学习源 MAC：交换表[源 MAC] = 入端口
2. 查目的 MAC：
  - 已知且目的端口不同 -> 精准转发

已知且目的端口等于入端口 -> 过滤丢弃

未知单播 -> 泛洪

广播帧 -> 泛洪

交换表项有老化时间，防止主机移动后旧表项长期存在。若交换机网络存在冗余链路，可能产生环路和广播风暴，可用生成树协议 STP 阻塞部分端口，构造无环逻辑拓扑。

### 3.8.3 共享式以太网

共享式以太网以集线器为中心，所有主机共享总带宽、同一冲突域和同一广播域，只能半双工通信，需要 CSMA/CD。

考点追踪：共享式与交换式以太网的区别（2016）

| 网络类型   | 工作层次  | 通信模式  | 带宽分配 | 是否使用 CSMA/CD |
|--------|-------|-------|------|--------------|
| 共享式以太网 | 物理层   | 半双工   | 共享   | 是            |
| 交换式以太网 | 数据链路层 | 全双工为主 | 独占   | 全双工下不需要      |

考点追踪：各类网络设备对冲突域和广播域的影响（2020、2022）

关键判断：集线器既不隔离冲突域，也不隔离广播域；交换机隔离冲突域但通常不隔离广播域；路由器既隔离冲突域，也隔离广播域。

## 3.9 本章小结及疑难点

第三章可以压缩成一句话：数据链路层负责一跳链路上的帧传输，重点解决帧边界、透明传输、差错检测、可靠传输、介质共享和二层转发。

### 3.9.1 连续 ARQ 的发送窗口上限

当接收窗口为 1、序号字段为  $n$  bit 时，发送窗口最大为：

$$W_T \leq 2^n - 1$$

如果发送窗口达到  $2^n$ ，一轮序号全部用完后，旧帧重传和新帧可能使用相同序号，接收方无法区分。

### 3.9.2 PPP 为什么不使用帧编号和确认机制

PPP 不通过编号和确认保证可靠交付，原因是：链路层可靠不等于端到端可靠；现代有线链路误码率低，链路层确认重传会增加开销。PPP 只用 FCS 检错，发现错帧即丢弃，可靠性由上层协议承担。

### 3.9.3 为什么冲突窗口内未发生冲突，后续就不会再发生冲突

CSMA/CD 中，冲突窗口为  $2\tau$ 。若发送开始后的  $2\tau$  内未检测到冲突，说明该信号已经传播到最远端，其他节点都能监听到信道忙，不会再开始发送，因此后续不会产生新的冲突。

这也是最小帧长的理论基础：发送时间必须不小于冲突窗口。

### 3.9.4 以太网速率提升后的调整

速率从 10Mb/s 提升到 100Mb/s 时，同样长度帧的发送时间缩短为原来的  $1/10$ 。若网段长度不变，可能导致发送方发完帧后仍检测不到远端冲突。100Base-T 通过缩短最大网段长度、缩短帧间最小间隔等方式，使 64B 最小帧长仍能满足冲突检测要求。

当前进度：已完成第 5 章全部正文内容（5.1 传输层提供的服务、5.2 UDP 协议、5.3 TCP 协议、5.4 本章小结及疑难点），本轮阅读处理 PDF 第 64–65 页的 5.4 “本章小结及疑难点”，未整理第 6 章内容，也未整理课后习题。

下次任务：本章 Markdown 笔记已完成，可按 AnyWorkflow 结束流程上传最终产物并结束当前 act。

## 第五章 传输层

本章是计算机网络考研中的高频章节，核心不是“背协议名”，而是理解传输层为什么要在 IP 之上再加一层、TCP 和 UDP 的服务差异，以及 TCP 如何通过确认、重传、流量控制、拥塞控制和连接管理实现可靠传输。

从体系结构角度看，网络层解决的是“主机到主机”的通信，传输层进一步解决“主机中进程到进程”的通信。应用程序真正需要的不是把数据送到某台主机，而是送到这台主机上的某个具体应用进程，因此端口、复用分用、TCP/UDP 服务模型是本章的入口。

### 5.1 传输层提供的服务

#### 5.1.1 传输层的功能

传输层位于网络层之上、应用层之下。它面向通信部分来说是最高层，面向用户功能来说又是最低层。可以这样理解各层的通信对象：

| 层次    | 主要解决的问题         | 逻辑通信对象    |
|-------|-----------------|-----------|
| 数据链路层 | 相邻结点之间传输帧       | 相邻结点到相邻结点 |
| 网络层   | 跨网络把分组送到目标主机    | 主机到主机     |
| 传输层   | 把数据交给目标主机中的正确进程 | 进程到进程     |

传输层向应用层屏蔽了底层网络的复杂性。应用进程通常不需要关心路径怎么选、路由器怎么转发、底层网络是否异构，只需要使用传输层提供的端到端逻辑通信服务。

传输层的主要功能如下。

##### 1. 提供应用进程之间的逻辑通信

网络层看见的是两台主机之间的通信，IP 数据报首部中的源 IP 地址和目的 IP 地址只能定位到主机，不能定位到主机中的具体应用进程。

真正发起通信和接收数据的是应用进程。例如一台主机可能同时运行浏览器、邮件客户端、DNS 查询程序等多个进程。传输层要在主机内部继续区分这些进程，使发送端进程的数据能交给接收端的对应进程。

图示说明：传输层在网络层主机到主机通信之上，提供端到端进程间逻辑通信。

图 5.1 的重点不是画法，而是层次边界：IP 协议的作用范围是主机到主机，传输层协议的作用范围是进程到进程。考题中如果问“端到端通信”或“进程间通信”，通常对应传输层；如果问“分组经过路由器转发到目的主机”，通常对应网络层。

##### 2. 复用和分用

**复用**发生在发送端：多个应用进程可以共用同一个传输层协议发送数据。例如多个应用都可以使用 TCP，也可以有多个应用使用 UDP。

**分用**发生在接收端：传输层收到报文后，根据首部中的相关字段，把数据交给正确的应用进程。对 TCP 和 UDP 来说，这个关键字段就是端口号。

容易混淆的是，网络层也有复用和分用，但分用依据不同：

| 层次    | 复用/分用依据   | 典型字段          |
|-------|-----------|---------------|
| 数据链路层 | 上层协议类型    | 帧的“类型”字段      |
| 网络层   | 上层协议类型    | IP 数据报的“协议”字段 |
| 传输层   | 应用进程      | 端口号字段         |
| 应用层   | 用户或应用交互入口 | 用户界面          |

也就是说，网络层用 IP 首部的“协议”字段判断应交给 TCP、UDP 还是 ICMP 等上层协议；传输层则用端口号判断应交给哪个应用进程。

3. 差错检测

传输层会对收到的整个报文进行差错检测，这里的“整个报文”包括首部和数据部分。

TCP 和 UDP 对差错的处理方式不同：

| 协议  | 发现传输层报文出错后的处理      |
|-----|--------------------|
| TCP | 要求发送方重传，从而配合实现可靠传输 |
| UDP | 直接丢弃出错的数据报，不负责重传   |

与此相比，IPv4 首部校验和只检查 IP 数据报首部，不检查数据部分。也就是说，网络层主要保证自己首部信息没有出错；传输层才进一步关注交付给应用进程的数据是否出错。

4. 提供面向连接和无连接的传输服务

传输层向应用层提供两类服务：

- **TCP**：面向连接，提供可靠的全双工逻辑信道。
- **UDP**：无连接，提供不可靠的逻辑信道。

TCP 之所以可靠，不是因为底层 IP 可靠，而是因为 TCP 在 IP 的基础上增加了确认、重传、计时器、流量控制、拥塞控制、连接管理等机制。UDP 则基本保留 IP “尽最大努力交付”的特点，只在传输层增加端口复用分用和差错检测等简单功能。

5.1.2 传输层的寻址与端口

传输层要实现进程到进程的通信，必须能够在一台主机内部区分不同应用进程，这就需要端口号。

1. 端口的作用

端口是传输层和应用层之间的服务访问点，可以理解为应用进程与传输层交互的接口。

发送数据时，应用进程把数据交给某个端口，传输层读取后封装成报文并发送；接收数据时，传输层根据目的端口号把数据交付给对应的应用进程。

TCP 和 UDP 首部中都有两个端口字段：



- **源端口号**：标识发送方应用进程，常用于接收方回传数据。
- **目的端口号**：标识接收方应用进程，是分用时最关键的依据。

2. 端口号的范围与分类

端口号长度为 16 bit，因此端口号空间大小为：

$$2^{16} = 65536$$

实际端口号取值范围是 0 到 65535。端口号只具有本地意义，即它只用于标识本机应用层中的进程；不同主机上的相同端口号之间没有直接联系。并且，TCP 的端口号空间和 UDP 的端口号空间彼此独立。

按用途可将端口号分为两大类。

| 类型       | 范围          | 作用                                |
|----------|-------------|-----------------------------------|
| 熟知端口号    | 0-1023      | 分配给重要的 TCP/IP 应用程序，使客户端能固定找到服务器进程 |
| 登记端口号    | 1024-49151  | 供非熟知端口号的应用程序使用，通常需要登记以避免冲突        |
| 客户端临时端口号 | 49152-65535 | 客户进程运行时由系统动态分配，通信结束后回收            |

常见熟知端口号如下，考题经常把“应用层协议”和“传输层协议”放在一起考。

| 应用程序/协议 | 熟知端口号 | 典型传输层协议            |
|---------|-------|--------------------|
| FTP     | 21    | TCP                |
| TELNET  | 23    | TCP                |
| SMTP    | 25    | TCP                |
| DNS     | 53    | UDP 为主，必要时也可使用 TCP |
| TFTP    | 69    | UDP                |
| HTTP    | 80    | TCP                |
| SNMP    | 161   | UDP                |

这里尤其要注意 DNS。基础查询通常使用 UDP 的 53 号端口，但在区域传送或响应报文过大等场景中也可能使用 TCP。若考试只按教材表格问“DNS 对应的典型传输层协议”，通常答 UDP；若题目强调大报文、区域传送或可靠传输，则要考虑 TCP。

3. 套接字

网络中用 IP 地址区分不同主机，用端口号区分同一主机中的不同应用进程。把 IP 地址和端口号拼接起来，就构成套接字：

$$\text{套接字 Socket} = (\text{IP 地址} : \text{端口号})$$

套接字可以唯一标识网络中某台主机上的一个通信端点。应用进程通信时，报文通常同时包含源端口号和目的端口号：源端口号方便对方回传，目的端口号用于把数据交给正确的应用进程。

需要进一步区分的是：一个 TCP 连接通常由四元组唯一确定：

(源 IP 地址, 源端口号, 目的 IP 地址, 目的端口号)

因此, 同一 IP 地址可以参与多个 TCP 连接; 同一端口号也可能出现在多个不同的 TCP 连接中。例如 Web 服务器可以同时和许多客户端建立连接, 这些连接的服务器端端口都可能是 80, 但客户端 IP 地址和客户端端口号不同, 所以连接仍可区分。

### 5.1.3 无连接服务与面向连接服务

TCP/IP 协议族在 IP 层之上定义了两个主要的传输层协议: TCP 和 UDP。

| 协议  | 中文名     | 服务类型 | 可靠性 | 典型特点                         |
|-----|---------|------|-----|------------------------------|
| TCP | 传输控制协议  | 面向连接 | 可靠  | 全双工、确认、重传、流量控制、拥塞控制、连接管理     |
| UDP | 用户数据报协议 | 无连接  | 不可靠 | 首部小、开销低、实时性好, 只提供简单复用分用和差错检测 |

TCP 要求通信双方在传输数据之前先建立连接, 在传输结束后释放连接。它不提供广播或多播服务, 而是面向两个端点之间的可靠通信。由于 TCP 需要确认、重传、流量控制、计时器和连接管理, 所以首部开销较大, 处理资源消耗较高。可靠性要求高的应用常使用 TCP, 例如 FTP、HTTP、SMTP、TELNET 等。

UDP 不建立连接。发送方可以直接发送 UDP 数据报, 接收方收到后通常不发送确认。UDP 在 IP 之上只增加了多路复用与分用、数据差错检测等功能。由于结构简单、开销小、执行效率高、实时性好, 它适用于对时延敏感、能容忍少量丢包的应用, 例如 TFTP、DNS、DHCP、RIP、SNMP 等。

一些典型互联网应用及其对应的传输层协议如下。

| 互联网应用   | TCP/IP 应用层协议  | TCP/IP 传输层协议 |
|---------|---------------|--------------|
| 域名解析    | 域名系统 DNS      | UDP          |
| 文件传送    | 简单文件传送协议 TFTP | UDP          |
| 路由选择    | 路由信息协议 RIP    | UDP          |
| IP 地址分配 | 动态主机配置协议 DHCP | UDP          |
| 网络管理    | 简单网络管理协议 SNMP | UDP          |
| 电子邮件    | 简单邮件传送协议 SMTP | TCP          |
| 远程终端接入  | 远程终端协议 TELNET | TCP          |
| 万维网     | 超文本传送协议 HTTP  | TCP          |
| 文件传送    | 文件传送协议 FTP    | TCP          |

最后要分清 IP 数据报和 UDP 数据报的层次差异。IP 数据报属于网络层, 在转发过程中路由器需要读取 IP 首部并进行存储转发; UDP 数据报作为 IP 数据报的数据载荷, 在网络层传输时对路由器不可见。也就是说, 路由器关心 IP 首部, 不关心里面装的是哪个 UDP 应用数据。

本节考点可以归纳为三句话：网络层定位主机，传输层定位进程；端口号是传输层分用的关键；TCP 用机制换可靠性，UDP 用简化换效率和实时性。

## 5.2 UDP

### 5.2.1 UDP 数据报

UDP 全称为用户数据报协议。它是在 IP 的数据报服务之上非常薄地加了一层，核心功能只有三类：**复用、分用、差错检测**。因此，UDP 不负责建立连接，不保证可靠交付，也不维护复杂的通信状态。

这并不表示 UDP “低级”或“没用”。很多应用选择 UDP，恰恰是因为它把控制权更多交给应用层：应用可以自己决定是否重传、如何控制发送速率、是否允许丢失少量数据，以及怎样在实时性和可靠性之间取舍。

考点追踪：UDP 协议的优点（2014、2024）

UDP 的主要特点可以从以下几个角度理解。

#### 1. UDP 是无连接的

发送 UDP 数据报前不需要建立连接，也没有释放连接的过程。发送方拿到应用层报文后，加上 UDP 首部就可以交给 IP 层。

对比 TCP，TCP 要在主机中维护连接状态，例如发送缓存、接收缓存、拥塞控制参数、序号、确认号等；UDP 不维护这些状态，所以建立通信的时延小，主机处理开销低。

#### 2. UDP 是面向报文的

应用层交给 UDP 一个报文，UDP 就加上首部后交给 IP 层；UDP 不会把多个报文合并，也不会把一个报文拆成多个 UDP 报文，而是尽量保持应用层报文边界。

这里要和 TCP 形成对比：TCP 面向字节流，应用交给 TCP 的数据会被看成连续字节序列，TCP 可以根据缓存、窗口、拥塞状态等因素自行分段。因此，TCP 不保留应用层“一个报文”的边界。

UDP 面向报文带来的结果是：**应用进程必须自己选择合适的报文大小**。

- 报文过长：交给 IP 层后可能发生 IP 分片，丢失任一片都会影响整个数据报，传输效率下降。
- 报文过短：UDP 首部和 IP 首部占比增大，首部开销相对变高。

#### 3. UDP 首部开销小

UDP 首部固定为 8B，由 4 个字段组成，每个字段都是 2B。TCP 首部至少 20B，因此在短报文、高实时性场景中，UDP 的额外开销更小。

#### 4. UDP 支持一对一、一对多、多对一和多对多通信

UDP 不建立端到端连接，因此它更适合广播、多播或简单请求-响应类通信。TCP 只支持一对一连接，不能直接提供广播或多播服务。

#### 5. UDP 没有拥塞控制

UDP 不会因为网络拥塞而自动降低源主机的发送速率。这对实时应用可能是优点，因为语音、视频等应用宁可丢掉少量数据，也不希望因为强行等待重传而产生明显延迟。

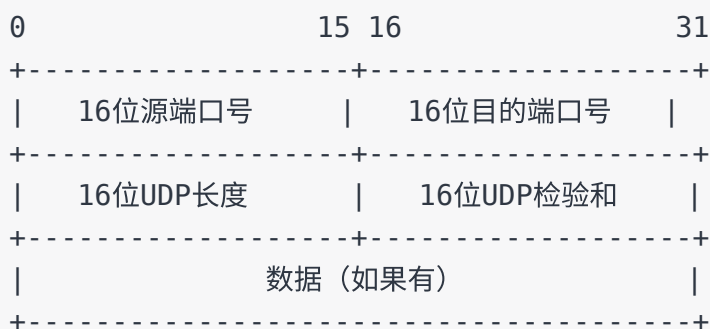
但从网络整体看，这也意味着 UDP 应用如果不在应用层主动控制发送速率，可能加重网络拥塞。因此，UDP 只是“不在传输层做拥塞控制”，并不代表应用可以无限制发送。

UDP 常见于以下类型的应用：

| 应用特点            | 典型例子     | 选择 UDP 的原因        |
|-----------------|----------|-------------------|
| 一次性传输少量数据       | DNS、DHCP | 建立 TCP 连接的开销相对不划算 |
| 简单、轻量的文件或路由信息交换 | TFTP、RIP | 协议简单，应用层可自行处理异常   |
| 实时多媒体           | 视频会议、流媒体 | 更关注低时延，允许少量丢包     |
| 网络管理            | SNMP     | 请求-响应数据较短，开销低     |

UDP 不保证可靠交付，但这不等于“应用一定不可靠”。可靠性可以由应用层自行实现，例如应用层自己编号、确认、重传、限速。考题中若问“UDP 是否可靠”，通常答：**UDP 本身不提供可靠交付；若需要可靠性，可以由应用层实现。**

UDP 数据报由 **UDP 首部** 和 **数据部分** 组成。首部固定为 8B，格式可以概括为：



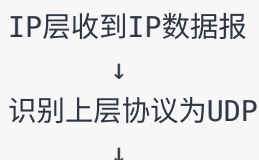
考点追踪：UDP 的首部格式及字段含义（2018）

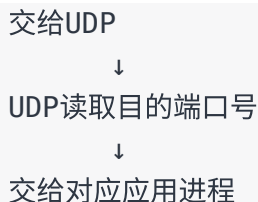
考点追踪：UDP 首部的长度（2021）

UDP 首部字段含义如下。

| 字段    | 长度     | 含义                   | 易错点                        |
|-------|--------|----------------------|----------------------------|
| 源端口号  | 16 bit | 发送进程的端口号             | 若不需要对方回信，可置为 0             |
| 目的端口号 | 16 bit | 接收进程的端口号             | UDP 分用的关键字段，必须有效           |
| 长度    | 16 bit | UDP 数据报总长度，包括首部和数据   | 最小值为 8B，因为可以只有首部没有数据       |
| 检验和   | 16 bit | 用于检测 UDP 数据报在传输中是否出错 | IPv4 中可置 0 表示不用；IPv6 中必须使用 |

接收端收到 UDP 数据报后，会根据目的端口号进行分用：





若接收方发现目的端口号不正确，也就是本机没有应用进程在该端口等待接收，则会丢弃该 UDP 数据报，并通常由 ICMP 向发送方返回“端口不可达”差错报文。

## 5.2.2 UDP 检验

UDP 检验和用于发现 UDP 数据报在传输过程中是否发生差错。它的计算方法和 IP 首部检验和类似，但检查范围更大：**UDP 检验和不仅检查 UDP 首部，还检查 UDP 数据部分，并且借助伪首部间接检查 IP 地址等关键信息。**

考点追踪：UDP 检验的原理（2025）

计算 UDP 检验和时，需要在 UDP 数据报前临时添加一个 **12B 的伪首部**。伪首部不属于 UDP 真正首部，不会向下传给网络层，也不会向上交给应用层。它只在计算检验和时临时参与运算。

伪首部的结构可以概括为：

| 字段       | 长度 | 作用               |
|----------|----|------------------|
| 源 IP 地址  | 4B | 检查源主机地址是否在传输中出错  |
| 目的 IP 地址 | 4B | 检查目的主机地址是否在传输中出错 |
| 全 0 字段   | 1B | 填充，使结构对齐         |
| 协议字段     | 1B | UDP 的协议号为 17     |
| UDP 长度   | 2B | 与 UDP 首部中的长度字段一致 |

把伪首部、UDP 首部和 UDP 数据放在一起参与检验和计算，可以理解为：UDP 虽然属于传输层，但它也要确认“这个 UDP 数据报是不是从正确的源 IP 发给正确的目的 IP”。这就是伪首部的意义。

UDP 检验和的计算过程如下。

1. 发送端先把 UDP 首部中的检验和字段置为 0。
2. 在 UDP 数据报前临时加上 12B 伪首部。
3. 把伪首部、UDP 首部、UDP 数据看成一串 16 bit 字。
4. 如果 UDP 数据部分长度为奇数字节，则在末尾临时补一个全 0 字节，使其能按 16 bit 分组；这个补 0 字节只参与计算，不真正发送。
5. 使用二进制反码求和：逐个 16 bit 字相加，若最高位产生进位，则把进位回卷加到最低位。
6. 对最后的和按位取反，得到 UDP 检验和，填入首部。

考点追踪：UDP 检验和的计算（2024）

二进制反码求和的关键规则是：

普通二进制相加：

$0 + 0 = 0$

$0 + 1 = 1$

$1 + 1 = 0$ ，并产生进位 1

若最高位产生进位：

把这个进位再加入到结果的最低位，这一步称为回卷。

最后：

对求和结果逐位取反，得到检验和。

用教材中的 3 个 16 bit 字举例，参与求和的数据为：

01100110 01100000

01010101 01010101

10001111 00001100

先将前两个字相加：

01100110 01100000

+ 01010101 01010101

-----

10111011 10110101

再与第三个字相加。若最高位产生进位，要回卷到最低位：

10111011 10110101

+ 10001111 00001100

-----

1 01001010 11000001

+1 ← 最高位进位回卷到最低位

-----

01001010 11000010

最后对结果取反，得到检验和：

01001010 11000010

取反后：

10110101 00111101

接收端验证时，会把收到的 UDP 数据报与伪首部重新组合，然后把包括检验和在内的所有 16 bit 字相加：

- 若结果为 16 位全 1，即 11111111 11111111，通常认为没有检测出差错。



- 若结果中存在 0，则说明检测出差错，接收端丢弃该 UDP 数据报。

需要注意两点。

1. 若 UDP 检验和检验出数据报错误，通常应丢弃该报文。虽然相关规范允许交付给上层并附带差错指示，但实际实现中很少采用这种方式。
2. 伪首部使 UDP 不仅能检验源端口、目的端口和数据内容，还能检验 IP 首部中的源地址和目的地址是否因传输错误而改变。

UDP 检验和的差错检测能力有限，它不能保证发现所有错误；但它计算简单、处理速度快，适合 UDP 的轻量设计目标。考试中最容易混淆的是：**伪首部只参与检验和计算，不是真正的 UDP 首部，也不会被发送。**

## 5.3 TCP

### 5.3.1 TCP 的特点

TCP 是建立在不可靠 IP 服务之上的可靠传输协议。IP 层只提供尽最大努力交付，传输过程中可能出现分组丢失、失序、重复、差错等问题；TCP 的任务就是在这种不可靠基础上，通过连接管理、序号、确认、重传、流量控制和拥塞控制等机制，让应用层看到一个可靠的字节流服务。

TCP 的主要特点如下。

#### 1. TCP 是面向连接的传输层协议

TCP 通信前要先建立连接，通信结束后要释放连接。这里的“连接”不是一条真实的物理链路，而是通信双方 TCP 实体中维护的一组状态信息，例如序号、确认号、窗口、缓存、计时器等。

因此，TCP 的可靠性来自这些状态和控制机制；代价是建立连接、释放连接和维护状态都会带来额外开销。

#### 2. 每条 TCP 连接只能有两个端点

TCP 只支持一对一通信，不直接支持一对多、多对一或多对多通信。需要广播、多播或轻量请求-响应时，通常更适合使用 UDP。

#### 3. TCP 提供可靠交付服务

TCP 要保证接收方收到的数据满足四个要求：**无差错、不丢失、不重复、按序到达**。这句话是理解 TCP 可靠传输的核心，后面的序号、确认、重传等机制都是围绕它展开的。

#### 4. TCP 支持全双工通信

TCP 连接建立后，通信双方的应用进程可以在任意时刻发送数据。为此，TCP 连接的两端都设有发送缓存和接收缓存。

- 发送缓存：暂存应用层交给 TCP、尚未发送或已发送但未确认的数据。
- 接收缓存：暂存已经到达但尚未被应用进程读取的数据，或等待按序交付的数据。

#### 5. TCP 是面向字节流的

应用程序与 TCP 交互时可能一次交给 TCP 一个数据块，但 TCP 不把这些数据块看成独立报文，而是把它们看成一串连续的、无结构的字节流。

这点与 UDP 的“面向报文”形成鲜明对比：UDP 会保留应用层报文边界；TCP 不保留应用层数据块边界。TCP 会根据接收方窗口、当前网络拥塞程度、缓存情况等动态决定每个报文段携带多少字节。

因此，应用层在使用 TCP 时不能假设“发送一次就一定接收一次”。应用层若需要区分消息边界，必须自己设计长度字段、分隔符或固定格式。

TCP 与 UDP 的关键差异可以概括如下。

| 对比角度 | TCP   | UDP             |
|------|-------|-----------------|
| 服务方式 | 面向连接  | 无连接             |
| 通信范围 | 一对一   | 一对一、一对多、多对一、多对多 |
| 数据视角 | 面向字节流 | 面向报文            |

| 对比角度 | TCP                     | UDP        |
|------|-------------------------|------------|
| 可靠性  | 提供可靠交付                  | 本身不保证可靠交付  |
| 首部开销 | 首部至少 20B                | 首部固定 8B    |
| 典型机制 | 序号、确认、重传、流量控制、拥塞控制、连接管理 | 复用、分用、差错检测 |

### 5.3.2 TCP 报文段

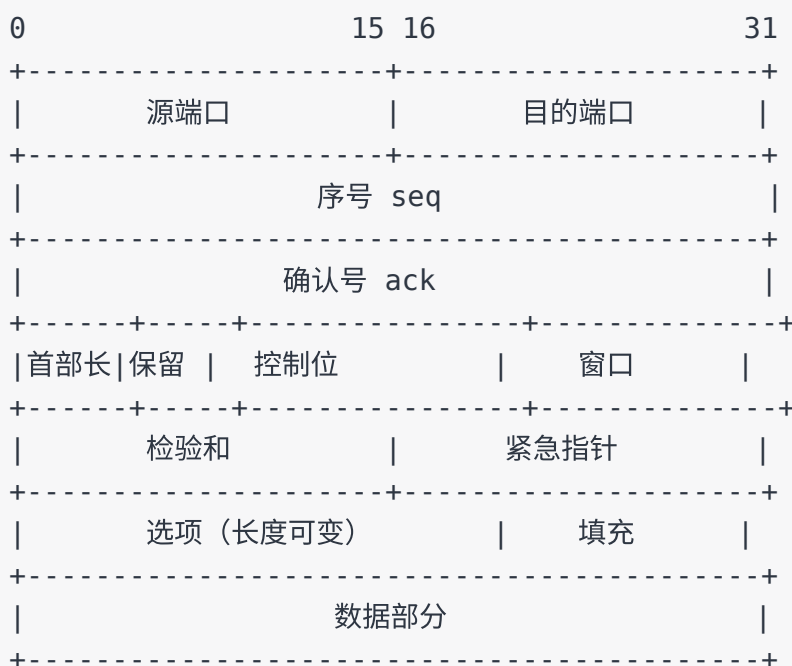
TCP 传送的数据单元称为 **TCP 报文段**。一个 TCP 报文段由 **TCP 首部** 和 **TCP 数据部分** 组成，整个 TCP 报文段会作为 IP 数据报的数据部分向下封装。

TCP 的功能几乎都体现在首部字段中。TCP 首部前 20B 是固定部分，后面可带长度可变的选项字段。由于选项字段长度可变，TCP 首部长度不是固定的，但最小长度为 20B。

考点追踪：TCP 首部格式及字段含义（2012）

考点追踪：TCP 序号与确认序号机制（2009、2016）

TCP 报文段首部可以用下面的结构概括。



TCP 首部字段含义如下。

| 字段       | 长度   | 作用                       | 易错点                               |
|----------|------|--------------------------|-----------------------------------|
| 源端口、目的端口 | 各 2B | 标识发送方和接收方应用进程            | 与 IP 地址共同确定通信端点                   |
| 序号       | 4B   | 指出本报文段所发送数据的第一个字节的编号     | 编的是字节，不是报文段                       |
| 确认号      | 4B   | 表示期望收到对方下一个报文段第一个数据字节的序号 | 确认号为 $N$ ，表示到 $N - 1$ 为止的数据均已正确收到 |

| 字段   | 长度    | 作用                        | 易错点                  |
|------|-------|---------------------------|----------------------|
| 数据偏移 | 4 bit | 表示 TCP 首部长度的，也指出数据部分起始位置  | 单位是 4B，最小值为 5，对应 20B |
| 保留   | 6 bit | 保留供以后使用                   | 目前应置 0               |
| 控制位  | 6 bit | URG、ACK、PSH、RST、SYN、FIN   | 常与连接建立、连接释放和确认机制结合考查 |
| 窗口   | 2B    | 表示接收方当前还能接收的最大数据量         | 以字节为单位，用于流量控制        |
| 检验和  | 2B    | 检验 TCP 首部和数据部分            | 计算时也要加入伪首部           |
| 紧急指针 | 2B    | 与 URG 配合指出紧急数据长度          | 只有 URG=1 时有效         |
| 选项   | 可变    | 支持 MSS、窗口扩大、时间戳、选择确认等扩展功能 | 选项最长 40B             |
| 填充   | 可变    | 使 TCP 首部长度为 4B 的整数倍       | 与数据偏移字段配合            |

**序号字段**占 4B，取值范围为  $0 \sim 2^{32} - 1$ 。序号用来给 TCP 字节流中的每一个字节编号。若某个报文段的序号为 301，数据长度为 100B，则该报文段最后一个数据字节的序号为 400，下一段数据应从序号 401 开始。

**确认号字段**占 4B，表示接收方期望收到对方下一个数据字节的序号。若确认号为  $N$ ，则表示序号  $N - 1$  及以前的所有数据均已正确收到。例如 B 已经完整收到 A 发送的序号 501 到 700 的数据，则 B 发给 A 的确认号应为 701。

考点追踪：TCP 首部的最小长度（2021）

**数据偏移字段**也叫首部长度的字段，占 4 bit。它的单位是 4B，因此 TCP 首部长度的为：

$$\text{TCP 首部长度的} = \text{数据偏移字段值} \times 4B$$

TCP 首部固定部分为 20B，所以数据偏移的最小值为 5；4 bit 最大可表示 15，因此 TCP 首部最大长度为 60B，选项字段最多为 40B。

TCP 的 6 个控制位是高频考点，含义如下。

| 控制位 | 名称  | 置 1 时的含义             | 常见场景              |
|-----|-----|----------------------|-------------------|
| URG | 紧急位 | 紧急指针字段有效             | 有紧急数据需要优先处理       |
| ACK | 确认位 | 确认号字段有效              | 连接建立后几乎所有报文段都置 1  |
| PSH | 推送位 | 接收方应尽快把数据交给应用进程      | 交互式通信，希望立即响应      |
| RST | 复位位 | TCP 连接出现严重错误，需要释放并重建 | 主机崩溃、非法报文段、拒绝连接请求 |
| SYN | 同步位 | 表示连接请求或连接接受报文        | 三次握手前两次使用         |
| FIN | 终止位 | 发送方没有更多数据要发送，请求释放连接  | 四次挥手中用于关闭发送方向     |

其中，ACK 位最容易和确认号字段混淆：**只有当 ACK=1 时，确认号字段才有效**。TCP 规定连接建立后，所有传送的报文段都必须把 ACK 置为 1。

**窗口字段**占 2B，范围为  $0 \sim 2^{16} - 1$ ，以字节为单位。它由本报文段的发送方填写，用来告诉对方：从确认号指出的字节序号开始，自己当前最多还能连续接收多少字节数据。例如确认号为 701，窗口字段为 1000，表示对方最多可从序号 701 开始连续发送 1000B，即序号 701 到 1700 的数据。

**检验和字段**占 2B，检验范围包括 TCP 首部和数据部分。与 UDP 类似，TCP 计算检验和时也要临时加入 12B 伪首部；区别是伪首部中的协议字段从 UDP 的 17 改为 TCP 的 6，长度字段改为 TCP 报文段长度。

**选项字段**长度可变，最长 40B。TCP 最初只定义了最大报文段长度 MSS（Maximum Segment Size）选项。MSS 表示 TCP 报文段中数据字段的最大长度，不包括 TCP 首部。后续又增加了窗口扩大选项、时间戳选项、选择确认等选项。注意：MSS 约束的是 TCP 数据部分长度，MTU 约束的是链路层帧数据部分的最大长度。

### 5.3.3 TCP 连接管理

TCP 是面向连接的协议，每条连接都经历三个阶段：**连接建立、数据传送、连接释放**。连接管理的目的，是确保双方能可靠地建立通信关系、分配必要资源，并在通信结束后正确释放资源。

TCP 建立连接时主要解决三个问题。

1. 让通信双方确认对方确实存在，并且愿意通信。
2. 允许双方协商相关参数，例如最大窗口值、是否使用窗口扩大选项、时间戳选项等。
3. 为传输实体分配必要资源，例如缓存空间、连接表项等。

TCP 连接的基本抽象是 **套接字**。每条 TCP 连接有两个端点，每个端点都是一个套接字：

套接字 = IP 地址 : 端口号

一条 TCP 连接由通信双方的两个套接字唯一确定。也就是说，同一 IP 地址可以参与多个 TCP 连接，同一个端口号也可以出现在多个不同的 TCP 连接中；只要四元组不同，连接就不同。

TCP 连接建立采用客户/服务器模式。主动发起连接建立的应用进程称为客户，等待连接请求的应用进程称为服务器。

#### TCP 连接的建立：三次握手

考点追踪：TCP 三次握手的字段解析（2011、2012、2016、2019、2023）

三次握手开始前，客户和服务器均处于 CLOSED 状态；服务器完成准备后进入 LISTEN 状态，等待客户的连接请求。

第一次握手：客户 → 服务器

SYN=1, seq=x

客户进入 SYN-SENT 状态

说明：SYN 报文段不能携带数据，但要消耗一个序号，所以下次序号为 x+1。

第二次握手：服务器 → 客户

SYN=1, ACK=1, seq=y, ack=x+1  
服务器进入 SYN-RCVD 状态  
说明：该报文段表示同意建立连接，也不能携带数据，同样消耗一个序号。

第三次握手：客户 → 服务器  
ACK=1, seq=x+1, ack=y+1  
客户进入 ESTABLISHED 状态；服务器收到后也进入 ESTABLISHED 状态  
说明：第三次握手可以携带数据；若不携带数据，则不消耗序号。

三次握手中的字段规律非常适合做选择题或综合题。

| 报文段   | SYN | ACK | seq     | ack     | 是否可携带数据 | 是否消耗序号    |
|-------|-----|-----|---------|---------|---------|-----------|
| 第一次握手 | 1   | 0   | $x$     | 无效      | 否       | 是         |
| 第二次握手 | 1   | 1   | $y$     | $x + 1$ | 否       | 是         |
| 第三次握手 | 0   | 1   | $x + 1$ | $y + 1$ | 可以      | 不携带数据时不消耗 |

从时间上看，若从客户发送 SYN 报文段的时刻开始计算，客户最早可在  $1RTT$  后开始发送数据；服务器最早可在  $1.5RTT$  后发送数据。这里的  $RTT$  是往返时间。

三次握手的本质不是“凑三次”，而是让双方都确认两件事：自己能发送、能接收，对方也能发送、能接收。若少一次确认，就可能导致一方误以为连接已经建立，从而造成资源浪费或异常状态。

TCP 连接的释放：四次挥手

TCP 连接是全双工的，两个方向的数据传输相互独立。因此，任何一方都可以主动关闭连接；一方发送 FIN 只表示“我这一方向没有数据要发了”，并不表示对方也不能继续发送数据。这就是 TCP 释放连接通常需要四次挥手的根本原因。

- 考点追踪：TCP 四次挥手的字段解析（2020、2023）
- 考点追踪：TCP 四次挥手状态变化（2021）
- 考点追踪：TCP 连接释放的时间分析（2016、2022、2024）

假设客户主动关闭连接，四次挥手可概括如下。

第一次挥手：客户 → 服务器  
FIN=1, seq=u  
客户进入 FIN-WAIT-1 状态  
说明：FIN 报文段即使不携带数据，也要消耗一个序号。

第二次挥手：服务器 → 客户  
ACK=1, seq=v, ack=u+1  
服务器进入 CLOSE-WAIT 状态；客户收到后进入 FIN-WAIT-2 状态  
说明：客户到服务器方向已经释放，但服务器到客户方向仍可继续发送数据，连接处于半关闭状态。

第三次挥手：服务器 → 客户

FIN=1, ACK=1, seq=w, ack=u+1

服务器进入 LAST-ACK 状态

说明：若服务器在半关闭期间又发送过数据，则  $w$  可能大于  $v$ 。

第四次挥手：客户 → 服务器

ACK=1, seq=u+1, ack=w+1

客户进入 TIME-WAIT 状态；服务器收到后进入 CLOSED 状态

说明：客户等待 2MSL 后，才进入 CLOSED 状态。

字段规律可以整理为：

| 报文段   | FIN | ACK  | seq     | ack     | 状态变化重点                                   |
|-------|-----|------|---------|---------|------------------------------------------|
| 第一次挥手 | 1   | 通常 1 | $u$     | 取决于此前通信 | 主动关闭方进入 FIN-WAIT-1                       |
| 第二次挥手 | 0   | 1    | $v$     | $u + 1$ | 被动关闭方进入 CLOSE-WAIT，主动关闭方收到后进入 FIN-WAIT-2 |
| 第三次挥手 | 1   | 1    | $w$     | $u + 1$ | 被动关闭方进入 LAST-ACK                         |
| 第四次挥手 | 0   | 1    | $u + 1$ | $w + 1$ | 主动关闭方进入 TIME-WAIT，被动关闭方收到后 CLOSED        |

需要特别记住以下结论。

1. 只有第一次和第三次挥手中的报文段用于请求关闭发送方向，因此 FIN=1。
2. FIN 报文段不能携带数据，但要消耗一个序号。
3. 后三次挥手的报文段中 ACK=1，因为它们都需要确认之前收到的报文段。
4. 协议本身并未绝对禁止 FIN 报文段携带数据，但主流操作系统一般不允许这样做，考试按教材处理即可。
5. 客户进入 TIME-WAIT 后要等待  $2MSL$ ，其中 MSL 是最长报文寿命。

TIME-WAIT 等待  $2MSL$  的主要目的有两个。

- **确保最后一个 ACK 能到达服务器。**如果第四次挥手的 ACK 丢失，服务器会重发 FIN，客户仍处于 TIME-WAIT 状态，就能再次发送 ACK。
- **让本连接中可能滞留在网络中的旧报文段自然消失**，避免旧报文段影响之后使用相同四元组的新连接。

若服务器在收到客户的 FIN 后不再发送数据，则从客户发送 FIN 的时刻起，客户释放连接的最短时间为  $1RTT + 2MSL$ ，服务器释放连接的最短时间为  $1.5RTT$ 。

除 TIME-WAIT 计时器外，TCP 还使用 **保活计时器** 防止客户端突然故障导致服务器长期空等。服务器每收到一次数据就重置计时器；若计时器超时仍未收到数据，就周期性发送探测报文。如果连续多次探测无响应，服务器可判定客户端故障并关闭连接。

本小节最容易出错的地方是：三次握手考 **SYN、ACK、seq、ack 的配合**；四次挥手考 **FIN 消耗序号、半关闭状态、TIME-WAIT 等待  $2MSL$** 。只要把“连接是全双工，两方向可独立关闭”理解清楚，四次挥手就不会死记硬背。



### 5.3.4 TCP 可靠传输

TCP 可靠传输的目标，是在不可靠的 IP 层之上，让接收方从缓存中读出的字节流和发送方发出的字节流完全一致。这里的“可靠”具体包括：无差错、不丢失、不重复、按序到达。

TCP 为实现这一目标，主要依靠 **检验和、序号、确认、重传** 等机制。其中，检验和用于发现差错；序号用于给字节流定位；确认用于告诉发送方哪些数据已经被正确接收；重传用于弥补丢失或长时间未确认的数据。

考点追踪：TCP 序号与确认机制（2011、2012、2013、2025）

**序号机制**的关键是：TCP 给字节流中的每一个字节编号，而不是给每一个报文段编号。TCP 首部中的序号字段，表示本报文段所发送数据的第一个数据字节的序号。

例如，发送方缓存中有 10B 数据，序号从 0 开始，如果将其划分为 4 个 TCP 报文段：

|       |       |   |   |  |       |   |   |  |       |   |  |       |   |
|-------|-------|---|---|--|-------|---|---|--|-------|---|--|-------|---|
| 字节序号： | 0     | 1 | 2 |  | 3     | 4 | 5 |  | 6     | 7 |  | 8     | 9 |
| 报文段：  | 第一个   |   |   |  | 第二个   |   |   |  | 第三个   |   |  | 第四个   |   |
| 首部序号： | seq=0 |   |   |  | seq=3 |   |   |  | seq=6 |   |  | seq=8 |   |

因此，如果题目给出某个 TCP 报文段的 `seq` 和数据长度，就可以立刻判断它覆盖的字节序号范围：

$$\begin{aligned} \text{首字节序号} &= seq \\ \text{末字节序号} &= seq + \text{数据长度} - 1 \\ \text{下一个数据字节序号} &= seq + \text{数据长度} \end{aligned}$$

**确认机制**的关键是：确认号表示接收方期望收到对方下一个报文段的第一个数据字节序号。若确认号为  $N$ ，就表示  $N - 1$  及以前的连续字节已经正确收到。

TCP 默认采用 **累计确认**。累计确认只确认按序到达的连续字节，而不是简单确认“我已经收到某个报文段”。例如接收方已经收到第一个和第三个报文段，但第二个报文段丢失，则即使第三个报文段已经到达，接收方仍然只能确认到第一个连续缺口之前。

|        |                             |         |       |
|--------|-----------------------------|---------|-------|
| 发送方发送： | [0,1,2]                     | [3,4,5] | [6,7] |
| 接收方收到： | [0,1,2]                     |         | [6,7] |
| 缺失部分：  |                             | [3,4,5] |       |
| 接收方确认： | ack=3                       |         |       |
| 含义：    | 已经连续收到 0~2，下一步仍期待序号 3 开始的数据 |         |       |

这也是 TCP 确认号容易出错的地方：**确认号不是“最后收到的报文段序号”，而是“下一个期望收到的字节序号”**。

TCP 的确认既可以单独发送，也可以在自己有数据要发送时顺带携带，这称为 **捎带确认**。捎带确认能减少单独确认报文段的数量，提高链路利用率。

**重传机制**用于处理丢失或迟迟未确认的数据。TCP 中触发重传的典型事件有两类：**超时** 和 **冗余 ACK**。

1. 超时重传

TCP 每发送一个报文段，就会对该报文段设置超时计时器。如果计时器到期仍未收到对应确认，TCP 就认为该报文段可能已经丢失，于是重传该报文段。

由于互联网路径复杂，往返时延不是固定值，所以 TCP 不能用一个死板的超时时间。TCP 会测量报文段发出到收到确认之间的往返时间 *RTT*，并维护一个加权平均往返时间 *RTTS*，再据此动态设置超时重传时间 *RTO*。

设置 *RTO* 时要把握一个平衡：

- *RTO* 太小：网络只是稍慢一点就会误判丢包，导致不必要重传。
- *RTO* 太大：真正丢包后要等很久才重传，传输延迟增大。

2. 冗余 ACK 与快速重传

超时重传的问题是等待时间可能较长。为了更快发现丢包，TCP 还利用冗余 ACK 来进行快速重传。

当接收方收到比期望序号更大的失序报文段时，会立即发送一个重复确认，仍然确认“下一个期望收到的字节序号”。这个重复确认就称为冗余 ACK。

例如，发送方 A 发送编号为 1、2、3、4、5 的 TCP 报文段，其中 2 号报文段丢失，而 3、4、5 号报文段先后到达接收方 B。由于 B 仍然期待 2 号报文段，所以它会反复发送对 1 号报文段之后位置的确认。发送方收到同一个报文段的 3 个冗余 ACK 后，就可以推断紧接在该确认报文段之后的报文段已经丢失，于是立即重传，而不必等待超时计时器到期。

这种机制称为 **快速重传**。

需要注意：正常确认不能算作冗余 ACK。只有在接收方收到失序的后续报文段后，反复发出的相同确认，才属于冗余 ACK。也就是说，触发快速重传时，发送方通常已经收到 4 个对同一位置的确认：1 个正常确认加 3 个冗余 ACK。

TCP 可靠传输这一节的考查重点，通常不是让你背“可靠”二字，而是给出 `seq`、`ack`、数据长度、丢失报文段位置，让你判断接收方应该返回什么确认号，以及发送方何时重传。

5.3.5 TCP 流量控制

流量控制的作用，是确保发送方的发送速率不会过快，使接收方能够及时处理收到的数据。它解决的是 **发送方发送能力与接收方接收能力之间的速度匹配问题**。

这里要先区分两个概念：

| 控制类型 | 控制目标     | 关注范围               | 典型机制                        |
|------|----------|--------------------|-----------------------------|
| 流量控制 | 不要把接收方压垮 | 端到端，发送进程与接收进程之间    | 接收窗口 <i>rwnd</i>            |
| 拥塞控制 | 不要把网络压垮  | 全局网络，涉及路由器、链路和大量主机 | 拥塞窗口 <i>cwnd</i> 、慢开始、拥塞避免等 |

TCP 使用滑动窗口机制实现流量控制。接收方维护一个 **接收窗口 *rwnd***，其大小根据当前接收缓存的剩余空间动态变化。接收方把这个窗口值写入 TCP 首部的“窗口”字段，通知发送方自己当前还能连续接收多少字节。

发送方的发送窗口不能超过接收方通告的 *rwnd*。从效果上看，*rwnd* 就是接收方给发送方划出的“最多还能发多少”的上限。

考点追踪：基于滑动窗口的流量控制机制（2016、2021）

图 5.9 展示了接收窗口逐渐变小的过程。假设数据只从 A 发往 B，B 只向 A 发送确认报文段。B 通过确认报文段中的窗口字段不断告诉 A 当前的 *rwnd*。

图示说明：TCP 滑动窗口流量控制中，发送方窗口受接收方通告窗口约束；零窗口后需要持续计时器探测。

可以把图中的过程理解为：

1. 连接建立时，B 告诉 A： *rwnd=400*，表示 B 当前最多还能接收 400B。
2. A 发送序号 1 至 100 的数据后，还能继续发送 300B。
3. A 发送序号 101 至 200 的数据后，还能继续发送 200B。
4. 序号 201 至 300 的数据在传输中丢失。
5. B 返回 *ACK=1, ack=201, rwnd=300*，表示 1 至 200 已按序收到，下一步期待 201 开始的数据，同时当前还允许 A 从 201 开始发送 300B。
6. A 继续发送 301 至 400、401 至 500，同时也会超时重发 201 至 300。由于窗口限制，A 可以重发旧数据，但不能随意发送窗口之外的新数据。
7. B 收到连续数据后，返回 *ack=501, rwnd=100*，表示允许 A 发送 501 至 600 共 100B。
8. A 发送 501 至 600 后，B 返回 *ack=601, rwnd=0*，表示到 600 为止的数据都已收到，但暂时不允许 A 再发送新数据。

这里最重要的结论是：**TCP 的窗口字段由接收方填写，表示接收方当前的接收能力；发送方必须根据这个值限制自己的发送窗口。**

当接收方通告 *rwnd=0* 时，发送方会暂停发送新数据。这时可能出现一个特殊问题：如果接收方后来缓存空出空间，并向发送方发送了非零窗口通知，但该通知在传输中丢失，那么发送方一直以为窗口为 0，不敢发送；接收方又以为自己已经通知过窗口变大，不再重复通知。双方就可能陷入相互等待的死锁状态。

为解决这个问题，TCP 为每条连接设置 **持续计时器**。只要发送方收到零窗口通知，就启动持续计时器；若计时器超时，发送方会发送一个 **零窗口探测报文段**。接收方在确认该探测报文段时，会把当前窗口值告诉发送方。

- 若窗口仍为 0，发送方收到确认后重新设置持续计时器，继续等待。
- 若窗口已经不为 0，发送方就可以恢复发送，死锁被打破。

最后，要把传输层的流量控制和数据链路层的流量控制区分开：

| 对比角度 | 传输层流量控制           | 数据链路层流量控制      |
|------|-------------------|----------------|
| 通信范围 | 端到端，即两个进程之间       | 相邻结点之间         |
| 控制对象 | 发送方与接收方的进程/主机缓存能力 | 相邻链路上的帧发送与接收能力 |

| 对比角度 | 传输层流量控制        | 数据链路层流量控制       |
|------|----------------|-----------------|
| 窗口变化 | 可根据接收方缓存情况动态变化 | 滑动窗口大小通常较固定     |
| 典型字段 | TCP 首部窗口字段     | 数据链路层协议中的窗口控制字段 |

本小节的考点可以浓缩为一句话：**流量控制看接收方，接收方用  $rwnd$  限制发送方；零窗口时靠持续计时器和零窗口探测避免死锁。**

### 5.3.6 TCP 拥塞控制

拥塞控制的目标，是防止过多数据同时注入网络，使网络中的路由器、链路和缓存不至于过载。它解决的是 **整个网络能不能承受当前负载** 的问题。

这里要再次区分流量控制和拥塞控制：

| 对比角度 | 流量控制                 | 拥塞控制                   |
|------|----------------------|------------------------|
| 控制目标 | 防止接收方来不及接收           | 防止网络整体过载               |
| 关注范围 | 端到端，主要看接收方缓存和处理能力    | 全局网络，涉及链路、路由器、主机等多种因素  |
| 主要窗口 | 接收窗口 $rwnd$          | 拥塞窗口 $cwnd$            |
| 反馈来源 | 接收方在 TCP 首部窗口字段中显式通告 | 发送方根据超时、冗余 ACK 等现象间接判断 |
| 本质   | 接收方让发送方慢一点           | 网络状态让发送方慢一点            |

拥塞发生时，端系统通常无法直接知道“哪条链路拥塞、哪个路由器队列溢出”，只能通过外在表现推断，例如时延明显增加、报文段迟迟得不到确认、发生超时等。教材中通常把 **发送方未按时收到确认而发生超时** 作为网络可能拥塞的重要信号。

TCP 进行拥塞控制时，发送方需要维护一个 **拥塞窗口  $cwnd$** 。它表示从网络拥塞角度看，发送方当前最多可以向网络中连续注入多少数据。发送窗口的实际上限由接收方能力和网络承载能力共同决定：

$$\text{发送窗口上限} = \min(rwnd, cwnd)$$

其中， $rwnd$  来自接收方通告，反映接收方还能接收多少； $cwnd$  由发送方根据网络情况动态维护，反映网络大致还能承受多少。

考点追踪：TCP 的滑动窗口机制（2010、2014–2016、2025）

TCP 拥塞控制常见算法包括 **慢开始、拥塞避免、快重传、快恢复**。可以先记住总原则：

- 如果网络未出现拥塞，就适当增大  $cwnd$ ，提高网络利用率。
- 如果检测到网络拥塞，就减小  $cwnd$ ，减少注入网络的分组数量。

#### 慢开始和拥塞避免

慢开始的基本思想是：连接刚建立时，发送方并不了解网络当前负载，如果一开始就大量发送数据，可能立即造成拥塞。因此先用较小的  $cwnd$  试探网络；如果确认正常返回，再逐步增大  $cwnd$ 。

慢开始中的“慢”不是指增长速度慢，而是指 **从很小的初始拥塞窗口开始试探**。典型教材设定为初始  $cwnd = 1MSS$ ，即一个最大报文段长度。慢开始算法规定：每收到一个对新报文段的确认，就把  $cwnd$  增加  $1MSS$ 。这样每经过一个传输轮次，即大约一个往返时间  $RTT$ ， $cwnd$  通常会按指数规律增长。

例如初始  $cwnd = 1$ ，若每个报文段都及时被确认，则大致变化为：

|          |                  |   |   |   |    |
|----------|------------------|---|---|---|----|
| 传输轮次：    | 0                | 1 | 2 | 3 | 4  |
| $cwnd$ ： | 1                | 2 | 4 | 8 | 16 |
| 增长特点：    | 从小窗口开始，按指数规律迅速增大 |   |   |   |    |

为了避免  $cwnd$  无限指数增长，TCP 设置 **慢开始门限  $ssthresh$** 。当  $cwnd$  增长到  $ssthresh$  附近时，就不再继续指数增长，而切换到拥塞避免。

拥塞避免的目标是让  $cwnd$  缓慢增长，降低拥塞风险。其策略是：每经过一个传输轮次，即收到本轮所有报文段的确认后， $cwnd$  只增加  $1MSS$ 。因此，拥塞避免阶段表现为 **线性增长**，也常称为“加法增大”。

两种算法的切换规则如下：

| 条件                | 使用算法 | $cwnd$ 增长方式   |
|-------------------|------|---------------|
| $cwnd < ssthresh$ | 慢开始  | 指数增长          |
| $cwnd > ssthresh$ | 拥塞避免 | 线性增长          |
| $cwnd = ssthresh$ | 两者均可 | 常规做法可直接进入拥塞避免 |

当发送方判断网络出现拥塞，例如发生超时，就执行“乘法减小”：

1. 将慢开始门限调整为当前拥塞窗口的一半，通常记为  $ssthresh = cwnd/2$ ，且不能小于 2。
2. 将拥塞窗口重置为  $cwnd = 1$ 。
3. 重新进入慢开始阶段，从小窗口再次试探网络。

|                                          |
|------------------------------------------|
| 考点追踪：慢开始算法的原理（2014、2015、2025）            |
| 考点追踪：拥塞避免算法的原理（2009、2017、2020、2022、2023） |
| 考点追踪：TCP 拥塞控制与传输效率分析（2016、2023）          |
| 考点追踪：TCP 拥塞窗口演化分析（2016、2023）             |

图 5.10 展示了慢开始和拥塞避免的典型变化过程。

|                                                      |
|------------------------------------------------------|
| 图示说明：慢开始阶段拥塞窗口指数增长，到达慢开始门限后进入拥塞避免并线性增长；超时时门限减半且窗口重置。 |
|------------------------------------------------------|

可以按下面的方式读图：

1. 初始时  $cwnd = 1$ ， $ssthresh = 16$ 。



2. 慢开始阶段中,  $cwnd$  近似按 1, 2, 4, 8, 16 指数增长。
3. 当  $cwnd$  到达  $ssthresh = 16$  后, 进入拥塞避免阶段,  $cwnd$  按线性规律增长。
4. 当  $cwnd = 24$  时发生网络拥塞, 执行乘法减小, 将  $ssthresh$  调整为 12, 并将  $cwnd$  重置为 1。
5. 之后再次慢开始; 当  $cwnd$  增长到新的  $ssthresh = 12$  后, 又进入拥塞避免阶段。

有一个细节容易出题: 在慢开始阶段, 如果下一轮按倍增会超过  $ssthresh$ , 则下一轮后的  $cwnd$  应等于  $ssthresh$ , 而不是机械地翻倍。例如某轮  $cwnd = 8$ 、 $ssthresh = 12$ , 则下一轮后  $cwnd = 12$ , 而不是 16。

拥塞避免这个名字并不表示它能完全避免拥塞。它只是让拥塞窗口线性缓慢增大, 从而使网络不那么容易进入拥塞状态。若网络频繁拥塞,  $ssthresh$  会不断被调低, 发送方注入网络的数据量也会明显减少。

### 快重传和快恢复

有时, 个别报文段可能因为偶然原因丢失, 但网络并没有真正出现严重拥塞。如果发送方必须等到超时计时器到期才重传, 就会误以为网络发生拥塞, 并重新执行慢开始, 导致传输效率明显下降。为此, TCP 引入了快重传和快恢复。

快重传的核心是 **尽早重传丢失的报文段**。接收方不应等到自己有数据要发送时才捎带确认, 而是在收到报文段后立即确认; 尤其是收到失序报文段时, 要立即对最后一个按序到达的报文段发送重复确认, 也就是冗余 ACK。

若发送方连续收到 3 个相同的冗余 ACK, 就可判断紧接在该确认号之后的报文段很可能已经丢失, 于是立即重传该报文段, 而不必等待超时计时器到期。

#### 考点追踪: 快重传算法的原理 (2019)

快恢复的思想是: 收到 3 个冗余 ACK 说明后续若干报文段已经到达接收方, 网络并非完全瘫痪, 因此没有必要像超时那样把  $cwnd$  直接重置为 1。此时发送方执行:

$$\begin{aligned} ssthresh &= cwnd/2 \\ cwnd &= ssthresh \end{aligned}$$

然后直接进入拥塞避免阶段, 使  $cwnd$  继续线性增长。快恢复跳过了从  $cwnd = 1$  开始的慢开始过程, 因此能减少传输速率的剧烈下降。

图 5.11 展示了同时包含超时和 3 个冗余 ACK 的拥塞控制过程。

图示说明: TCP 拥塞控制中, 超时和 3 个冗余 ACK 的窗口调整不同; 超时更激进, 冗余 ACK 后进入快恢复。

读图时要分清两种事件的处理方式:

| 事件             | 判断含义             | $ssthresh$ 调整       | $cwnd$ 调整         | 后续阶段       |
|----------------|------------------|---------------------|-------------------|------------|
| 超时             | 网络很可能已经拥塞        | $ssthresh = cwnd/2$ | $cwnd = 1$        | 慢开始        |
| 连续收到 3 个冗余 ACK | 个别报文段丢失，网络可能仍能传输 | $ssthresh = cwnd/2$ | $cwnd = ssthresh$ | 快恢复后进入拥塞避免 |

例如图 5.11 中，前 20 个轮次与图 5.10 类似。第 12 轮次发生超时，此时按超时处理：将  $ssthresh$  调整为  $24/2 = 12$ ，并把  $cwnd$  重置为 1，重新慢开始。到第 21 轮次时  $cwnd = 16$ ，发送方连续收到 3 个冗余 ACK，于是不再把  $cwnd$  重置为 1，而是将  $ssthresh = 16/2 = 8$ ，同时令  $cwnd = 8$ ，直接进入拥塞避免阶段。

TCP 拥塞控制的整体流程可以用图 5.12 概括。

图示说明：TCP 拥塞控制流程由慢开始、拥塞避免、快重传、快恢复组成，发送窗口还需同时受接收窗口限制。

把四种算法放在一起记，可以得到下面的主线：

TCP 连接建立

↓

慢开始： $cwnd$  从小开始，指数规律增大

↓  $cwnd$  达到或超过  $ssthresh$

拥塞避免： $cwnd$  线性增大

↓

若发生超时： $ssthresh = cwnd/2$ ， $cwnd = 1$ ，重新慢开始

若收到 3 个冗余 ACK： $ssthresh = cwnd/2$ ， $cwnd = ssthresh$ ，快恢复后拥塞避免

最后再把拥塞控制和流量控制合起来看。发送方真正能发送多少数据，不是只看  $rwnd$ ，也不是只看  $cwnd$ ，而是同时受二者限制：

$$\text{实际发送窗口} \leq \min(rwnd, cwnd)$$

如果题目只给接收窗口，就按流量控制分析；只给拥塞窗口变化图，就按慢开始、拥塞避免、超时、冗余 ACK 分析；如果题目同时给  $rwnd$  和  $cwnd$ ，一定取较小者作为发送窗口上限。

本小节的核心考点可以浓缩为：慢开始指数增大，拥塞避免线性增大；超时后  $cwnd$  归 1，3 个冗余 ACK 后  $cwnd$  减半并快恢复；实际发送窗口取  $rwnd$  和  $cwnd$  的较小值。



# 5.4 本章小结及疑难点

## 5.4.1 MSS 设置与 IP 分片

**MSS（最大报文段长度）** 是 TCP 报文段中数据部分的最大长度。MSS 设置的核心矛盾是：既要提高链路利用率，又要避免 IP 层分片。

如果 MSS 设置得太小，每个 TCP 报文段携带的数据很少，却仍然需要附加 TCP 首部和 IP 首部。典型情况下，TCP 首部 20B，IP 首部 20B，若数据部分只有 1B，则一个 IP 数据报中有效载荷占比极低；再考虑数据链路层首部和尾部，实际链路利用率会更低。因此，MSS 过小会造成明显的首部开销浪费。

如果 MSS 设置得太大，TCP 报文段交给 IP 层后可能超过某条链路的 MTU，从而发生 IP 分片。分片本身会增加处理开销，更重要的是：接收端必须把所有分片重新组装成原来的 TCP 报文段，只要任一分片丢失，整个 TCP 报文段就必须重传。这样反而会增加传输开销和时延。

因此，MSS 的合理原则是：**在不发生 IP 分片的前提下尽可能大**。但互联网路径是动态变化的，不同路径上的 MTU 可能不同，最佳 MSS 很难完全确定。教材给出的典型结论是：TCP 默认 MSS 可取 536B，这意味着互联网上主机应能接收长度为 536B 数据加 20B TCP 首部，即 556B 的 TCP 报文段。

可以用下面的表格理解 MSS 取值影响：

| MSS 情况 | 好处         | 问题                               | 结论  |
|--------|------------|----------------------------------|-----|
| 过小     | 不容易分片      | 首部占比高，链路利用率低                     | 不推荐 |
| 过大     | 单个报文段携带数据多 | 可能触发 IP 分片，任一分片丢失会导致整个 TCP 报文段重传 | 有风险 |
| 适中且不分片 | 兼顾效率与可靠性   | 需要结合路径 MTU                       | 最理想 |

考试中若问 MSS 与 MTU 的关系，可以抓住一句话：**MSS 是 TCP 数据部分长度，MTU 是链路层帧能承载的最大数据报长度；MSS 应尽量使封装后的 IP 数据报不超过路径 MTU。**

## 5.4.2 TCP 重传机制与 GBN、SR 的关系

从表面上看，TCP 使用累计确认，似乎更像后退 N 帧协议 GBN。因为接收方通常只确认“按序收到的最后一个字节之后的下一个序号”，发送方据此知道该确认号以前的数据都已被正确接收。

但 TCP 又不完全等同于 GBN。GBN 中，接收方会丢弃所有失序到达的帧；一旦第  $k$  个帧丢失，即使第  $k + 1$  到第  $N$  个帧已经到达接收方，也会被丢弃，发送方之后需要从第  $k$  个帧开始重传后续多个帧。

TCP 的做法更灵活。TCP 接收方不会简单丢弃正确到达但失序的报文段，而是可以将它们缓存起来，并不断发送冗余 ACK，指出期望收到的下一个字节序号。这样一来，当发送方发现某个报文段丢失后，通常只需要重传丢失的报文段，已经被接收方缓存的后续报文段不必全部重传。

此外，TCP 还可以支持 SACK（选择确认）选项。启用 SACK 后，接收方能明确告诉发送方哪些非连续数据块已经成功接收，发送方就可以更准确地重传缺失部分。这使 TCP 更接近选择重传 SR 的

思想。

因此，TCP 的重传机制可以概括为：

| 角度      | TCP 的表现                  |
|---------|--------------------------|
| 确认方式    | 以累计确认为基础，类似 GBN          |
| 失序数据处理  | 可缓存失序报文段，不像典型 GBN 那样直接丢弃 |
| 重传行为    | 通常只重传丢失部分，更接近 SR         |
| SACK 选项 | 进一步增强选择性确认能力             |

所以不能简单说 TCP 就是 GBN 或 SR。更准确的说法是：**TCP 基础行为类似 GBN，但通过缓存失序报文段、冗余 ACK、快重传和 SACK 等机制，具有 SR 的选择性重传特征。**

### 5.4.3 超时与 3 个冗余 ACK 为什么处理不同

TCP 拥塞控制中有一个高频辨析：**为什么超时后把 *cwnd* 置为 1，而收到 3 个冗余 ACK 时只是让 *cwnd* 减半？**

本质原因是二者反映的网络拥塞严重程度不同。

#### 1. 发生超时

超时意味着发送方在较长时间内没有收到应有的 ACK。造成超时的原因可能是报文段丢失，也可能是 ACK 丢失或延迟过大，但从拥塞控制角度看，这通常说明网络可能已经严重拥塞，甚至局部瘫痪。为了尽快缓解拥塞，TCP 采取更强的抑制措施：将 *ssthresh* 设为当前 *cwnd* 的一半，并把 *cwnd* 重置为 1 MSS，重新进入慢开始阶段。

#### 2. 连续收到 3 个冗余 ACK

收到冗余 ACK 说明接收方已经收到了丢失报文段之后的若干报文段，否则就不会连续触发重复确认。这说明网络仍然具备一定传输能力，只是某个报文段很可能丢失，拥塞程度相对较轻。因此，TCP 采用快重传和快恢复：立即重传推测丢失的报文段，同时将 *cwnd* 减半，而不是重置为 1。这样既能降低发送速率，又不至于让吞吐量剧烈下降。

两种事件对比如下：

| 事件        | 网络含义               | 处理强度 | <i>cwnd</i> 变化 |
|-----------|--------------------|------|----------------|
| 超时        | 拥塞可能严重，ACK 都不能及时返回 | 强抑制  | 置为 1 MSS，重新慢开始 |
| 3 个冗余 ACK | 后续报文段仍能到达，网络尚可通信   | 适度让步 | 减半后进入快恢复/拥塞避免  |

记忆时可以抓住一句话：**超时代表更严重拥塞，所以从头试探；冗余 ACK 代表网络仍能传输，所以减半退让。**

### 5.4.4 TCP 为什么必须三次握手

TCP 建立连接采用三次握手，而不是两次握手，主要是为了防止已失效的连接请求报文段突然到达服务器，造成错误连接和资源浪费。

设想采用两次握手：客户端 A 向服务器 B 发送连接请求 SYN，该 SYN 因网络滞留迟迟未到；A 超时后重发 SYN，B 收到新的 SYN 后建立连接，双方完成数据传输并释放连接。之后，先前滞留的旧 SYN 又到达 B。

如果是三次握手，B 收到旧 SYN 后会返回 SYN-ACK，但 A 发现该确认不是当前请求的响应，就会丢弃，不再发送第三次 ACK，因此连接不会真正建立。

如果是两次握手，B 收到旧 SYN 后可能立即认为连接已经建立，并开始等待 A 发送数据。但 A 并没有建立这条连接的意图，也不会发送数据，于是 B 的连接资源被长期占用，造成资源浪费。

三次握手的关键价值可以总结为：

第一次：客户端发起连接请求，告诉服务器“我想建立连接”。

第二次：服务器确认客户端请求，并告诉客户端“我同意建立连接”。

第三次：客户端再次确认，告诉服务器“我已收到你的同意，这条连接确实有效”。

从考试角度看，三次握手不是单纯为了“双方都知道彼此能收发”，更重要的是 **避免历史连接请求报文段导致服务器误建立连接**。

### 5.4.5 TCP 初始序号为什么不能固定

TCP 建立连接时不能每次都使用相同、固定的初始序号。原因在于网络中可能存在滞留的旧报文段，如果新连接继续使用旧连接相同的初始序号，接收方就可能把旧数据误认为新连接中的有效数据。

例如主机 A 和 B 频繁建立 TCP 连接。某次连接释放后，网络中仍可能滞留一些旧 TCP 报文段。随后 A 和 B 又建立一条新连接。如果新连接仍使用相同初始序号，那么旧报文段的序号可能刚好落入新连接的接收窗口中，B 就可能误收这些旧数据，导致数据混乱或协议错误。

为避免这种问题，新连接的初始序号必须尽量不同，并随时间变化。教材强调 TCP 不应使用固定初始序号，而应采用随机化、随时间递增的初始序号。这样可以降低旧报文段落入新连接有效窗口的概率。

可以把这个问题和三次握手联系起来理解：

| 机制     | 主要防止的问题             |
|--------|---------------------|
| 三次握手   | 旧 SYN 报文段导致服务器误建立连接 |
| 初始序号变化 | 旧数据报文段被新连接误收        |

两者本质上都在解决同一类问题：**互联网中报文段可能延迟、重复、乱序到达，TCP 必须能区分旧连接数据和新连接数据**。

### 5.4.6 理想链路下 TCP 可靠交付是否多余

即使假设互联网中所有链路都不出错、所有结点都不故障，TCP 的可靠交付功能仍然不是多余的。原因是 IP 层本身是无连接、不可靠的，它不承诺数据报按序、完整且不重复地到达。

即使链路不发生比特差错，网络层仍可能出现以下情况：

1. IP 数据报独立选路，可能经由不同路径到达目的主机，因此到达顺序可能不同于发送顺序。
2. 路由异常可能导致数据报在网络中循环，TTL 减为 0 后被丢弃。

3. 路由器发生拥塞时，来不及处理或缓存的数据报可能被直接丢弃。

这些问题都发生在网络层，与链路层是否出错、结点是否故障并不完全等价。也就是说，“链路不出错”只能说明帧传输时比特更可靠，不能保证 IP 数据报不会丢失、失序或重复。

因此，只有依靠 TCP 的确认、重传、序号、缓存、流量控制和拥塞控制等机制，才能向应用进程提供完整、有序、无重复的字节流服务。

本章最后可用一条主线收束：**IP 只负责尽力把数据报送到目的主机，UDP 在其上加端口和简单差错检测，TCP 则进一步用连接、序号、确认、重传、窗口和拥塞控制把不可靠网络改造成面向进程的可靠字节流服务。**

# 第六章 应用层

本章位于网络体系结构的最高层，直接面向用户的网络应用。408 考查应用层时，通常不会要求死记大量应用细节，而是围绕**典型应用协议的工作过程、使用的传输层协议、关键端口、报文交互特点**进行选择题或综合题考查。

本章核心内容可以按“应用模型 + 典型协议”理解：

| 模块       | 核心问题          | 典型考法                 |
|----------|---------------|----------------------|
| 网络应用模型   | 应用程序之间如何组织通信  | 区分 C/S 与 P2P 的角色、优缺点 |
| DNS      | 域名如何转换为 IP 地址 | 递归查询、迭代查询、本地域名服务器作用  |
| FTP      | 文件如何传输        | 控制连接与数据连接分离          |
| E-mail   | 邮件如何发送与接收     | SMTP、POP3、IMAP 的分工   |
| WWW/HTTP | 浏览器如何访问网页     | URL、HTTP 连接方式、请求响应过程 |

复习时建议把每个协议都放回“应用层协议运行在端系统进程之间”这一主线中理解：应用层协议定义的是**应用进程之间如何交换报文**，真正的数据传输仍然依赖传输层提供的端到端服务。

## 6.1 网络应用模型

网络应用模型用于描述网络应用程序之间的通信方式和组织结构。应用层不是研究比特如何在线路中传输，而是研究**不同主机上的应用进程如何协同完成网络服务**。

常见模型主要有两类：

- 1. **客户/服务器模型**，即 C/S 模型。
- 2. **对等模型**，即 P2P 模型。

这两种模型不是具体协议，而是应用系统组织方式。HTTP、FTP、电子邮件等很多传统应用主要采用 C/S 思想；文件共享、分布式下载等场景常体现 P2P 思想。

### 6.1.1 客户/服务器模型

考点追踪：C/S 模型和 P2P 模型的特点（2019）

客户/服务器模型即 Client/Server，简称 C/S。它的核心思想是：网络中存在一个始终处于开启状态的主机作为**服务器**，负责响应多个**客户机**发来的服务请求。

C/S 模型的一般工作过程是：

- 1. **服务器启动并等待请求**：服务器程序长期运行，处于监听状态。
- 2. **客户主动发起请求**：客户程序知道服务器地址，并向服务器发送请求。
- 3. **服务器处理并返回结果**：服务器解析请求，处理后把结果返回给客户。

可以把它理解成“客户提问，服务器回答”的结构。服务器通常不需要事先知道所有客户的地址；客户则必须知道服务器的位置，否则无法主动建立联系。

C/S 模型中的角色关系比较明确：

| 角色  | 地位    | 主要任务           |
|-----|-------|----------------|
| 客户  | 服务请求方 | 主动发起通信，等待服务器响应 |
| 服务器 | 服务提供方 | 被动等待请求，集中处理服务  |

典型例子包括：

- Web 应用：浏览器是客户，Web 服务器是服务器。
- 文件传输：FTP 客户端请求文件，FTP 服务器提供文件。
- 电子邮件：邮件客户程序向邮件服务器发送或读取邮件。

C/S 模型的主要特点是：**客户是服务请求方，服务器是服务提供方**。这种结构清晰，便于集中管理，所以在传统互联网服务中非常常见。

不过，C/S 模型也有明显缺点：

### 1. 服务器压力集中

网络中各计算机地位不平等，服务器承担主要管理和服务任务。当访问量很大时，服务器容易成为系统瓶颈。

### 2. 客户之间通常不直接通信

例如在普通 Web 访问中，两个浏览器之间不会直接交换网页数据，而是分别与 Web 服务器通信。

### 3. 可扩展性受服务器能力限制

单个服务器的处理能力、存储能力、带宽有限。客户数量不断增加时，需要通过服务器集群、负载均衡等方式扩展。

学习 C/S 模型时要注意：服务器是“长期运行并被动等待”的一方，客户是“主动请求”的一方。考试中常把“谁主动、谁被动”“谁必须知道谁的地址”“客户之间是否直接通信”作为判断点。

## 6.1.2 P2P 模型

P2P 是 Peer-to-Peer 的缩写，通常译为对等模型。它的基本思想是：网络中的内容不再全部集中存储在中心服务器上，每个节点都可以同时具有下载和上传能力，节点之间可以直接通信。

在 P2P 模型中，计算机没有固定的客户和服务器角色划分。任意节点都称为**对等方**，既可以向其他节点请求资源，也可以向其他节点提供资源。

与 C/S 模型相比，P2P 模型的角色关系更灵活：

| 模型  | 节点角色           | 通信方式       | 资源位置      |
|-----|----------------|------------|-----------|
| C/S | 客户和服务器分工明确     | 客户通常与服务器通信 | 资源集中在服务器  |
| P2P | 每个节点既可请求也可提供服务 | 节点之间可直接通信  | 资源分散在多个节点 |

从实现机制看，P2P 并不是完全脱离 C/S 思想。一个节点访问其他节点资源时，它相当于客户；当它向其他节点提供资源时，它又相当于服务器。因此，P2P 可以看成是把“客户”和“服务器”的角色分散到了大量普通节点上。

P2P 模型的主要优点包括：

### 1. 减轻中心服务器压力

任务被分散到大量节点上，不再完全依赖单一服务器，能够提高系统整体效率和资源利用率。



## 2. 节点之间可以直接通信

数据可以在对等节点之间传输，不必每次都经过中心服务器转发。

## 3. 扩展性较好

在 C/S 模型中，服务器性能和带宽限制了并发能力；而在 P2P 网络中，随着节点数量增加，整体可用资源也可能随之增加。

## 4. 健壮性较强

单个节点失效通常不会导致整个网络瘫痪，因为其他节点仍然可以继续提供服务。

但 P2P 也不是无条件更好，它的缺点同样需要掌握：

### 1. 会占用普通用户主机资源

节点在获取资源的同时，也要向其他节点提供服务，这会消耗本机 CPU、内存、磁盘 I/O 和网络带宽。

### 2. 可能造成网络拥塞

频繁的 P2P 下载和上传会带来大量流量。资料中提到，某些年份 P2P 应用曾占互联网流量的很高比例，因此 ISP 往往会对 P2P 应用采取限制或反对态度。

### 3. 管理和控制更复杂

由于资源分散在大量节点上，节点加入、退出、失效都比较频繁，系统维护比集中式服务器更复杂。

本节复习时可以抓住一句话：**C/S 强调中心服务器，P2P 强调节点对等与资源分散**。选择题中如果看到“中心服务器瓶颈”“客户之间不直接通信”，多与 C/S 相关；如果看到“节点既是客户又是服务器”“资源分布在多个节点”“系统容量随节点增加而自然增长”，多与 P2P 相关。

## 6.2 域名系统

考点追踪：DNS 使用的传输层及以下协议（2018、2021）

域名系统 DNS，即 Domain Name System，是互联网中负责把**便于人记忆的主机名转换为便于机器处理的 IP 地址**的命名系统。

如果只有 IP 地址，用户访问网站需要记住一串数字；而有了 DNS，用户只需要输入域名，系统就能自动完成域名到 IP 地址的映射。DNS 的作用类似互联网中的“电话簿”：人查名字，系统查号码。

DNS 需要掌握三条主线：

1. 域名空间是层次结构。

2. 域名服务器也是层次结构。

3. 域名解析可以采用递归查询和迭代查询。

DNS 是应用层协议。它通常使用 UDP，熟知端口号为 53。部分场景如区域传送或响应报文过大时可能使用 TCP，但 408 常考的基本结论是：**DNS 查询通常基于 UDP，端口号为 53。**

### 6.2.1 层次域名空间

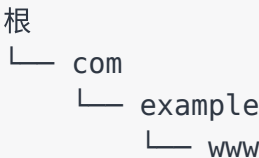
域名采用层次树状结构，整个域名空间从根开始向下划分。每一级域名之间用点分隔，越靠右层次越高。

例如：



www.example.com

可以理解为：



一个完整域名通常由若干标号组成，每个标号之间用点分隔。最右边是顶级域名，向左依次是二级域名、三级域名等。域名的层次结构使互联网命名管理可以分布式进行，而不需要由单一机构保存所有主机名与 IP 地址的映射。

常见顶级域名可以分为几类：

| 类型        | 示例              | 含义             |
|-----------|-----------------|----------------|
| 国家或地区顶级域名 | cn、us、jp        | 按国家或地区划分       |
| 通用顶级域名    | com、org、net、edu | 按组织类型或用途划分     |
| 基础结构域名    | arpa            | 常用于反向域名解析等基础设施 |

域名空间的核心考法是：域名是分层的，解析过程也可以沿着这种层次结构逐级定位。

### 6.2.2 域名服务器

DNS 不使用单个服务器保存全部域名信息，而是使用分布式的域名服务器系统。这样可以避免单点故障，也能提高查询效率和管理灵活性。

常见域名服务器包括：

| 域名服务器   | 作用                                |
|---------|-----------------------------------|
| 根域名服务器  | 位于 DNS 层次结构最高层，知道顶级域名服务器的位置       |
| 顶级域名服务器 | 管理某个顶级域名下的域名信息，如 com、cn           |
| 权限域名服务器 | 保存某个区域内主机名到 IP 地址的权威映射            |
| 本地域名服务器 | 用户主机默认联系的 DNS 服务器，通常由 ISP、学校或单位提供 |

其中，本地域名服务器特别重要。主机进行 DNS 查询时，通常不会直接从根域名服务器开始问，而是先把请求交给本地域名服务器。本地域名服务器再代表主机去查询其他服务器，或利用缓存直接返回结果。

DNS 采用分布式层次结构的原因主要有：

- 1. 避免集中式服务器压力过大。
- 2. 避免单点故障。
- 3. 便于按域名层次分区管理。
- 4. 可以利用缓存提高查询效率。

### 6.2.3 域名解析过程

域名解析就是把域名转换为 IP 地址的过程。常见查询方式有递归查询和迭代查询。

**递归查询**的特点是：被询问的服务器必须返回最终结果，或者返回查询失败。也就是说，客户只问一次，后续查询由被问的服务器负责完成。

**迭代查询**的特点是：被询问的服务器如果不知道最终结果，就告诉查询方下一步应该询问哪个服务器。查询方再继续向下一个服务器查询。

两者可以用一个对比表记忆：

| 查询方式 | 谁继续查       | 返回内容          | 记忆方式      |
|------|------------|---------------|-----------|
| 递归查询 | 被询问的服务器继续查 | 最终 IP 或失败     | “你替我查到底”  |
| 迭代查询 | 查询方自己继续查   | 下一步服务器地址或最终结果 | “你告诉我该问谁” |

一个典型域名解析过程可以表示为：

主机 → 本地域名服务器 → 根域名服务器 → 顶级域名服务器 → 权限域名服务器

在很多教材和考试题中，主机到本地域名服务器之间常使用递归查询；本地域名服务器向根、顶级、权限域名服务器的查询常使用迭代查询。

例如访问 `www.example.com` 时，过程可理解为：

1. 主机向本地域名服务器询问 `www.example.com` 的 IP 地址。
2. 若本地域名服务器没有缓存，就向根域名服务器查询。
3. 根域名服务器告诉本地域名服务器：应去询问 `com` 顶级域名服务器。
4. 顶级域名服务器告诉本地域名服务器：应去询问 `example.com` 的权限域名服务器。
5. 权限域名服务器返回 `www.example.com` 对应的 IP 地址。
6. 本地域名服务器把 IP 地址返回给主机，并可把结果缓存一段时间。

DNS 缓存是提高效率的重要机制。域名服务器可以缓存最近查询过的映射结果，后续再查询同一域名时，就可以直接返回缓存结果，减少层次查询次数。但缓存记录通常有有效期，过期后需要重新查询，以避免使用过时地址。

复习 DNS 时，可以抓住三句话：

1. DNS 是应用层协议，通常使用 UDP，端口号 53。
2. DNS 是分布式、层次化的命名系统。
3. 递归查询要求被问者给最终结果，迭代查询只给下一步线索。

### 6.3 文件传输协议

文件传输协议 FTP，即 File Transfer Protocol，用于在主机之间传送文件。它属于应用层协议，使用客户/服务器方式工作，并且基于 TCP 提供可靠传输。

FTP 与普通文件下载的直观区别在于：FTP 不只传输文件本身，还需要进行登录、命令控制、目录切换等交互。因此，FTP 在连接设计上采用了**控制连接和数据连接分离**的方式。

### 6.3.1 FTP 的工作原理

考点追踪：FTP 使用的传输层协议（2013）

FTP 的基本工作过程是：客户端与 FTP 服务器建立连接，用户完成身份认证后，通过命令请求上传、下载、列目录等操作，服务器再通过数据连接传输具体文件或目录列表。

FTP 的典型特点包括：

#### 1. 基于 TCP

FTP 需要可靠传输，因此使用 TCP。

#### 2. 采用客户/服务器模式

FTP 客户端主动发起连接，FTP 服务器长期运行并等待请求。

#### 3. 需要登录认证

传统 FTP 通常要求用户名和口令，也可以支持匿名访问。

#### 4. 屏蔽不同系统的文件细节差异

不同操作系统的文件格式、目录结构可能不同，FTP 提供统一的文件传输接口。

从应用层看，FTP 的任务不是研究文件在物理链路上怎样变成比特，而是规定客户与服务器之间如何发送命令、如何建立数据连接、如何传送文件内容。

### 6.3.2 控制连接与数据连接

考点追踪：FTP 控制连接与数据连接（2021）

FTP 最核心的考点是：**FTP 使用两个并行的 TCP 连接。**

| 连接   | 端口       | 作用        | 持续时间           |
|------|----------|-----------|----------------|
| 控制连接 | 服务器端口 21 | 传送控制命令和响应 | 整个会话期间一直保持     |
| 数据连接 | 服务器端口 20 | 传送文件或目录列表 | 每次文件传输时建立，用完关闭 |

控制连接用于传输命令，例如登录、切换目录、请求下载文件等。它在整个 FTP 会话期间保持打开，因此也称为**带外控制**：控制信息和数据不是混在同一连接中传输，而是分开传输。

数据连接用于真正传输文件数据。每当需要传送文件或目录列表时，FTP 会单独建立数据连接，传输结束后关闭。

这种分离设计带来的结果是：

#### 1. 控制命令不会被大文件传输阻塞。

即使正在传输大文件，控制连接仍然可以保持会话控制。

#### 2. 文件传输结束后，数据连接可立即关闭。

这样可以节省资源。

#### 3. 考试中要区分两个端口。

FTP 控制连接使用 TCP 21 端口，数据连接通常使用 TCP 20 端口。

复习 FTP 时，一句话最重要：**FTP 基于 TCP，控制连接端口 21，数据连接端口 20，控制连接贯穿整个会话，数据连接按需建立和释放。**

## 6.4 电子邮件

电子邮件系统是互联网中最典型的应用之一。它允许用户异步发送和接收消息：发送方不需要和接收方同时在线，邮件可以先存放在邮件服务器中，等待接收方之后读取。

电子邮件要掌握三类协议或机制：

- 1. **SMTP**：负责发送邮件。
- 2. **POP3/IMAP**：负责读取邮件。
- 3. **MIME**：扩展邮件内容类型，使邮件能包含非 ASCII 文本和附件。

电子邮件系统不是简单的“用户 A 直接把邮件发给用户 B”。实际中，用户通常通过用户代理与邮件服务器交互，再由邮件服务器之间完成转发。

### 6.4.1 电子邮件系统的组成结构

考点追踪：电子邮件的组成结构（2014）

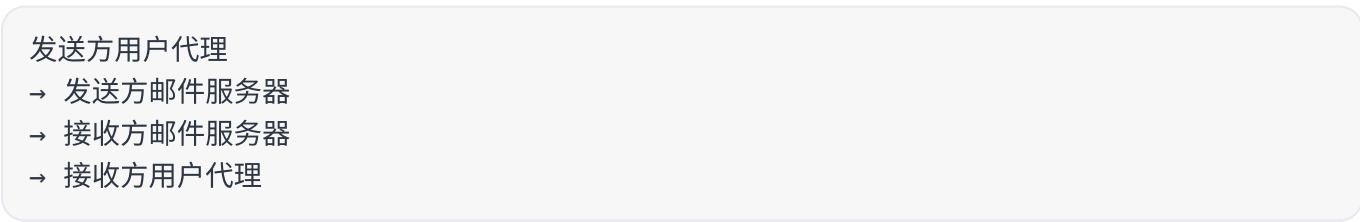
电子邮件系统主要由三部分组成：

| 组成部分    | 作用                               |
|---------|----------------------------------|
| 用户代理 UA | 用户收发、编辑、阅读邮件的软件，如邮件客户端或 Web 邮箱界面 |
| 邮件服务器   | 发送、接收、转发和保存邮件                    |
| 邮件协议    | 规定邮件发送和读取的规则，如 SMTP、POP3、IMAP    |

用户代理是用户直接使用的程序。它负责写信、显示邮件、管理邮件等，但通常不直接承担跨互联网的邮件转发任务。

邮件服务器是电子邮件系统的核心。每个用户通常在某个邮件服务器上拥有一个邮箱。发送方的邮件先提交到发送方邮件服务器，再通过互联网传送到接收方邮件服务器，最后存入接收方邮箱。

可以用下面的结构理解邮件传送：



其中：

- 用户代理到发送方邮件服务器、发送方邮件服务器到接收方邮件服务器，主要使用 SMTP。
- 接收方用户代理从接收方邮件服务器读取邮件，通常使用 POP3 或 IMAP。

电子邮件采用这种服务器中转结构，是因为接收方可能不在线。只要接收方邮件服务器在线，邮件就可以先被保存下来，等接收方之后再读取。

### 6.4.2 电子邮件格式与 MIME

电子邮件本身有固定格式，通常由**信封**和**内容**两部分组成。信封信息用于邮件传输过程，例如发件人、收件人等；邮件内容则是用户真正看到的部分。

邮件内容又可分为首部和主体：

首部  
空行  
主体

首部中常见字段包括：

| 首部字段     | 含义    |
|----------|-------|
| To       | 收件人地址 |
| Subject  | 邮件主题  |
| From     | 发件人地址 |
| Date     | 发送日期  |
| Reply-To | 回复地址  |

传统 SMTP 最初只能传送 7 位 ASCII 文本，无法直接表示中文、图片、音频、视频、附件等复杂内容。为了解决这个问题，引入了 MIME，即 Multipurpose Internet Mail Extensions，多用途互联网邮件扩展。

MIME 的作用不是替代 SMTP，而是**扩展邮件内容表示能力**。它通过在邮件首部增加字段，说明邮件主体的内容类型、编码方式和分段结构，使非 ASCII 内容可以被编码后通过 SMTP 传输。

MIME 主要解决三类问题：

- 1. 支持非 ASCII 文本，例如中文。
- 2. 支持多媒体内容，例如图片、音频、视频。
- 3. 支持附件和多部分邮件结构。

可以这样理解 SMTP 与 MIME 的关系：

SMTP：负责把邮件从一台邮件服务器推送到另一台邮件服务器  
MIME：负责把复杂邮件内容编码成 SMTP 可以传送的形式

因此，MIME 不负责邮件传输路径，它只是让邮件内容更加丰富。

### 6.4.3 SMTP 和 POP3

SMTP 即 Simple Mail Transfer Protocol，简单邮件传输协议。它是互联网电子邮件系统中最重要发送协议。

考点追踪：SMTP 协议的功能（2019）

SMTP 的基本特点如下：

| 项目   | 内容  |
|------|-----|
| 协议层次 | 应用层 |

| 项目    | 内容               |
|-------|------------------|
| 传输层协议 | TCP              |
| 熟知端口号 | 25               |
| 主要作用  | 发送邮件、邮件服务器之间传送邮件 |
| 工作方式  | 推送方式             |

SMTP 使用客户/服务器方式。发送方邮件服务器作为 SMTP 客户，接收方邮件服务器作为 SMTP 服务器。发送方主动把邮件“推”到接收方服务器，而不是接收方主动去“拉”。

SMTP 通信过程通常可以分为三个阶段：

### 第一阶段：连接建立

发送方 SMTP 客户主动与接收方 SMTP 服务器建立 TCP 连接，服务器使用端口 25。连接建立后，服务器返回服务就绪信息，通常以 220 开头。

随后客户发送 HELO 命令，表明自己的身份。若服务器同意，则返回 250 OK。

### 第二阶段：邮件传送

邮件传送从 MAIL FROM 命令开始，用于说明发件人地址：

```
MAIL FROM:<发件人地址>
```

若服务器准备接收邮件，返回 250 OK。

然后客户使用 RCPT TO 命令逐个指定收件人地址：

```
RCPT TO:<收件人地址>
```

每个 RCPT 命令都会触发服务器响应。若收件人地址有效并且服务器可接收，则可能返回 250 OK；若用户不存在，则可能返回 550 No such user here。

RCPT 命令的作用是**先验证收件人地址是否有效**。这样做可以避免在发送很长的邮件正文后才发现收件人不存在，从而减少网络资源浪费。

收到所有 RCPT 命令的肯定响应后，客户端发送 DATA 命令，表示即将传送邮件内容。服务器正常情况下会回复：

```
354 Start mail input; end with <CRLF>.<CRLF>
```

其中，CRLF 表示回车换行。随后客户端开始逐行发送邮件内容，并用下面的序列表示邮件正文结束：

```
<CRLF>.<CRLF>
```

也就是“单独一行只有一个点”。这是 SMTP 中识别邮件正文结束的重要标志。

### 第三阶段：连接释放

邮件发送完成后，SMTP 客户发送：

```
QUIT
```

SMTP 服务器返回 221，表示服务器同意释放 TCP 连接。至此，本次邮件传送过程结束。

可以把 SMTP 的主线压缩为：

```
建立 TCP 连接 → HELO 标识身份 → MAIL FROM 指定发件人 → RCPT TO 指定收件人 → DATA 发送正文 → QUIT 释放连接
```

复习时尤其要抓住三点：

- 1. SMTP 基于 TCP，端口号为 25。
- 2. SMTP 是发送协议，用“推”的方式传送邮件。
- 3. SMTP 原本只能直接传送 7 位 ASCII 文本，非 ASCII 内容需要借助 MIME 编码。

考点追踪：POP3 协议的功能与特点（2015，2018，2025）

### POP3 和 IMAP

POP 是 Post Office Protocol 的缩写，即邮局协议。当前常用版本是 POP3。它是一种结构简单、功能有限的邮件读取协议，用于让用户代理从接收端邮件服务器读取邮件。

POP3 同样采用客户/服务器模式，在传输层基于 TCP，使用端口号 110。

POP3 支持两种常见工作模式：

| 工作模式  | 含义                     | 结果         |
|-------|------------------------|------------|
| 下载并保留 | 用户从服务器读取邮件后，邮件仍保留在服务器上 | 以后仍可再次访问   |
| 下载并删除 | 邮件一旦被成功读取，就从服务器删除      | 服务器不再保存该邮件 |

POP3 的重点是“读取邮箱中的邮件”，而不是发送邮件。它的功能较简单，适合把邮件从服务器取到本地，但不擅长在服务器端进行复杂管理。

IMAP 是 Internet Message Access Protocol 的缩写，即互联网报文存取协议。与 POP3 相比，IMAP 功能更强，主要体现在：

- 1. 支持在服务器端创建文件夹。
- 2. 支持在不同文件夹之间移动邮件。
- 3. 支持对远程邮箱中的邮件进行搜索等在线操作。
- 4. 支持选择性获取邮件内容，例如只读取邮件首部，或只读取某封 MIME 邮件的一部分。

IMAP 服务器需要维护用户的会话状态信息，因此实现比 POP3 更复杂。但它更适合多设备同步和低带宽环境：用户可以先查看邮件标题，判断有用后再下载完整邮件正文或附件。

随着 WWW 的普及，出现了大量基于 Web 的电子邮件服务，如 Hotmail、Gmail 等。此类服务中，用户浏览器与邮件服务器之间的交互，包括发送和读取邮件，通常通过 HTTP 完成；只有在不



同邮件服务器之间传递邮件时，才使用 SMTP。

这一点很容易考：**SMTP 负责邮件服务器之间或用户代理到邮件服务器的发送过程；POP3/IMAP 负责用户从邮件服务器读取邮件；Web 邮件中浏览器与邮件服务器之间常使用 HTTP。**

## 6.5 万维网

### 6.5.1 万维网的概念与组成结构

万维网即 World Wide Web，简称 WWW。它是一个分布式、联机式的信息存储空间。在这个空间中，任何有用的事物都可以被称为资源，并且由统一资源定位符 URL 唯一标识。

用户访问 WWW 时，通常不需要关心资源到底存放在哪台具体主机上，只要点击超链接或输入 URL，就可以通过 HTTP 获取所需内容。万维网的便利性正来自“超链接”这种组织方式：一个网页可以指向另一个网页，使用户能够从一个站点跳转到另一个站点，按需获取丰富信息。

万维网中常见的三类基础标准如下：

考点追踪：HTTP 使用的传输层协议（2018）

| 标准   | 全称                                  | 作用                                            |
|------|-------------------------------------|-----------------------------------------------|
| URL  | Uniform Resource Locator，统一资源定位符    | 标识万维网上各种文档，保证每个文档在整个 WWW 范围内具有唯一标识            |
| HTTP | HyperText Transfer Protocol，超文本传输协议 | 应用层协议，基于 TCP 连接提供可靠数据传输，是浏览器与服务器之间交换网页内容的主要协议 |
| HTML | HyperText Markup Language，超文本标记语言   | 描述网页文档结构，通过标记组织文字、声音、图像、视频等内容并在浏览器中呈现         |

URL 可以看作互联网上资源位置和访问方式的简洁表示。它的一般形式为：

<协议>://<主机>:<端口>/<路径>

各部分含义如下：

| 部分 | 含义         | 例子或说明                  |
|----|------------|------------------------|
| 协议 | 获取资源所使用的协议 | http、https、ftp 等       |
| 主机 | 存放资源的主机地址  | 可以是域名，也可以是 IP 地址       |
| 端口 | 服务器进程监听的端口 | 某些情况下可省略，HTTP 默认端口为 80 |
| 路径 | 资源在服务器上的位置 | 某些情况下可省略，通常默认表示根目录     |

URL 中的端口和路径在某些情况下可以省略。例如使用 HTTP 访问网页时，如果没有显式写端口，就默认使用 80 号端口；如果没有写路径，通常表示访问服务器的根目录或默认首页。

万维网本质上由大量网络站点和网页组成，是互联网最主要的应用之一。它采用客户/服务器模式工作：

浏览器（万维网客户程序） → 发送请求 → 万维网服务器  
浏览器（万维网客户程序） ← 返回网页文档 ← 万维网服务器

运行在用户主机上的浏览器是万维网客户程序；存放网页文档并持续提供访问服务的主机运行服务器程序，称为万维网服务器。客户程序向服务器发起请求，服务器将用户请求的网页文档返回给客户端。

复习时不要把 WWW 与 Internet 完全等同。Internet 是底层互联网络和协议体系，WWW 是运行在 Internet 之上的一种重要应用。应用层学习 WWW，重点不是网页设计，而是理解：**资源用 URL 标识，网页用 HTML 描述，浏览器与服务器之间主要用 HTTP 交换文档，而 HTTP 基于 TCP。**

## 6.5.2 超文本传输协议

HTTP 即 HyperText Transfer Protocol，超文本传输协议。它规定了浏览器如何向万维网服务器请求文档，以及服务器如何把文档传送给浏览器。站在应用层角度看，HTTP 定义的是**浏览器与 Web 服务器之间请求和响应报文的格式及交互规则**。

HTTP 是一种**面向事务**的应用层协议。这里的“事务”可以理解为一次完整的“请求—响应”过程：浏览器请求某个资源，服务器返回相应资源或错误信息。网页中的文本、图片、声音、视频等对象，本质上都可以通过 HTTP 请求与响应来获取。

### HTTP 的操作过程

浏览器访问一个 Web 页面时，表面上只是用户点击链接或输入 URL，背后通常经历以下过程：

#### 1. 解析服务器域名

浏览器首先根据 URL 中的主机名，通过 DNS 查询得到服务器的 IP 地址。

#### 2. 建立 TCP 连接

浏览器向服务器的 HTTP 服务端口发起 TCP 连接请求。HTTP 默认端口号是 **80**。

#### 3. 发送 HTTP 请求报文

TCP 连接建立后，浏览器向服务器发送 HTTP 请求。若请求清华大学网站首页，典型请求方法可能是：

```
GET /index.htm
```

#### 4. 服务器返回 HTTP 响应报文

服务器收到请求后，定位并组织所需资源，把网页文件或状态信息封装在 HTTP 响应中返回给浏览器。

#### 5. 浏览器解析并显示页面

浏览器收到响应后，解析 HTML 文件，并把最终页面显示给用户。如果 HTML 中还引用了图片、脚本、样式表等其他对象，浏览器还需要继续发起新的 HTTP 请求获取这些对象。

#### 6. 释放或保持 TCP 连接

根据 HTTP 版本和连接方式不同，TCP 连接可能在传输结束后立即释放，也可能被保留以便后续继续复用。

可以把一次基本 Web 访问概括为：

用户输入 URL

- DNS 解析域名得到 IP 地址
- 浏览器与 Web 服务器建立 TCP 连接
- 浏览器发送 HTTP 请求
- 服务器返回 HTTP 响应
- 浏览器解析并显示页面
- 释放或复用 TCP 连接

考点追踪：Web 浏览涉及的核心协议栈（2014，2021）

需要注意，访问一个网页不只涉及 HTTP。完整通信过程中可能涉及应用层的 DHCP、DNS、HTTP，传输层的 UDP、TCP，网络层的 IP、ARP，以及数据链路层的以太网协议等。考研题常把“输入网址后发生了什么”作为综合题入口，本节重点仍然放在 HTTP 这一应用层协议本身。

### HTTP 的特点

HTTP 的第一个重要特点是：**HTTP 使用 TCP 作为传输层协议**。因此，HTTP 不需要自己处理数据丢失、乱序和重传问题，可靠传输由 TCP 提供。

考点追踪：HTTP 使用的传输层协议（2018）

但是，HTTP 本身并不等于“有连接协议”。HTTP 报文交换之前虽然要先建立 TCP 连接，但 HTTP 自身并不维护专门的 HTTP 连接状态。可以理解为：**TCP 是有连接的，HTTP 协议本身是无连接、无状态的应用层协议**。

HTTP 的第二个重要特点是：**HTTP 是无状态的**。服务器通常不会自动记住同一客户此前的请求历史。也就是说，同一个浏览器第二次访问某个页面时，服务器默认并不因为之前服务过它而自动改变响应。

这种无状态设计使服务器实现更简单，也更容易支持大量并发访问。但现实应用往往又需要识别用户状态，例如购物车、登录会话、个性化推荐等，因此常引入 Cookie 机制。

考点追踪：Cookie 的工作原理（2015）

Cookie 的基本过程如下：

1. 用户第一次访问某个启用 Cookie 的网站。
2. 服务器为该用户生成一个唯一的 Cookie 标识码，例如 12345，并在后台数据库中创建对应记录。
3. 服务器在 HTTP 响应报文中加入类似下面的首部字段：

```
Set-Cookie: 12345
```

4. 浏览器收到响应后，把该 Cookie 与服务器域名一起保存在本地。
5. 用户以后再次访问同一网站时，浏览器会在 HTTP 请求报文中自动带上 Cookie：

6. 服务器根据 Cookie 标识码查询后台记录，从而识别用户并提供个性化服务。

因此，Cookie 不是改变了 HTTP “无状态”的本质，而是在应用层通过额外字段和服务器端记录，给无状态协议补上了用户识别能力。

### HTTP 连接方式与页面加载时延

HTTP 连接方式是本节最容易出题的内容之一。HTTP/1.0 默认使用**非持续连接**，HTTP/1.1 默认使用**持续连接**。

非持续连接的特点是：每请求一个对象，就单独建立一次 TCP 连接，收到该对象后释放连接。若一个网页由一个 HTML 文档和多个图片对象组成，则浏览器需要为这些对象分别建立连接或并行建立多个连接。

在非持续连接下，请求一个 Web 文档所需时间可粗略理解为：

$$\text{总时间} = \text{文档传输时间} + 2\text{RTT}$$

其中，一个 RTT 用于完成 TCP 连接建立过程，另一个 RTT 用于发送 HTTP 请求并接收响应的开始部分。每个对象获取都需要承担这样的 RTT 开销，因此对象越多，额外时延越明显。现代浏览器通常会建立多个并行 TCP 连接，以同时请求多个对象，减少等待时间。

持续连接的特点是：服务器发送响应后仍保持 TCP 连接，使同一客户可以继续通过该连接发送后续 HTTP 请求并接收响应。浏览器得到基本 HTML 文档后，发现其中引用了图片等对象，就可以复用已有连接请求这些对象，避免为每个对象重新建立 TCP 连接。

HTTP/1.1 的持续连接又可以分为两种工作方式：

| 方式     | 工作特点                        | 时延影响                            |
|--------|-----------------------------|---------------------------------|
| 非流水线方式 | 客户端必须等前一个响应到达后，才能发送下一个请求    | 每个后续对象仍可能产生额外等待                 |
| 流水线方式  | 客户端可以连续发送多个对象请求，服务器也可连续返回响应 | 若请求和响应连续传输，获取多个引用对象可明显减少 RTT 开销 |

#### 考点追踪：HTTP/1.1 页面加载时延分析（2011，2022）

在流水线方式下，如果所有请求和响应都能连续传输，则获取全部引用对象可能只需约 **1RTT** 的额外等待，而不是每个对象都各自消耗一个 RTT。这里不能只机械套公式，还要注意 TCP 的发送窗口、拥塞控制和对象大小：即使 HTTP/1.1 支持持续连接，实际传输时间仍可能受到 TCP 初始拥塞窗口和数据量的影响。

易错点是：持续连接减少的是**重复建立 TCP 连接和等待响应的空闲时间**，并不意味着数据传输本身不占时间，也不意味着所有对象一定瞬间完成传送。

### HTTP 报文结构

HTTP 是面向文本的协议。HTTP 报文中的字段通常是 ASCII 字符串，字段长度不固定。HTTP 报文分为两类：

- 1. **请求报文**：从浏览器发送到服务器，用于请求资源。
- 2. **响应报文**：从服务器发送到浏览器，用于返回资源或状态信息。

HTTP 请求报文和响应报文的基本结构相似，都可以分为三部分：

| 部分   | 请求报文中的名称 | 响应报文中的名称 | 作用                     |
|------|----------|----------|------------------------|
| 开始行  | 请求行      | 状态行      | 表明请求方法、资源路径、协议版本，或响应状态 |
| 首部行  | 首部行      | 首部行      | 携带浏览器、服务器、报文主体等附加信息    |
| 实体主体 | 实体主体     | 实体主体     | 携带需要传送的数据内容，可为空        |

开始行和首部行均以回车换行符 `CRLF` 结束。首部行结束后，还需要一个空行，表示首部部分结束，然后才可能出现实体主体。

请求报文的一般形式可表示为：

请求方法 URL HTTP版本  
首部字段名： 字段值  
首部字段名： 字段值  
  
实体主体

响应报文的一般形式可表示为：

HTTP版本 状态码 短语  
首部字段名： 字段值  
首部字段名： 字段值  
  
实体主体

考点追踪：HTTP 常用请求方法及功能（2015）

HTTP 常见请求方法如下：

| 方法      | 基本功能            | 记忆要点                |
|---------|-----------------|---------------------|
| GET     | 请求读取由 URL 标识的信息 | 最常见，用于获取网页或对象       |
| HEAD    | 请求读取资源的首部信息     | 只要首部，不返回主体，便于检查资源属性 |
| POST    | 向服务器提交数据        | 常用于表单提交、上传内容        |
| CONNECT | 用于代理服务器场景       | 常见于建立隧道连接           |

在早期 HTTP/1.0 中，常见方法主要有 GET、HEAD、POST。HTTP/1.1 在此基础上扩展了 OPTIONS、PUT、DELETE、TRACE、CONNECT 等方法。408 重点一般不是背全部方法，而是区

分 GET、HEAD、POST 的基本作用。

HTTP 响应报文中的状态码用于说明服务器处理请求的结果，常见类别如下：

| 状态码范围 | 含义    | 例子                      |
|-------|-------|-------------------------|
| 1xx   | 通知信息  | 请求已收到，继续处理              |
| 2xx   | 成功    | 200 OK 表示请求成功           |
| 3xx   | 重定向   | 资源位置发生变化，需要进一步操作        |
| 4xx   | 客户端错误 | 404 Not Found 表示请求资源不存在 |
| 5xx   | 服务器错误 | 服务器在处理请求时出错             |

HTTP 请求报文示例与协议端口小结

资料中给出了一个用 Wireshark 捕获的 HTTP 请求报文示例。它可以从分层封装角度帮助理解：以太网帧中先有 MAC 首部，接着是 IP 数据报首部、TCP 报文段首部，最后的 TCP 数据部分才是应用层传下来的 HTTP 请求报文。

这说明应用层数据并不是独立在线路上裸传，而是逐层封装后传输。以 HTTP 请求为例，基本封装关系可理解为：

HTTP 请求报文

→ TCP 报文段

→ IP 数据报

→ 数据链路层帧

→ 物理网络传输

在示例中，HTTP 请求行可能类似：

```
GET /face/20.gif HTTP/1.1
```

其中：

| 字段           | 含义          |
|--------------|-------------|
| GET          | 请求方法，表示读取资源 |
| /face/20.gif | 请求的资源路径     |
| HTTP/1.1     | 使用的 HTTP 版本 |

常见应用层协议及其传输层协议、熟知端口号可以归纳如下：

| 应用协议     | 使用的传输层协议 | 熟知端口号 |
|----------|----------|-------|
| FTP 数据连接 | TCP      | 20    |
| FTP 控制连接 | TCP      | 21    |

| 应用协议   | 使用的传输层协议 | 熟知端口号 |
|--------|----------|-------|
| TELNET | TCP      | 23    |
| SMTP   | TCP      | 25    |
| DNS    | UDP      | 53    |
| TFTP   | UDP      | 69    |
| HTTP   | TCP      | 80    |
| POP3   | TCP      | 110   |
| SNMP   | UDP      | 161   |

这张表非常适合选择题速记。复习时尤其要把容易混淆的端口放在一起：FTP 控制连接 21、FTP 数据连接 20；SMTP 25、POP3 110；DNS 53；HTTP 80。

## 6.6 本章小结及疑难点

### 6.6.1 客户进程端口号与服务器进程端口号

服务器进程通常使用**熟知端口号**。这些端口号固定、公开，便于客户进程找到对应服务。例如 HTTP 默认使用 80 端口，DNS 使用 53 端口，SMTP 使用 25 端口。

客户进程通常使用**临时端口号**。临时端口号由操作系统在连接发起时自动分配，只在本次通信中有效。客户向服务器发起连接时，会连接到服务器的熟知端口，并把自己的临时端口号告诉服务器。这样，通信双方就可以通过一对端口号建立唯一的端到端联系：

服务器端：熟知端口号  
客户端：临时端口号

例如浏览器访问 Web 服务器时，浏览器使用临时端口，服务器使用 80 端口。TCP 连接建立后，服务器返回数据时就能根据客户的 IP 地址和临时端口号，把响应准确送回对应浏览器进程。

### 6.6.2 互联网与万维网的区别

互联网 Internet 是全球范围的计算机网络互联系统，采用 TCP/IP 协议族作为通信基础，提供主机之间的连通性。它更像是底层的“路”。

万维网 WWW 是构建在互联网之上的一种应用层系统，由互相链接的网页和网站组成，用户通过浏览器并借助 HTTP 协议访问网页资源。它更像是在互联网这条“路”上运行的一种“车”。

二者关系可以概括为：

Internet：底层互联网络和协议基础  
WWW：运行在 Internet 之上的应用层服务

因此，不能把互联网和万维网完全等同。WWW 是互联网中最重要、最常用的应用之一，但互联网还支持电子邮件、FTP、DNS、远程登录等许多其他应用。



### 6.6.3 HTTP/1.1 持续连接为何仍可能需要更多 RTT

HTTP/1.1 默认支持持续连接，但持续连接并不意味着一个页面及其所有图片都能在一个 RTT 内全部传完。原因在于 HTTP 运行在 TCP 之上，而 TCP 的传输过程受到发送窗口、接收窗口和拥塞控制影响。

例如 TCP 连接刚建立时，初始拥塞窗口可能较小。如果网页文件和图片对象总大小超过当前可发送窗口，服务器就不能一次性把所有数据发完，而要等待 ACK 返回后才能继续扩大窗口并发送后续数据。

资料中的分析思路可以概括为：

1. 第一个 RTT：完成 TCP 连接建立的前两次握手。
2. 第二个 RTT：第三次握手捎带 HTML 请求，服务器发送一部分 HTML 数据。
3. 后续 RTT：客户端确认收到的数据，并继续请求或接收图片等对象；TCP 拥塞窗口逐步增大。
4. 当对象较大、数量较多或窗口较小时，即使链路带宽足够，也可能至少需要多个 RTT 才能完成传输。

所以做题时不能只看“HTTP/1.1 支持持续连接”，还要看题目是否考虑 TCP 初始拥塞窗口、对象大小和确认过程。若题目明确忽略传输层拥塞控制，则可以用应用层交互次数估算；若题目强调 TCP 窗口，则必须把窗口增长也考虑进去。

### 6.6.4 主机刚开机后访问 Web 站点的完整通信过程

若一台以太网主机刚开机，尚未配置网络参数，并通过有线方式接入局域网，然后访问某个 Web 站点，完整过程大致如下。

#### 第一步：DHCP 获取网络配置。

主机启动后，先通过 DHCP 自动获取 IP 地址、子网掩码、默认网关和 DNS 服务器地址等信息。DHCP 属于应用层协议，通常使用 UDP；UDP 数据报封装在 IP 数据报中，最终再封装成以太网帧发送。

#### 第二步：DNS 解析域名。

用户在浏览器地址栏输入域名后，浏览器需要通过 DNS 查询目标服务器的 IP 地址。DNS 属于应用层协议，查询通常使用 UDP。若 DNS 服务器与主机在同一子网内，主机需要通过 ARP 获取 DNS 服务器的 MAC 地址；若 DNS 服务器在其他网络中，主机需要通过 ARP 获取默认网关的 MAC 地址，再由默认网关转发。

#### 第三步：HTTP 获取网页。

获得目标 Web 服务器的 IP 地址后，浏览器向服务器 80 端口发起 TCP 连接。连接建立后，浏览器发送 HTTP GET /index.html 请求，服务器返回包含网页内容的 HTTP 响应。浏览器解析 HTML 文件，如果页面还引用图片、视频、脚本等其他资源，就继续发起 HTTP 请求获取这些对象。

#### 第四步：TCP 连接释放与页面呈现。

相关资源传输完成后，TCP 连接根据连接方式被释放或继续保持。浏览器解析 HTML 与相关资源，最终完成网页展示。

这个综合过程可以压缩记忆为：

DHCP 获取本机网络参数  
→ DNS 把域名解析为 IP 地址

- ARP 获得下一跳 MAC 地址
- TCP 建立到 Web 服务器 80 端口的连接
- HTTP 请求和响应传输网页资源
- 浏览器解析并显示页面

这类题的关键是不要只写 HTTP。刚开机意味着网络参数尚未配置，所以要先想到 DHCP；访问域名意味着要进行 DNS 解析；跨网络通信要想到默认网关和 ARP；真正获取网页时才进入 TCP 与 HTTP 阶段。